

UNIWERSYTET MARII CURIE-SKŁODOWSKIEJ
W LUBLINIE
Wydział Matematyki, Fizyki i Informatyki

Kierunek: **Informatyka**

Nikita Lysiuk
nr albumu: 320828

Symulacja i wizualizacja cieczy w czasie rzeczywistym za pomocą Rust i Vulkan API

Real-Time Fluid Simulation and Visualization
using Rust and the Vulkan API

Praca licencjacka
napisana w Katedrze Oprogramowania Systemów Informatycznych
Instytutu Informatyki i Matematyki UMCS
pod kierunkiem **dr hab. Beaty Byliny**

Lublin 2026

Spis treści

Wstęp **7**

1 Metodyka i podstawy teoretyczne symulacji i wizualizacji cieczy **9**

1.1 Porównanie podejścia lagrange'owskiego i eulerowskiego 9

1.1.1 Istota podejścia eulerowskiego 10

1.1.2 Istota podejścia lagrange'owskiego 11

1.2 Sformułowanie matematyczne metody cząsteczkowej SPH i jej wariantu DFSPH 12

1.2.1 Równania Naviera-Stokesa jako fundament teoretyczny 12

1.2.2 Interpolacja jądrowa i dyskretyzacja SPH 14

1.2.3 Algorytm Divergence-Free SPH (DFSPH) 15

1.3 Wizualizacja objętościowa metodą Raymarching 17

2 Przegląd technologii i środowisko programistyczne **21**

2.1 Istniejące rozwiązania symulacji cieczy 21

2.1.1 Symulacja FLIP w programie Blender 22

2.1.2 Symulacja FLIP w silniku Unreal Engine 5 22

2.1.3 Biblioteka SPlisHSPlasH 23

2.1.4 Podsumowanie rozwiązań 24

2.2 Język programowania Rust 25

2.3 API graficzne Vulkan 27

2.3.1 Instancja, urządzenie fizyczne i logiczne 28

2.3.2 Kolejki i pule poleceń 29

2.3.3 Łańcuch wymiany i dynamiczne renderowanie 30

2.3.4 Zasoby: bufory, obrazy i deskryptory 31

2.3.5 Potok obliczeniowy i graficzny 33

2.4 Biblioteka Vulkano 37

2.5 Narzędzia programistyczne 38

2.5.1 Tracy Profiler 39

2.5.2 RenderDoc 39

2.6 Wybór funkcji wygładzającej 40
2.6.1 Sześcienna funkcja sklejana 40
2.6.2 Funkcja Wendlanda C2 41
2.6.3 Dobór jądra dla solvera DFSPH 42
2.7 Wyszukiwanie sąsiedztwa cząstek 43
2.7.1 Siatka haszująca według Simona Greena 44
2.7.2 Skompresowane listy sąsiedztwa jako alternatywa 44
2.8 Modelowanie zjawisk fizycznych 46
2.8.1 Lepkość 46
2.8.2 Napięcie powierzchniowe 47
2.8.3 Warunki brzegowe 47
2.9 Potok wizualizacji 48
2.9.1 Rekonstrukcja pola gęstości 48
2.9.2 Wyznaczanie izopowierzchni metodą Raymarching 49
2.9.3 Model optyczny cieczy 50

3 Implementacja i oryginalne rozwiązania techniczne 53
3.1 Założenia projektowe 54
3.2 Prezentacja aplikacji 55
3.3 Architektura systemu 56
3.4 Warstwa aplikacji 58
3.4.1 Zarządzanie oknem 59
3.4.2 Kamera 59
3.4.3 Scena i parametry symulacji 61
3.5 Inicjalizacja GPU z Vulkano 63
3.5.1 Kontekst Vulkan 63
3.5.2 Wzorzec ComputeStep 64
3.5.3 Deskryptory i synchronizacja 65
3.6 Struktury danych cząstek 67
3.6.1 Organizacja danych: AOS i SOA 67
3.6.2 Bufory cząstek na GPU 68
3.7 Wyszukiwanie sąsiadów 69
3.7.1 Haszowanie przestrzenne 69
3.7.2 Potok wyszukiwania 70
3.8 Sortowanie na GPU 71
3.8.1 Sortowanie bitoniczne 71
3.8.2 Sortowanie pozycyjne 72
3.8.3 Porównanie algorytmów 74

3.9 Solver DFSPH 78
3.9.1 Pętla podkroków 80
3.9.2 Obliczanie gęstości 81
3.9.3 Korekcja gęstości 82
3.9.4 Korekcja dywergencji 83
3.9.5 Warunek CFL i dobór Δt 84
3.10 Siły fizyczne 87
3.10.1 Lepkość XSPH 87
3.10.2 Obsługa kolizji 87
3.11 Wizualizacja 88
3.11.1 Splatting gęstości do tekstury 3D 88
3.11.2 Raymarching 89
3.11.3 Model oświetlenia 92
3.11.4 Środowisko HDRI 94

Podsumowanie **97**

Spis rysunków **101**

Bibliografia **105**

Wstęp

Realistyczne odwzorowanie zachowania cieczy od lat pozostaje jednym z najbardziej wymagających zadań grafiki komputerowej. Woda towarzyszy człowiekowi na co dzień, dlatego widz natychmiast dostrzega każde uproszczenie: nienaturalny rozprysk, brak załamania światła czy zbyt gładką powierzchnię fali. Jednocześnie zapotrzebowanie na wiarygodne symulacje płynów stale rośnie — od produkcji filmowej i gier komputerowych, przez wizualizacje naukowe, po inżynierskie analizy przepływów. W produkcji offline, gdzie pojedyncza klatka może powstawać godzinami, jakość symulacji ogranicza wyłącznie budżet obliczeniowy; w zastosowaniach interaktywnych ograniczenie jest znacznie bardziej rygorystyczne, ponieważ pełny krok fizyki wraz z renderowaniem musi zmieścić się w kilkunastu milisekundach.

Spełnienie tego wymagania stało się możliwe dzięki ewolucji procesorów graficznych, które z wyspecjalizowanych układów rasteryzujących przekształciły się w masowo równoległe maszyny obliczeniowe ogólnego przeznaczenia. Współczesne GPU wykonuje jednocześnie dziesiątki tysięcy wątków, co naturalnie odpowiada strukturze symulacji cząsteczkowych, w których ten sam zestaw operacji stosowany jest niezależnie do każdego elementu cieczy. Pełne wykorzystanie tego potencjału wymaga jednak narzędzi dających programiście bezpośrednią kontrolę nad sprzętem: niskopoziomowych API graficznych oraz języków systemowych łączących wydajność z gwarancjami poprawności. Tymczasem dojrzałe systemy symulacji cieczy — zarówno pakiety animacyjne, jak i biblioteki badawcze — wykonują centralne obliczenia w przeważającej mierze na procesorze głównym, a obecność karty graficznej wykorzystują głównie do wyświetlania wyników.

Celem niniejszej pracy jest zaprojektowanie i implementacja interaktywnego silnika symulacji cieczy, w którym kompletny potok obliczeniowy — od dynamiki cząstek, przez wymuszanie nieściśliwości, po wolumetryczną wizualizację — wykonywany jest w czasie rzeczywistym w całości na procesorze graficznym. Fundament fizyczny systemu stanowi cząsteczkowa metoda SPH w wariancie DFSPH, wymuszającym nieściśliwość cieczy przez dwuetapową korekcję gęstości i dywergencji pola prędkości. Warstwę technologiczną tworzą język programowania Rust oraz niskopoziomowe API graficzne Vulkan — połączenie oferujące statyczne bezpieczeństwo pamięci przy zachowa-

niu jawnej kontroli nad zasobami sprzętowymi. Zakres pracy obejmuje sformułowanie podstaw teoretycznych metody SPH i algorytmu DFSPH, uzasadniony dobór stosu technologicznego, implementację solvera oraz potoku wizualizacji wraz z autorskimi rozwiązaniami architektonicznymi, a także ilościową analizę wydajności i zbieżności zaimplementowanych algorytmów.

Praca składa się z trzech rozdziałów. W rozdziale pierwszym przedstawiono podstawy teoretyczne: porównanie eulerowskiego i lagrange'owskiego opisu ruchu płynów, równania Naviera-Stokesa, dyskretyzację jądrową metody SPH wraz z algorytmem DFSPH oraz koncepcję wolumetrycznej wizualizacji metodą przemarszu promienia. Rozdział drugi zawiera przegląd istniejących rozwiązań symulacji cieczy oraz charakterystykę wybranych technologii: języka Rust, API Vulkan i biblioteki Vulkano, narzędzi diagnostycznych, a ponadto analizę porównawczą funkcji wygładzających, struktur wyszukiwania sąsiedztwa i modeli zjawisk fizycznych. Rozdział trzeci dokumentuje implementację systemu i prezentuje: architekturę aplikacji, organizację danych w pamięci GPU, potok wyszukiwania sąsiadów wraz z porównaniem dwóch algorytmów sortowania, kolejne etapy solvera DFSPH oraz potok renderowania — każdorazowo z uzasadnieniem podjętych decyzji i pomiarami wydajności. Pracę zamyka podsumowanie zestawiające osiągnięte rezultaty oraz kierunki dalszego rozwoju systemu.

Rozdział 1

Metodyka i podstawy teoretyczne symulacji i wizualizacji cieczy

Ciecze stanowią jedno z najbardziej fascynujących, a zarazem złożonych zjawisk fizycznych, których cyfrowa rekonstrukcja od dziesięcioleci pozostaje wyzwaniem na styku informatyki i fizyki obliczeniowej. Historyczne korzenie stosowanych dziś metod sięgają XVIII-wiecznych prac Eulera i Lagrange'a, którzy niezależnie wypracowali dwa odmienne sposoby matematycznego opisu ruchu materii.

1.1 Porównanie podejścia lagrange'owskiego i eulerowskiego

Numeryczna symulacja cieczy wymaga w pierwszej kolejności wyboru paradygmatu opisu jej ruchu. Oba dominujące podejścia — eulerowskie i lagrange'owskie — stanowią numeryczne przybliżenia rozwiązań równań Naviera-Stokesa: układu równań różniczkowych opisującego ewolucję gęstości i pędu lepkiej cieczy w czasie i przestrzeni. Pierwsze udane próby ich numerycznego rozwiązania dla cieczy o swobodnej powierzchni datuje się na rok 1965, kiedy Francis Harlow i James Eddie Welch z Los Alamos National Laboratory zaproponowali eulerowską metodę znaczników i komórek (ang. *marker and cell*, MAC) [24], opartą na analizie parametrów fizycznych w ustalonych punktach przestrzeni. Chociaż algorytmy siatkowe ewoluowały i są z sukcesami stosowane do dziś, charakteryzują się istotnymi ograniczeniami w obliczu gwałtownych zjawisk — silnych rozprysków czy szybkich zmian topologii powierzchni cieczy — co stało się motywacją do opracowania podejścia lagrange'owskiego.

W odpowiedzi na te trudności, w 1977 roku środowisko astrofizyków zaproponowało alternatywną koncepcję — wygładzoną hydrodynamikę cząstek (ang. *smoothed particle hydrodynamics*, SPH) [19]. Metoda ta czerpie bezpośrednią inspirację z la-

grange'owskiego opisu materii, traktując ciecz jako zbiór dyskretnych elementów masy swobodnie przemieszczających się w przestrzeni. Od czasu publikacji prac Gingolda i Monaghana, technika SPH doczekała się licznych wariantów optymalizacyjnych, stając się jednym z najchętniej wybieranych narzędzi w grafice komputerowej czasu rzeczywistego.

Analizując oba podejścia, należy zauważyć, że każde z nich oferuje unikalny zestaw zalet i wad. Metody siatkowe gwarantują wysoką stabilność numeryczną i precyzję obliczeń gradientów ciśnienia. Z kolei metody cząsteczkowe cechują się naturalnym zachowaniem masy oraz brakiem ograniczeń co do rozmiaru domeny obliczeniowej. Współczesne dążenie do maksymalizacji realizmu przy jednoczesnym zachowaniu wydajności doprowadziło do powstania metod hybrydowych, takich jak FLIP (ang. *fluid-implicit particle*) [9]. Stały się one de facto standardem w przemyśle filmowym oraz mediach interaktywnych, łącząc stabilność siatki eulerowskiej z elastycznością cząstek lagrange'owskich. Rozwiązania te, choć niezwykle skuteczne, wykraczają poza zakres niniejszej pracy, która koncentruje się na czystym sformułowaniu cząsteczkowym jako rozwiązaniu najlepiej wykorzystującym architekturę nowoczesnych procesorów graficznych.

1.1.1 Istota podejścia eulerowskiego

Podejście eulerowskie, sformułowane przez Leonharda Eulera w połowie XVIII wieku [16, 17], stanowi jeden z dwóch głównych sposobów opisu ruchu materii w mechanice kontinuum. W przeciwieństwie do śledzenia pojedynczych cząstek, koncepcja ta skupia się na stałych punktach w przestrzeni, przez które przepływa płyn. Zamiast pytać „gdzie przemieścił się dany element cieczy?", analizujemy pytanie „jaka jest prędkość i ciśnienie w tym konkretnym miejscu w danym momencie?" [10]. Matematycznie podejście to opiera się na analizie pól fizycznych. Zmiana dowolnej właściwości płynu Q (np. gęstości lub prędkości) jest opisywana przez tzw. pochodną materiałową, która pozwala połączyć obserwację w stałym punkcie z faktem, że płyn jest w ruchu:

$$\frac{DQ}{Dt} = \frac{\partial Q}{\partial t} + \mathbf{u} \cdot \nabla Q. \tag{1.1}$$

Wzór ten pokazuje, że całkowita zmiana wartości wynika z dwóch czynników: lokalnej zmiany w czasie ($\frac{\partial Q}{\partial t}$) oraz efektu transportu tej wartości przez poruszający się płyn ($\mathbf{u} \cdot \nabla Q$), gdzie $\mathbf{u}$ oznacza pole prędkości cieczy, a ∇ — operator gradientu. Współczesne techniki komputerowe, opierając się na tych założeniach, wykorzystują metody siatkowe. Jak wspomniano wcześniej, pionierami tego rozwiązania byli Francis Harlow i James Eddie Welch, którzy w 1965 roku wprowadzili metodę MAC [24]. Ich kluczowym pomysłem było nałożenie na przestrzeń nieruchomej, regularnej siatki

obliczeniowej. W takim modelu parametry płynu są obliczane tylko w węzłach lub na ściankach komórek tej siatki, co pozwala na numeryczne rozwiązanie skomplikowanych równań ruchu. Podejście oparte na siatce charakteryzuje się wysoką precyzją matematyczną i stabilnością obliczeń. Stała struktura siatki ułatwia wyznaczanie gradientów ciśnienia, co jest kluczowe dla zachowania realizmu w symulacjach płynów o dużej objętości (np. dymu czy spokojnej wody). Jego największą słabością jest tzw. dyfuzja numeryczna, czyli nienaturalne rozmywanie się detali płynu podczas przemieszczania się między komórkami. Ponadto siatki mają trudności z dokładnym śledzeniem swobodnej powierzchni cieczy, co wymaga stosowania dodatkowych technik, takich jak *metoda poziomic* (ang. *Level-Set*) lub *metoda śledzenia objętości* (ang. *Volume of Fluid*), które przy gwałtownych ruchach i rozpryskach tracą drobne szczegóły, takie jak pojedyncze krople.

Podsumowując, podejście eulerowskie jest niezwykle skuteczne tam, gdzie płyn wypełnia całą dostępną przestrzeń, jednak staje się mniej wydajne w scenariuszach wymagających symulacji dynamicznych rozprysków i szybkich zmian kształtu cieczy.

1.1.2 Istota podejścia lagrange'owskiego

Podejście lagrange'owskie, sformułowane przez Josepha Louisa de Lagrange'a w jego fundamentalnych pracach nad teorią ruchu płynów [31], oferuje zgoła odmienną perspektywę obserwacji materii niż model eulerowski. Zamiast analizować zmiany parametrów w stałych punktach przestrzeni, badacz podąża za konkretnym elementem masy wzdłuż jego trajektorii. W tym ujęciu współrzędne każdego punktu cieczy są funkcjami czasu oraz ich pozycji początkowej, co pozwala na bezpośrednie śledzenie historii ruchu poszczególnych fragmentów materii.

Matematycznie ruch w opisie lagrange'owskim wyraża się poprzez funkcję mapowania $\mathbf{x} = \chi(\mathbf{X}, t)$ [4], gdzie $\mathbf{X}$ to pozycja referencyjna cząstki w chwili początkowej, a $\mathbf{x}$ to jej aktualne położenie. Kluczową cechą tej metody jest naturalne zachowanie masy oraz uproszczenie równań adwekcji, które w tym modelu stają się tożsame z ruchem samej cząstki. Pozwala to na uniknięcie problemu dyfuzji numerycznej, będącej największą słabością metod siatkowych, oraz umożliwia modelowanie zjawisk o nieskończonym zasięgu bez konieczności definiowania sztywnej domeny obliczeniowej.

Mimo solidnych fundamentów matematycznych, praktyczne zastosowanie tego podejścia do złożonych symulacji cieczy pojawiło się dopiero w 1977 roku wraz z metodą SPH [19]. Motywacja do jej stworzenia płynęła z potrzeb astrofizyki — badacze tacy jak Gingold i Monaghan poszukiwali sposobu na modelowanie dynamicznych układów gwiazdowych i procesów formowania się galaktyk, gdzie materia ulega ekstremalnym deformacjom i przemieszczeniom w próżni. Tradycyjne metody oparte na siatce okazy-

wały się całkowicie niewydolne w symulowaniu obiektów, które nie posiadają stałych granic i swobodnie ewoluują w otwartej przestrzeni.

Wprowadzenie SPH jako alternatywy dla metod siatkowych było zatem podyktowane koniecznością obsłużenia scenariuszy, w których topologia płynu zmienia się w sposób gwałtowny i nieprzewidywalny. Największym atutem tego podejścia jest brak konieczności stosowania algorytmów śledzenia powierzchni. Ciecz jest definiowana przez samą obecność cząstek, co sprawia, że tworzenie się kropel, łączenie fal czy rozpryski są efektem ubocznym fizyki cząsteczkowej, a nie dodatkowym zadaniem geometrycznym. Metoda wprowadza jednak nowe wyzwania obliczeniowe, z których najistotniejszym jest brak stałych relacji między elementami. W każdym kroku symulacji konieczne jest ponowne wyznaczanie sąsiedztwa cząstek, co wymaga zaawansowanych struktur danych. Ponadto uzyskanie stabilności ciśnienia jest w tym modelu znacznie trudniejsze niż na regularnej siatce, co doprowadziło do powstania złożonych wariantów algorytmu — takich jak bezdywergencyjna hydrodynamika cząstek (ang. *divergence-free SPH*, DFSPH) [6], wymuszająca nieściśliwość poprzez zerowanie dywergencji pola prędkości, czy ciecz oparta na pozycjach (ang. *position based fluids*, PBF), stosująca więzy geometryczne zamiast równań ciśnienia.

Podsumowując, podejście lagrange'owskie przekształca problem dynamiki cieczy z analizy pól w analizę oddziaływań między elementami dyskretnymi. Jest to koncepcja, która mimo większego skomplikowania w zakresie optymalizacji struktur danych, oferuje niezrównaną elastyczność w wizualizacji dynamicznych i silnie rozproszonych zjawisk wodnych.

1.2 Sformułowanie matematyczne metody cząsteczkowej SPH i jej wariantu DFSPH

W celu przejścia od modelu ciągłego do postaci obliczeniowej konieczne jest sformułowanie równań Naviera–Stokesa w sposób umożliwiający ich dyskretyzację w ramach metody cząsteczkowej.

1.2.1 Równania Naviera-Stokesa jako fundament teoretyczny

Zarówno metody siatkowe, jak i bezsiatkowe (takie jak SPH), stanowią numeryczne przybliżenia rozwiązań układu różniczkowych równań cząstkowych, nazwanego na cześć Claude'a-Louisa Naviera oraz Sir George'a Gabriela Stokesa. Prace nad tymi równaniami, prowadzone w latach 1822–1850, zdefiniowały sposób, w jaki współczesna fizyka opisuje ruch płynów lepkich [4]. Każda analiza z zakresu symulacji cieczy

musi opierać się na tym fundamencie, który de facto stanowi sformułowanie podstawowych praw zachowania masy i pędu w mechanice kontinuum [36].

Pełny opis dynamiki cieczy wymaga spełnienia dwóch kluczowych warunków:

1. Zachowanie masy (Równanie ciągłości): Równanie to opisuje relację między gęstością cieczy ρ a jej przepływem, opisanym polem prędkości $\mathbf{u}$. W formie lagrange'owskiej przyjmuje ono postać:

$$\frac{D\rho}{Dt} = -\rho \nabla \cdot \mathbf{u}. \tag{1.2}$$

   Informuje ono, że zmiana gęstości elementu masy jest bezpośrednio związana z dywergencją pola prędkości ($\nabla \cdot \mathbf{u}$). W przypadku cieczy nieściśliwych, gęstość powinna pozostać stała ($\frac{D\rho}{Dt} = 0$), co wymusza bezdywergencyjny charakter pola prędkości.

2. Zachowanie pędu (Równanie Cauchy'ego): Opisuje zachowanie ośrodka ciągłego pod wpływem działających na niego sił [32]:

$$\rho \frac{D\mathbf{u}}{Dt} = -\nabla p + \nabla \cdot \tau + \rho \mathbf{a}, \tag{1.3}$$

   gdzie poszczególne symbole oznaczają:

   - $\frac{D}{Dt}$ — pochodna materiałowa (operator $\frac{\partial}{\partial t} + \mathbf{u} \cdot \nabla$), łącząca zmianę lokalną z efektem adwekcji,
   - ρ — gęstość masy cieczy,
   - $\mathbf{u}$ — wektor prędkości przepływu,
   - p — ciśnienie izotropowe (składowa determinująca siłę ciśnienia),
   - τ — tensor naprężeń dewiacyjnych (odpowiedzialny za opór lepki),
   - $\mathbf{a}$ — przyspieszenia masowe (np. grawitacja $\mathbf{g}$).

W kontekście niniejszej pracy, równania te dostarczają ramy analitycznej dla algorytmu SPH. Pokazują one, że ruch cieczy jest wynikiem bilansu sił: wewnętrznego gradientu ciśnienia (powstającego w odpowiedzi na zmiany gęstości), oporu wynikającego z lepkości oraz oddziaływań zewnętrznych. To właśnie sprzężenie między równaniem ciągłości a równaniem pędu pozwala wyznaczyć ciśnienie p, które koryguje prędkości cząstek tak, aby zachować stałą objętość cieczy. Zadaniem metody SPH jest przekształcenie tych ciągłych zależności w dyskretne interakcje między punktami masy.

1.2.2 Interpolacja jądrowa i dyskretyzacja SPH

Istota metody SPH polega na śledzeniu właściwości fizycznych cieczy poprzez analizę dyskretnych punktów masy. Fundamentem tej techniki jest matematyczna koncepcja dystrybucji Diraca δ oraz powiązana z nią tożsamość Diraca. W ujęciu idealnym, cząstkę można traktować jako nieskończenie mały punkt, w którym cała siła skupiona jest w jednym miejscu ($r = 0$, gdzie r oznacza odległość między cząstkami), a poza nim wynosi zero:

$$\delta(r) = \begin{cases} \infty & \text{jeśli } r = 0, \\ 0 & \text{w przeciwnym razie.} \end{cases} . \tag{1.4}$$

Dodatkowo, musi zostać spełniony warunek normalizacji, oznaczający, że niezależnie od wybranego obszaru całkowania, masa całkowita pozostaje jednostkowa:

$$\int \delta(r)\,\mathrm{d}v = 1. \tag{1.5}$$

W rzeczywistości komputerowej bezpośrednia implementacja nieskończenie małych punktów nie jest możliwa ze względu na ograniczoną precyzję maszynową oraz konieczność operowania na skończonych zbiorach danych. Niezbędny jest zatem sposób na dyskretyzację tych założeń. Rozwiązaniem jest wprowadzenie funkcji jądra (lub funkcji wygładzającej) W. Jej rolą jest „rozmycie" właściwości cząstki z punktu do obszaru o określonym promieniu oddziaływania h. Gradient tej funkcji opisuje nie tylko siłę wpływu sąsiednich cząstek, ale także kierunek zmian tych oddziaływań [30].

Dzięki temu dowolną właściwość fizyczną $A(\mathbf{x})$ możemy przybliżyć poprzez splot z funkcją jądra:

$$A(\mathbf{x}) \approx (A * W)(\mathbf{x}) = \int A(\mathbf{x}')W(\mathbf{x} - \mathbf{x}', h)\,\mathrm{d}\mathbf{x}'. \tag{1.6}$$

Ponieważ system komputerowy operuje na skończonej pamięci, nie możemy rozważać całki po bezgranicznej przestrzeni. W procesie dyskretyzacji całka zostaje zastąpiona sumą po skończonej liczbie sąsiadujących cząstek [38]:

$$\langle A(\mathbf{x}_i)\rangle \approx \sum_j \frac{m_j}{\rho_j} A(\mathbf{x}_j)W(\mathbf{x}_i - \mathbf{x}_j, h). \tag{1.7}$$

Sumowanie przebiega po sąsiednich cząstkach j, gdzie m_j oraz ρ_j oznaczają odpowiednio masę i gęstość j-tej cząstki, $\mathbf{x}_j$ — jej położenie, a iloraz m_j/ρ_j pełni rolę przybliżonej objętości tej cząstki.

Aby funkcja jądra była użyteczna w stabilnej symulacji cieczy, musi spełniać szereg rygorystycznych warunków matematycznych [30]:

- **Warunek normalizacji:** $\int W(r', h)\,\mathrm{d}v' = 1$, co gwarantuje zachowanie całkowitej wartości przybliżanej wielkości.

- **Warunek Diraca:** $\lim_{h\to 0} W(r,h) = \delta(r)$, co oznacza, że wraz ze zmniejszaniem promienia h, przybliżenie dąży do rozwiązania idealnego.
- **Dodatniość i symetria:** $W(r,h) \geqslant 0$ oraz $W(r,h) = W(-r,h)$.
- **Zwarta podpora:** $W(r,h) = 0$ dla $r \geqslant h$. Jest to kluczowy warunek z punktu widzenia wydajności — komputer musi analizować tylko te cząstki, które znajdują się w fizycznym zasięgu oddziaływania, co drastycznie redukuje liczbę operacji.

Praktycznym przykładem zastosowania tej teorii jest obliczanie gęstości ρ_i dla wybranej cząstki. Stosując powyższą dyskretyzację, otrzymujemy sumę mas sąsiadów ważoną funkcją jądra:

$$\rho_i = \sum_j m_j W(\mathbf{x}_i - \mathbf{x}_j, h). \tag{1.8}$$

W tym przypadku gęstość w mianowniku wzoru ogólnego (1.7) redukuje się, pozostawiając prostą zależność od masy, co pozwala na efektywne obliczanie lokalnego zagęszczenia cieczy przy skończonych zasobach obliczeniowych.

1.2.3 Algorytm Divergence-Free SPH (DFSPH)

Mimo że klasyczne sformułowanie SPH pozwala na wizualnie atrakcyjne symulacje, boryka się ono z problemem wysokiej ściśliwości cieczy oraz niestabilnością przy większych krokach czasowych. Tradycyjne rozwiązania, oparte na równaniach stanu (ang. *Equation of State*, EOS) — wzorach wyznaczających ciśnienie bezpośrednio z lokalnej gęstości cieczy — wymagają bardzo małych wartości Δt, aby uniknąć „zapadania się" objętości cieczy. Rozwiązaniem tych problemów jest algorytm DFSPH, zaproponowany przez Jana Bendera i Dana Koschiera [6].

Istota DFSPH sprowadza się do rygorystycznego wymuszenia nieściśliwości poprzez iteracyjne rozwiązanie dyskretnego układu równań wynikających z warunku nieściśliwości z różnymi wyrazami źródłowymi. Podejście to stabilizuje ciecz na dwóch poziomach.

Poziom zagęszczenia. Celem tego etapu jest utrzymanie stałej objętości cieczy poprzez korekcję prędkości cząstek. Algorytm wyznacza błąd gęstości jako różnicę między gęstością bieżącej a spoczynkową ρ_0 i uwzględnia wpływ zmian wynikających z adwekcji. Poniższy algorytm oblicza wyraz źródłowy — wielkość określającą, o ile i w jakim kierunku prędkości cząstek muszą zostać skorygowane; sama aktualizacja wektorów prędkości stanowi odrębny krok procedury (algorytm 1.1).

Algorithm 1.1: Wyznaczanie wyrazu źródłowego gęstości [6]

dla każdej cząstki i **wykonaj**
- pobierz gęstość bieżącą ρ_i i spoczynkową ρ_0;
- oblicz sumę dywergencji d_i z różnic prędkości względem sąsiadów j;
- $source_i \leftarrow \frac{\rho_0 - \rho_i}{\Delta t} - d_i$;
- przekaż $source_i$ do solvera ciśnienia;

Poziom prędkości. Nawet po skorygowaniu gęstości pole prędkości może zawierać błędy, które w kolejnym kroku czasowym spowodowałyby ponowne zagęszczenie materii. Drugi solver działa wyłącznie na wektorach prędkości, eliminując ich zbieżność lub rozbieżność. Analogicznie do poprzedniego, algorytm wyznacza wyłącznie wyraz źródłowy korekcji — aktualizacja prędkości na jego podstawie stanowi osobny krok (algorytm 1.2).

Algorithm 1.2: Wyznaczanie wyrazu źródłowego dywergencji [6]

dla każdej cząstki i **wykonaj**
- wyzeruj sumę dywergencji d_i;
- **dla każdego sąsiada** j *cząstki* i **wykonaj**
  - dodaj do d_i iloczyn masy sąsiada m_j, różnicy prędkości $(\mathbf{v}_i - \mathbf{v}_j)$ i gradientu jądra ∇W_{ij};
- $source_i \leftarrow -d_i$;

Wybór algorytmu DFSPH jest kluczowy dla systemów pracujących w czasie rzeczywistym. Dzięki dwuetapowej korekcji, metoda ta jest znacznie bardziej odporna na błędy numeryczne niż popularne alternatywy, takie jak PBF czy nieściśliwa hydrodynamika cząstek z globalnym solverem ciśnienia (ang. *Implicit Incompressible SPH*, IISPH) — metoda sprzęgająca równania ciśnienia wszystkich cząstek w jeden wspólny układ równań, co zapewnia stabilność przy znacznie większych krokach czasowych niż klasyczne SPH.

Główną zaletą DFSPH jest możliwość stosowania znacznie większego kroku czasowego Δt bez ryzyka „eksplozji" symulacji. Przekłada się to bezpośrednio na całkowity czas obliczeń potrzebny do wygenerowania sekwencji ruchu. Jako punkt odniesienia posłużyła symulacja przełamania tamy z udziałem 125 tys. cząstek, przeprowadzona przez okres jednej sekundy przy różnych stałych krokach czasowych [6]. W tabeli 1.1 zestawiono całkowity czas obliczeń dla każdej z metod, potwierdzając dominację DFSPH w scenariuszach wymagających dużych kroków czasowych.

Przedstawione sformułowanie matematyczne metody SPH oraz algorytmu DFSPH

Tabela 1.1: Porównanie całkowitego czasu obliczeń [s] czterech wariantów SPH w scenariuszu przełamania tamy (125 tys. cząstek, 1 s symulacji) przy różnych stałych krokach czasowych Δt. Mniejsza wartość oznacza krótszy czas obliczeń [6].

Δt [ms]	IISPH [s]	PBF [s]	PCISPH [s]	**DFSPH [s]**
4.0	318.1	613.5	1085.7	51.3
2.0	267.9	508.4	1021.2	59.4
1.0	220.7	338.0	656.9	107.5
0.5	225.6	240.5	394.9	210.8
0.25	402.2	354.0	440.9	372.0

definiuje fizyczny fundament systemu. Dzięki dyskretyzacji kontinuum cieczy za pomocą chmury punktów oraz rygorystycznemu wymuszeniu nieściśliwości przez dwuetapowy solver ciśnienia, możliwe jest uzyskanie stabilnej i wydajnej symulacji w czasie rzeczywistym.

Jednak wynik symulacji w czystej postaci lagrange'owskiej to jedynie zbiór danych o pozycjach i prędkościach cząstek — surowa „chmura punktów", która nie posiada wizualnych cech ciągłej cieczy, takich jak gładka powierzchnia, załamania światła czy przezroczystość. Aby przekształcić te dane w realistyczny obraz, konieczne jest zastosowanie zaawansowanych technik graficznych.

1.3 Wizualizacja objętościowa metodą Raymarching

Przekształcenie surowych danych cząsteczkowych, pochodzących z symulacji SPH, w ciągły i fotorealistyczny obraz cieczy wymaga zastosowania metod pośredniej reprezentacji materii. W przeciwieństwie do klasycznych technik graficznych, które opierają się na generowaniu jawnej geometrii, w niniejszej pracy zdecydowano się na wykorzystanie bezpośredniego renderowania objętościowego. Koncepcja ta, zaproponowana przez Marca Levoya [35], zakłada formowanie obrazu bezpośrednio z danych wolumetrycznych bez konieczności kosztownej ekstrakcji izopowierzchni do postaci siatki wielokątów.

Rekonstrukcja ciągłego pola skalarnego. Symulacja w ujęciu lagrange'owskim dostarcza jedynie dyskretny zbiór punktów masowych. Aby umożliwić analizę wizualną cieczy jako obiektu ciągłego, konieczne jest odtworzenie pola gęstości w każdym punkcie przestrzeni symulacji. Wykorzystuje się do tego znaną z matematycznego sfor-

mułowania SPH tożsamość interpolacyjną:

$$\rho(\mathbf{x}) = \sum_j m_j W(|\mathbf{x} - \mathbf{x}_j|, h), \tag{1.9}$$

gdzie $\mathbf{x}$ jest dowolnym punktem w przestrzeni, m_j oraz $\mathbf{x}_j$ — masą i położeniem j-tej cząstki, h — promieniem wygładzania, a W funkcją wygładzającą. Równanie to opisuje przejście od punktowej reprezentacji materii do trójwymiarowego, ciągłego pola objętościowego, spopularyzowanego w nowoczesnej grafice czasu rzeczywistego m.in. przez prace Inigo Quileza [41]. W praktyce obliczeniowej pole to poddawane jest dyskretyzacji — przestrzeń symulacji dzieli się na regularną strukturę o rozdzielczości $N_x \times N_y \times N_z$, gdzie każdy węzeł przechowuje próbkowaną wartość gęstości $\rho_{i,j,k} = \rho(\mathbf{x}_{i,j,k})$.

Numeryczna integracja promieni. Głównym narzędziem wizualizacji tak przygotowanego pola jest przemarsz promienia (ang. Raymarching). Jest to metoda numerycznej integracji światła przechodzącego przez ośrodek objętościowy, która dzięki pełnej paralelizacji idealnie nadaje się do realizacji w środowisku GPU [52]. W modelu tym dla każdego piksela obrazu generowany jest promień rzutujący:

$$\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}, \tag{1.10}$$

gdzie $\mathbf{o}$ to punkt obserwacji (kamera), a $\mathbf{d}$ to wektor kierunku promienia. Promień ten „kroczy" przez pole objętościowe w małych krokach Δt, a w każdym punkcie $\mathbf{x}_k = \mathbf{o} + k\Delta t\mathbf{d}$ następuje próbkowanie gęstości.

Wizualizacja opiera się na przybliżeniu całki objętościowej, która określa końcowy kolor piksela na podstawie lokalnej absorpcji i transmisji światła w cieczy. Kluczową zaletą tego podejścia w kontekście wydajności jest fakt, że proces ten odbywa się całkowicie w przestrzeni ekranu, zazwyczaj wewnątrz shaderów fragmentów — programów GPU wywoływanych raz dla każdego piksela obrazu w celu obliczenia jego końcowego koloru. Oznacza to, że system nie buduje fizycznej siatki wielokątów, lecz dynamicznie wyznacza punkty przecięcia promienia z izopowierzchnią cieczy [42].

Aby umożliwić realistyczne oświetlenie cieczy (obliczanie odbić i załamań światła), konieczne jest wyznaczenie wektora normalnego do jej powierzchni. Ponieważ powierzchnia w renderingu objętościowym nie jest zdefiniowana wprost, wektor normalny $\mathbf{n}$ w punkcie $\mathbf{x}$ aproksymuje się jako znormalizowany gradient pola gęstości $\rho(\mathbf{x})$:

$$\mathbf{n}(\mathbf{x}) = \frac{\nabla\rho(\mathbf{x})}{|\nabla\rho(\mathbf{x})|}. \tag{1.11}$$

Gradient ten wskazuje kierunek najszybszego wzrostu gęstości, co pozwala matematycznie traktować granice obszarów o wysokiej gęstości jako fizyczną barierę cieczy.

Dzięki temu możliwe jest zastosowanie klasycznych modeli oświetlenia, takich jak model Phonga, oraz zaawansowanych efektów optycznych wynikających z praw Snella i Fresnela [12].

Zastosowanie rekonstrukcji objętościowej i przemarszu promienia stanowi pomost między dyskretnymi danymi fizycznymi a ciągłym obrazem. Pozwala to na uzyskanie wysokiej jakości wizualizacji z naturalnie wyglądającymi detalami powierzchni, zachowując jednocześnie wysoką wydajność obliczeniową na nowoczesnych układach graficznych.

Rozdział 2

Przegląd technologii i środowisko programistyczne

Przedstawione w poprzednim rozdziale podstawy teoretyczne — od eulerowskiego i lagrange'owskiego opisu ruchu, przez sformułowanie metody SPH, aż po techniki bezpośredniego renderowania objętościowego — definiują ramy matematyczne niezbędne do budowy kompletnego systemu symulacji. Jednak transformacja tych abstrakcyjnych modeli w działającą aplikację czasu rzeczywistego wymaga starannego doboru stosu technologicznego. Wybór odpowiedniego języka programowania, interfejsów programowania aplikacji graficznych oraz struktur danych ma kluczowy wpływ na ostateczną wydajność i stabilność systemu. W niniejszym rozdziale dokonano przeglądu technologii oraz środowiska programistycznego, które umożliwiły implementację opisanych mechanizmów z wykorzystaniem pełnej mocy obliczeniowej współczesnych procesorów i układów graficznych.

2.1 Istniejące rozwiązania symulacji cieczy

Współczesne techniki symulacji cieczy ewoluowały w stronę metod hybrydowych oraz optymalizowanych podejść cząsteczkowych, aby sprostać wymaganiom realizmu w nauce i rozrywce. Współczesne rozwiązania obejmują szeroki wachlarz podejść — od metod czysto eulerowskich i czysto cząsteczkowych po hybrydy obu. Jak omówiono w rozdziale pierwszym, aktualnym standardem przemysłowym jest metoda FLIP, łącząca stabilność siatki eulerowskiej z elastycznością cząstek lagrange'owskich [9]. Dominuje ona w silnikach przeznaczonych do tworzenia gier i animacji, gdzie liczy się zarówno wydajność, jak i jakość wizualna.

Obok rozwiązań opartych na FLIP istnieje jednak ekosystem narzędzi skoncentrowanych wyłącznie na formułowaniu cząsteczkowym SPH. Szczególne miejsce zajmuje

tu biblioteka SPlisHSPlasH, tworzona przez tę samą grupę badawczą, która zaproponowała algorytm DFSPH będący przedmiotem niniejszej pracy [5, 6]. Przegląd reprezentatywnych narzędzi z obu obszarów pozwala uchwycić różnorodność podejść projektowych oraz wyznaczyć punkt, w którym lokuje się system opisany w dalszej części pracy.

2.1.1 Symulacja FLIP w programie Blender

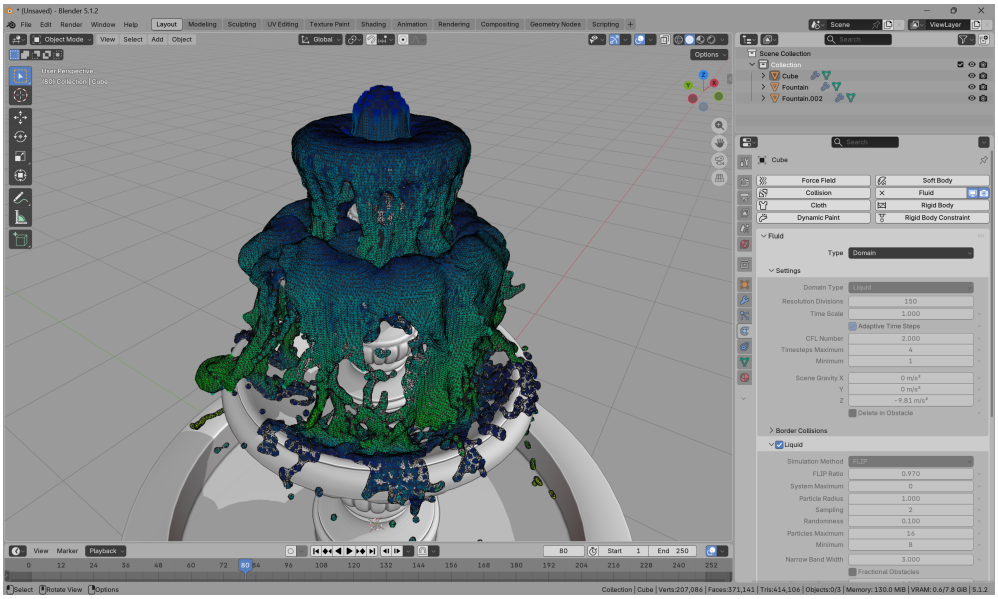
Blender jest wolnym i otwartoźródłowym pakietem do modelowania, animacji i renderowania trójwymiarowego, rozwijanym przez Blender Foundation od 1994 roku [7]. Narzędzia symulacji cieczy dostępne w tym programie opierają się na silniku Mantaflow, zintegrowanym z Blenderem począwszy od wersji 2.82 [49]. Implementacja ta korzysta z metody FLIP, w której domenę obliczeniową opisuje siatka eulerowska, a cząstki lagrange'owskie pełnią rolę tracerów adwekcji — znaczników śledzących, które przenoszą prędkość przepływu między kolejnymi krokami symulacji. Centralna logika obliczeniowa wykonywana jest na procesorze głównym.

Blender jest narzędziem zorientowanym na fizycznie dokładną animację komputerową — jego narzędzia symulacyjne nie stawiają sobie za cel pracy w czasie rzeczywistym, lecz maksymalną jakość wyników. Narzędzie zaprojektowano z myślą o generowaniu sekwencji animacji, w których jakość fizyczna i wizualna stawiana jest ponad ograniczeniami czasu rzeczywistego. Wynik symulacji jest zapisywany na dysku jako pamięć podręczna danych wolumetrycznych, a następnie renderowany niezależnie za pomocą rendererów Cycles lub EEVEE. Podejście to sprawdza się doskonale w produkcji filmowej i reklamowej, lecz ze swej natury wyklucza interaktywność i sprzężenie zwrotne w czasie rzeczywistym. Typowy widok interfejsu konfiguracji domeny i podglądu pola cieczy przedstawia rysunek 2.1.

2.1.2 Symulacja FLIP w silniku Unreal Engine 5

Unreal Engine 5 jest zaawansowanym silnikiem aplikacji interaktywnych, rozwijanym przez Epic Games i stosowanym zarówno w produkcji gier, jak i przy tworzeniu efektów wizualnych dla filmów i symulacji [14]. Jednym z jego kluczowych podsystemów jest Niagara — elastyczny system efektów wizualnych i cząsteczkowych, wykonujący obliczenia w całości na GPU. Możliwości symulacji cieczy udostępnia wtyczka NiagaraFluids, implementująca metodę FLIP dla trójwymiarowych cieczy.

Środowisko pracy oparte jest na edytorze węzłów, widocznym na rysunku 2.2, który oferuje gotowe szablony symulacji przeznaczone do bezpośredniego umieszczenia na poziomie gry. Szablony te stanowią punkt wyjścia do dalszej personalizacji — modularna architektura komponentów pozwala twórcy dostosować każdy aspekt symulacji



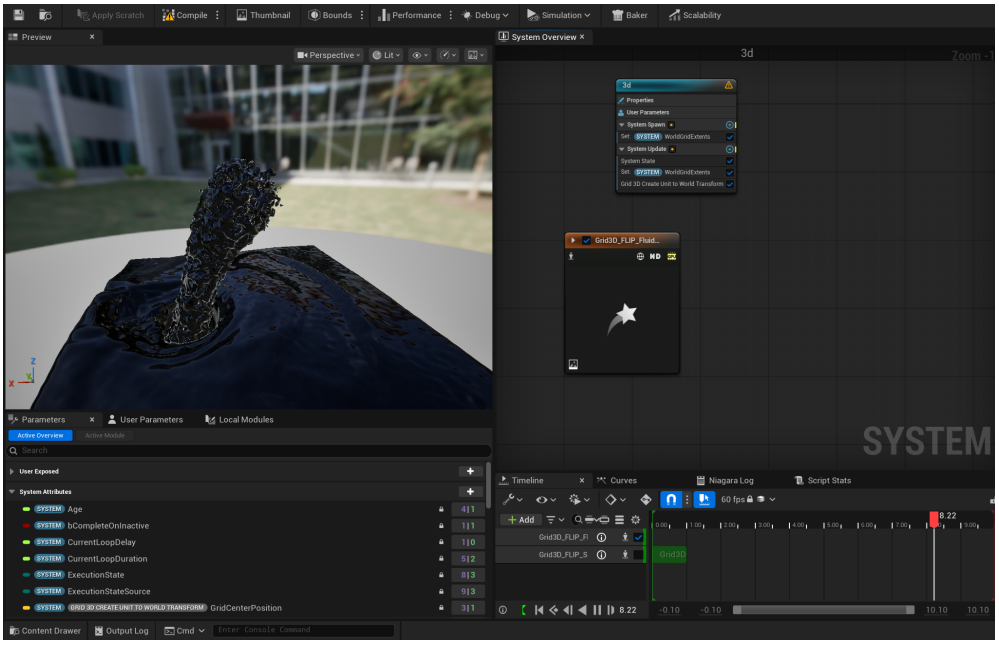
Rysunek 2.1: Interfejs symulacji FLIP w programie Blender z widocznym panelem konfiguracji domeny i podglądem cieczy. Źródło: opracowanie własne na podstawie dokumentacji [8]

do konkretnego zadania, od konfiguracji sił i warunków brzegowych, aż po modyfikację poszczególnych kroków solvera. Silnik jest zatem rozwiązaniem skalowalnym: umożliwia zarówno szybkie składanie efektów przez artystów — osoby bez specjalistycznego przygotowania programistycznego, konfigurujące symulację wyłącznie przez interfejs graficzny — jak i głębszą ingerencję algorytmiczną przez programistów efektów, którzy mają możliwość pisania własnych modułów obliczeniowych i modyfikowania wewnętrznej logiki solvera.

2.1.3 Biblioteka SPlisHSPlasH

SPlisHSPlasH (ang. *Smoothed Particle Hydrodynamics Simulation Library*) jest otwartą biblioteką symulacyjną tworzoną przez grupę badawczą Jana Bendera [5]. W odróżnieniu od dwóch poprzednich rozwiązań, biblioteka ta opiera się wyłącznie na metodzie SPH w formie cząsteczkowej, bez stosowania pomocniczej siatki eulerowskiej. Pod względem przyjętego modelu fizycznego stanowi ona najbliższy odpowiednik systemu opisanego w niniejszej pracy.

Biblioteka implementuje zestaw nowoczesnych solverów nieściśliwości, w tym m.in. DFSPH. Ponadto wspierane są modele fenomenologiczne lepkości, napięcia powierzchniowego i sił wirowania. Jej architektura opiera się wyłącznie na języku C++, a cały potok symulacyjny wykonywany jest na procesorze głównym. Jedynym krokiem dopuszczającym opcjonalne przeniesienie obliczeń na GPU jest wyszukiwanie sąsiedztwa, realizowane przez zewnętrzną bibliotekę CompactNSearch — jej wariant cuNSearch

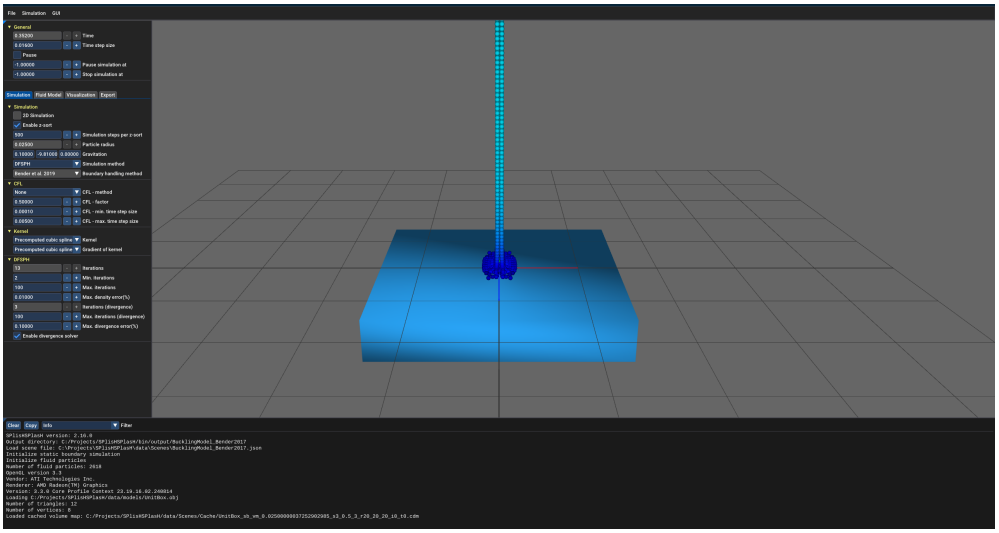


Rysunek 2.2: Interfejs edytora Niagara w silniku Unreal Engine 5 z aktywną symulacją 3D FLIP zrealizowaną przy użyciu wtyczki NiagaraFluids. Źródło: opracowanie własne na podstawie dokumentacji [14]

umożliwia wykonanie tego kroku na karcie graficznej [29]. Warstwa wizualizacyjna korzysta z OpenGL 3.3 wyłącznie do renderowania wyników symulacji, co widoczne jest na rysunku 2.3. SPlisHSPlasH jest szeroko stosowana w środowisku naukowym jako platforma referencyjna do testowania i porównywania nowych algorytmów SPH [30].

2.1.4 Podsumowanie rozwiązań

Podsumowując przegląd istniejących rozwiązań, można wskazać na istotne różnice w ich architekturze oraz zastosowaniach. Blender jest środowiskiem nastawionym na generowanie wysokiej jakości animacji, które wymagają wcześniejszego przeliczenia. Unreal Engine 5 z systemem Niagara oferuje wysoką wydajność w środowisku interaktywnym, lecz opiera się na złożonej, wysokopoziomowej architekturze silnika gry. SPlisHSPlasH z kolei jest biblioteką koncentrującą się na obliczeniach równoległych na procesorze w celach naukowych.



Rysunek 2.3: Symulacja cieczy w bibliotece SPlisHSPlasH z aktywnym solverem DFSPH i panelem konfiguracji parametrów fizycznych. Źródło: opracowanie własne na podstawie dokumentacji biblioteki [5]

2.2 Język programowania Rust

Rust jest wieloparadygmatowym językiem programowania systemowego, zaprojektowanym przez pracowników Mozilla Research i opublikowanym w pierwszej stabilnej wersji w 2015 roku [28]. Od 2021 roku jego rozwój nadzoruje niezależna fundacja Rust Foundation. Język został stworzony z myślą o wypełnieniu niszy zajmowanej dotychczas przez C i C++ — oferuje porównywalną wydajność obliczeniową przy jednoczesnym wyeliminowaniu całej klasy błędów wynikających z ręcznego zarządzania pamięcią.

Centralną innowacją języka jest system własności, który pozwala kompilatorowi weryfikować poprawność zarządzania pamięcią statycznie — bez konieczności stosowania odśmiecacza pamięci. W odróżnieniu od języków zarządzanych, takich jak Java czy C#, programy w Rust nie ponoszą kosztów przerw wywołanych odśmiecaniem. Jednocześnie, w odróżnieniu od C i C++, kompilator gwarantuje brak niezdefiniowanych zachowań w bezpiecznym kodzie: wiszące wskaźniki, przepełnienia buforów i wyścigi danych są weryfikowane statycznie i odrzucane już na etapie kompilacji. Brak wskaźników null jest egzekwowany przez typ `Option<T>`, natomiast obsługa błędów opiera się na typie `Result<T, E>`, wymuszającym jawne rozpatrzenie każdego przypadku wyjątkowego. Rust nie zapewnia jednak automatycznej ochrony w blokach oznaczonych jako `unsafe` — programista przejmuje wówczas pełną odpowiedzialność za poprawność operacji niskopoziomowych, co jest niezbędne przy bezpośredniej komunikacji z API graficznym.

W odróżnieniu od języków obiektowych Rust nie posiada klas — dane i zachowa-

nie są rozdzielone: struktury (ang. *structs*) przechowują pola, a bloki *impl* definiują operujące na nich metody. Rolę interfejsów pełnią cechy (ang. *traits*) — określają kontrakt metod, który dowolny typ może spełnić niezależnie od swojej wewnętrznej reprezentacji.

Ekosystem języka Rust opiera się na wbudowanym menedżerze pakietów Cargo oraz publicznym repozytorium crates.io. Podstawową jednostką dystrybucji kodu jest skrzynka (ang. *crate*) — pakiet kodu Rust przyjmujący formę biblioteki lub programu wykonywalnego. Repozytorium crates.io pełni rolę analogiczną do PyPI w Pythonie czy npm w JavaScript, pozwalając na deklaratywne zarządzanie zależnościami bezpośrednio w pliku konfiguracyjnym projektu, bez potrzeby instalowania zewnętrznych narzędzi. Poza samym językiem, o praktycznej użyteczności Rusta w znacznej mierze decyduje bogaty zbiór dojrzałych skrzynek społecznościowych, które — choć nie wchodzą w skład biblioteki standardowej — pozwalają zrealizować większość typowych zadań inżynierskich bez sięgania po rozwiązania spoza języka. Zarządzanie oknem aplikacji i obsługę pętli zdarzeń zapewnia `winit` [56]. Obliczenia algebraiczne po stronie CPU — wektory i macierze przekształceń — realizuje `glam` [20], biblioteka zoptymalizowana pod kątem instrukcji SIMD. Budowę interfejsów użytkownika umożliwia `egui` [15] wraz z warstwą integracyjną `egui_winit_vulkano` [13]. Obsługę błędów na poziomie bibliotek i aplikacji upraszczają odpowiednio `thiserror` [51] i `anyhow` [50], natomiast ujednoliconą infrastrukturę logowania dostarcza para `log` [48] i `simple_logger` [45].

Tabela 2.1: Porównanie języków Rust i C++ pod względem cech istotnych dla programowania systemowego

Cecha	C++	Rust
Bezpieczeństwo pamięci	Manualne	Gwarantowane przez kompilator
Odśmiecacz pamięci	Nie	Nie
Niezdefiniowane zachowania	Możliwe	Niemożliwe (safe Rust)
Wyścigi danych	Możliwe	Niemożliwe (czas kompilacji)
Wskaźniki null	Tak	Nie (`Option<T>`)
Zerowe koszty abstrakcji	Tak	Tak
Standardowy menedżer pakietów	Brak	Cargo
Dojrzałość ekosystemu	Wysoka (od 1983)	Rosnąca (od 2015)

Rust jest coraz powszechniej postrzegany jako naturalny następca C++ w nowych projektach systemowych. Organizacje takie jak Microsoft, Google i Linux Foundation włączyły go już do swoich stosów technologicznych jako odpowiedź na rosnące wymagania w zakresie bezpieczeństwa oprogramowania. Jako język oferujący zerowe koszty

abstrakcji, rozbudowany system typów i statyczne gwarancje poprawności bez narzutu czasowego, Rust eliminuje całą klasę błędów trudnych do debugowania w złożonych systemach wielowątkowych — co czyni go szczególnie trafnym wyborem dla implementacji silnika symulacji cieczy opisanego w niniejszej pracy.

2.3 API graficzne Vulkan

Vulkan jest niskopoziomowym, wieloplatformowym interfejsem programowania graficznego, opracowanym przez konsorcjum Khronos Group i opublikowanym w 2016 roku [47]. Jego geneza wynika z narastających ograniczeń architektury OpenGL, która przez ponad dwie dekady rozrastało się o kolejne warstwy abstrakcji i niejawne mechanizmy zarządzania zasobami. Sterownik OpenGL pełnił rolę rozbudowanego pośrednika: sam synchronizował polecenia GPU, podejmował decyzje o alokacji pamięci i kompilował shadery w nieokreślonych momentach, co prowadziło do znacznego narzutu procesora głównego, uniemożliwiało efektywną wielowątkowość i skutkowało nieprzewidywalnym zachowaniem w sytuacjach granicznych. Bezpośrednim poprzednikiem Vulkana jest biblioteka Mantle [1], opracowana przez AMD we współpracy z deweloperami gier w 2013 roku, stanowiąca pierwszą propozycję niskopoziomowego API graficznego, która wykazała, że jawne przejęcie przez programistę kontroli nad sprzętem przekłada się na mierzalne zyski wydajnościowe.

Centralną filozofią Vulkana jest jawność i determinizm. Każdy zasób, taki jak bufor pamięci, obraz, potok obliczeniowy, jest tworzony i zarządzany przez programistę w sposób jawny, a każda operacja synchronizacji między procesorem a GPU musi być ręcznie zdefiniowana. Rezygnacja z globalnego automatu stanów na rzecz izolowanych, niezależnych obiektów umożliwia równoległe budowanie potoków poleceń w wielu wątkach bez ryzyka konfliktu stanu. Ujednoliconym wzorcem inicjalizacji w API Vulkan jest wykorzystanie struktur opisowych. Tworzenie dowolnego obiektu wymaga od programisty wypełnienia dedykowanej struktury konfiguracyjnej, która precyzyjnie definiuje jego parametry, a następnie przekazania jej do odpowiedniej funkcji tworzącej. Podejście to, stosowane konsekwentnie w całym standardzie, sprawia, że konfiguracja systemu jest w pełni jawna. Pozwala to na precyzyjne określenie zależności między obiektami już na poziomie kodu, co eliminuje domyślne stany sterownika graficznego.

Drugą fundamentalną cechą Vulkana jest system rozszerzeń, zaprojektowany jako integralna część architektury API od pierwszej wersji specyfikacji. Rdzeń Vulkana jest celowo minimalny i stabilny: obejmuje wyłącznie operacje wspólne dla wszystkich implementacji sprzętowych. Wszelkie funkcjonalności specyficzne dla producenta lub nowsze możliwości renderowania udostępniane są przez rozszerzenia aktywowane jawnie przy tworzeniu instancji lub urządzenia. Rozszerzenia dzielą się na trzy kategorie:

rozszerzenia producenta sprzętu (np. `VK_NV_*` lub `VK_AMD_*`), rozszerzenia wieloproducentowe (`VK_EXT_*`) oraz rozszerzenia ratyfikowane przez Khronos (`VK_KHR_*`).

Ostatnia kategoria powstaje w drodze konsensusu wszystkich członków organizacji Khronos — AMD, NVIDIA, Intel, ARM, Google, Apple i innych — i pełni dwojaką rolę. Po pierwsze, dostarcza krytyczne funkcjonalności obsługiwane przez wszystkie współczesne karty graficzne, które z różnych powodów nie trafiły do rdzenia specyfikacji; przykładem jest `VK_KHR_swapchain` umożliwiający prezentację obrazu na ekranie systemu operacyjnego, czy `VK_KHR_ray_tracing_pipeline` wprowadzający sprzętowe śledzenie promieni. Po drugie, rozszerzenia `KHR` pełnią rolę oficjalnej „*poczekalni*" dla nowych technologii: zamiast pospiesznie modyfikować stabilny rdzeń, Khronos publikuje rozszerzenie, pozwala deweloperom je przetestować w warunkach produkcyjnych, a jeśli okaże się ono dojrzałe — włącza je do kolejnej wersji specyfikacji rdzeniowej (ang. *core specification*). Mechanizm ten był stosowany konsekwentnie w wersjach Vulkan 1.1, 1.2 i 1.3, do których przeniesiono łącznie kilkadziesiąt wcześniejszych rozszerzeń `KHR`.

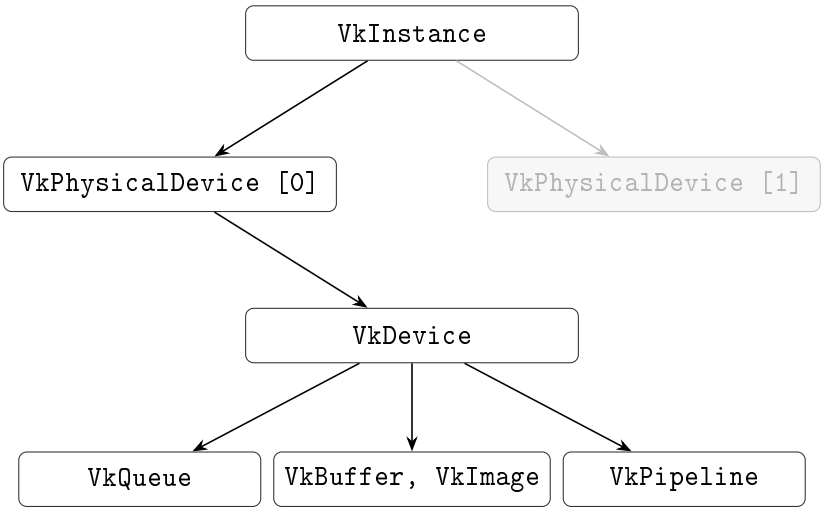
Podejście to umożliwia producentom sprzętu udostępnianie możliwości swoich układów bez naruszania globalnej kompatybilności, pozwalając jednocześnie programiście na jawne deklarowanie funkcjonalności wymaganych przez aplikację.

2.3.1 Instancja, urządzenie fizyczne i logiczne

`VkInstance` jest punktem wejścia do środowiska uruchomieniowego Vulkana i korzeniem całej hierarchii obiektów API. Jej utworzenie ładuje sterownik Vulkana i inicjalizuje rozszerzenia działające na poziomie instancji, które dotyczą platformy systemowej, a nie konkretnego układu graficznego. Przykładem jest `VK_KHR_surface`, które definiuje abstrakcję powierzchni okienkowej niezależną od systemu operacyjnego, oraz towarzyszące mu rozszerzenia platformowe, takie jak `VK_KHR_win32_surface` czy `VK_KHR_xcb_surface`, odpowiadające za integrację z konkretnym podsystemem okienkowym. Rozszerzenia instancji aktywuje się przy jej tworzeniu, deklarując ich nazwy w strukturze `VkInstanceCreateInfo`.

Po utworzeniu instancji możliwe jest wyliczenie dostępnych urządzeń fizycznych. `VkPhysicalDevice` reprezentuje jeden fizyczny układ graficzny widoczny dla systemu. Na stacji roboczej może znajdować się kilka takich układów, w tym zintegrowany procesor graficzny wbudowany w jednostkę centralną oraz dedykowana karta graficzna. Vulkan pozwala programiście odpytać każde z nich o właściwości: typ urządzenia (dyskretna karta graficzna, układ zintegrowany, CPU), limity sprzętowe, obsługiwane formaty obrazów i funkcjonalności. Na tej podstawie aplikacja może samodzielnie wybrać najbardziej odpowiedni układ lub umożliwić taki wybór użytkownikowi.

Wykorzystując wybrany układ fizyczny, programista tworzy urządzenie logiczne `VkDevice`, stanowiące centralny obiekt API, z którym powiązane są wszystkie kolejki poleceń, bufory, obrazy oraz potoki obliczeniowe. Na tym etapie aktywowane są rozszerzenia działające na poziomie urządzenia, czyli dotyczące możliwości konkretnego układu graficznego. Rozróżnienie poziomów ma praktyczne znaczenie: `VK_KHR_surface` jest rozszerzeniem instancji, bo integracja z oknem systemu operacyjnego jest kwestią platformową; `VK_KHR_swapchain`, czyli mechanizm prezentowania klatek na ekranie, jest już rozszerzeniem urządzenia, bo wymaga fizycznego wyjścia wyświetlacza. Układ graficzny w serwerze bez monitora może nie obsługiwać `VK_KHR_swapchain` w ogóle, lecz nadal będzie w pełni zdolny do obliczeń ogólnego przeznaczenia na GPU. Taki podział pozwala aplikacji jawnie deklarować wymagane zależności sprzętowe zamiast zakładać obecność możliwości, których dany układ może nie posiadać. Relację między trzema warstwami inicjalizacyjnymi ilustruje rysunek 2.4.



Rysunek 2.4: Hierarchia inicjalizacyjna Vulkana. Źródło: opracowanie własne na podstawie specyfikacji [47]

2.3.2 Kolejki i pule poleceń

Nowoczesne karty graficzne udostępniają wiele sprzętowych kolejek poleceń, przez które procesor przekazuje pracę do wykonania na GPU. Vulkan grupuje je w rodziny kolejek, z których każda obsługuje określony zestaw operacji. Typowa karta graficzna eksponuje co najmniej rodzinę graficzną, obsługującą rysowanie i prezentację klatek, oraz rodzinę obliczeniową, przeznaczoną do obliczeń ogólnego przeznaczenia. Wiele układów udostępnia też wyspecjalizowaną rodzinę transferową, zoptymalizowaną sprzętowo do kopiowania danych między pamięcią systemową procesora głównego a dedykowaną pamięcią karty graficznej. W nomenklaturze Vulkana strona procesoro-

wa systemu, obejmująca procesor główny i pamięć RAM, określana jest terminem *host*, zaś strona układu graficznego wraz z jego pamięcią wideo terminem *device*. Programista odpytuje wybrany `VkPhysicalDevice` o dostępne rodziny i deklaruje, z których z nich chce korzystać, w strukturze `VkDeviceCreateInfo` przekazywanej przy tworzeniu urządzenia logicznego. Efektem jest zbiór uchwytów `VkQueue`, z których każdy reprezentuje jedną kolejkę danej rodziny.

Samo polecenie nie jest przekazywane do kolejki bezpośrednio jako wywołanie funkcji. Zamiast tego Vulkan stosuje model rejestrowania odroczonych poleceń. Programista tworzy `VkCommandPool`, obiekt zarządzający pamięcią przeznaczoną na polecenia, powiązany z konkretną rodziną kolejek. Z puli tej alokowane są bufory poleceń `VkCommandBuffer`, w których rejestruje się sekwencje operacji GPU. Operacje te obejmują przesłania danych między buforami, bariery synchronizacji, uruchomienia potoków graficznych oraz wywołania obliczeniowe, określane terminem *dispatch*. Bufor poleceń rejestruje całą sekwencję operacji po stronie procesora głównego, a następnie przekazywany jest do kolejki jednym wywołaniem funkcji wysyłającej. GPU przetwarza zarejestrowane polecenia asynchronicznie względem procesora, co jest zasadniczą różnicą w stosunku do modelu OpenGL, gdzie każde wywołanie API było natychmiastowym żądaniem wykonywanym przez sterownik.

Rozdzielenie rejestrowania od wysyłania przynosi dwie praktyczne konsekwencje. Po pierwsze, wiele wątków procesora może jednocześnie rejestrować polecenia w niezależnych buforach, korzystając z osobnych pul powiązanych z tą samą rodziną kolejek. Po drugie, bufor poleceń reprezentujący stały zestaw pracy, który nie zmienia się między klatkami, może być zarejestrowany raz i wysyłany wielokrotnie bez ponownego budowania.

2.3.3 Łańcuch wymiany i dynamiczne renderowanie

Renderowanie w czasie rzeczywistym polega na cyklicznym generowaniu kolejnych obrazów klatki i przesyłaniu ich na ekran. Konieczność synchronizacji tego procesu z monitorem wynika z fundamentalnej właściwości sprzętu wyświetlającego: monitor odświeża obraz ze stałą częstotliwością, odczytując zawartość bufora wideo z pamięci karty graficznej. Jeśli aplikacja zapisuje nową klatkę bezpośrednio do obszaru, który monitor właśnie odczytuje, na ekranie pojawia się artefakt rozrywania obrazu (ang. *screen tearing*): górna część klatki pochodzi z poprzedniego obrazu, a dolna z aktualnie zapisywanego. Rozwiązaniem jest oddzielenie bufora wyświetlanego od bufora roboczego i podmiana ich zawartości dopiero po zakończeniu cyklu odświeżania. W odróżnieniu od OpenGL, które ukrywało ten mechanizm za pojęciem domyślnego bufora ramki, Vulkan wymaga jawnego skonstruowania tej infrastruktury w postaci łańcucha

wymiany (ang. *swapchain*).

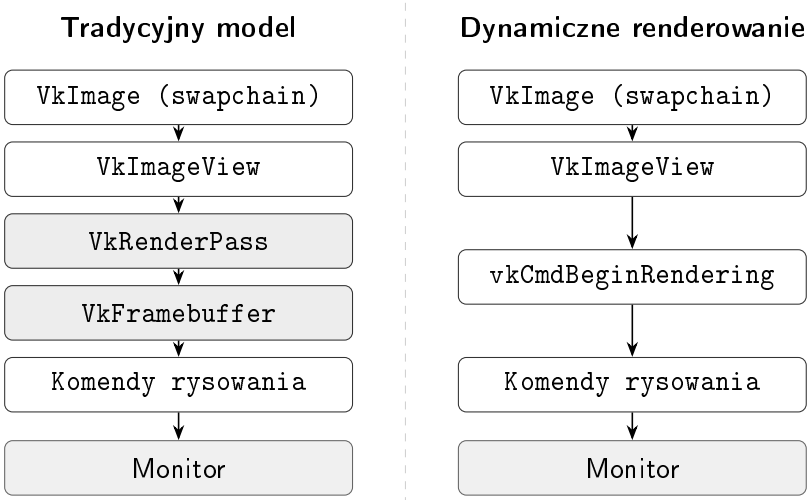
Łańcuch wymiany jest zestawem obrazów zarządzanych przez system okienkowy, przeznaczonych do prezentacji na ekranie. Aplikacja pobiera jeden z dostępnych obrazów, rysuje do niego klatkę, a następnie oddaje go do kolejki prezentacji, skąd trafia on na ekran w odpowiednim momencie. Liczba obrazów w łańcuchu jest konfigurowalna: przy dwóch obrazach jeden jest wyświetlany, a drugi przygotowywany, co określa się mianem podwójnego buforowania (ang. *double buffering*); przy trzech obrazach aplikacja może przygotowywać kolejną klatkę jeszcze zanim poprzednio złożona dotrze na ekran, co nosi nazwę potrójnego buforowania (ang. *triple buffering*) i zmniejsza ryzyko przestojów renderowania kosztem nieznacznie wyższego opóźnienia. Sposób synchronizacji z częstotliwością odświeżania monitora określa tryb prezentacji: tryb `FIFO` odpowiada klasycznej synchronizacji pionowej, a tryb `MAILBOX` zastępuje oczekujący obraz nowszym, jeśli ten jest już gotowy, minimalizując widoczne opóźnienie klatki.

Vulkan oferuje dwa podejścia do przekazania poleceń rysowania do obrazu łańcucha wymiany: model tradycyjny oparty na jawnie skompilowanych obiektach przejścia renderowania i bufora ramki oraz dynamiczne renderowanie, wprowadzone jako funkcja rdzeniowa w wersji 1.3. W modelu tradycyjnym przejście renderowania (`VkRenderPass`) jest deklaratywnym opisem struktury operacji rysowania: definiuje, jakie załączniki obrazowe będą używane, w jakim formacie, co ma się stać z ich zawartością na początku przejścia (wyczyszczenie lub zachowanie) i na końcu (zachowanie lub odrzucenie). Opis ten jest kompilowany przez sterownik z wyprzedzeniem, co pozwala mu na optymalizację dostępów do pamięci. Bufor ramki (`VkFramebuffer`) wiąże konkretne widoki obrazów (`VkImageView`) z załącznikami zadeklarowanymi w przejściu renderowania. Ponieważ łańcuch wymiany dostarcza zestaw obrazów, a nie jeden, aplikacja tworzy osobny bufor ramki dla każdego obrazu w łańcuchu i wybiera właściwy przy budowaniu bufora poleceń danej klatki. Oba podejścia, tradycyjne i dynamiczne, zestawia rysunek 2.5.

W modelu dynamicznego renderowania programista deklaruje używane załączniki bezpośrednio w buforze poleceń, tuż przed poleceniami rysowania, zamiast opisywać je z wyprzedzeniem w dedykowanym obiekcie kompilowanym przez sterownik. Eliminuje to potrzebę utrzymywania zgodności między przejściami renderowania a potokami graficznymi i lepiej odpowiada architekturom, w których zestaw załączników zmienia się między krokami renderowania.

2.3.4 Zasoby: bufory, obrazy i deskryptory

Pojęcie `VkImage` pojawiło się już przy omawianiu łańcucha wymiany jako kontener klatki gotowej do prezentacji. W szerszym sensie obraz jest zasobem GPU re-



Rysunek 2.5: Porównanie ścieżek renderowania w Vulkanie. Źródło: opracowanie własne na podstawie specyfikacji [47]

prezentującym wielowymiarową tablicę danych pikselowych, opisaną formatem (np. `VK_FORMAT_R32_SFLOAT`), rozmiarem i liczbą poziomów mipmap (prekalkulowanych kopii obrazu w malejących rozdzielczościach). Sprzęt przechowuje obrazy w wewnętrznym układzie kafelkowym, zoptymalizowanym pod kątem lokalności dostępów podczas próbkowania i zapisu. Programista nie adresuje obrazu bezpośrednio: każdy dostęp odbywa się przez `VkImageView`, który deklaruje format interpretacji i zakres odczytywanego podzasobu.

Bufor (`VkBuffer`) jest ciągłym obszarem pamięci GPU bez narzuconej struktury wewnętrznej. Jego semantykę określają flagi użycia przekazane przy tworzeniu oraz sposób, w jaki shadery go odczytują i zapisują. Tworzenie bufora składa się z dwóch etapów: najpierw tworzy się obiekt `VkBuffer` opisujący rozmiar i zamierzone użycie, a następnie przydziela się mu pamięć wybraną z odpowiedniego stosu. Vulkan eksponuje wiele stosów pamięci o różnych własnościach: pamięć lokalna dla urządzenia (ang. *device-local*) rezyduje wyłącznie po stronie układu graficznego i oferuje najwyższą przepustowość, natomiast pamięć widoczna dla procesora (ang. *host-visible*) jest dostępna do bezpośredniego zapisu przez procesor główny. W praktyce zarządzanie tym procesem deleguje się do biblioteki Vulkan Memory Allocator (VMA) [21], opracowanej przez AMD: programista deklaruje wyłącznie przeznaczenie bufora, a VMA dobiera właściwy stos pamięci i zarządza jego fragmentacją. Aby załadować dane przygotowane po stronie procesora do bufora rezydującego w pamięci lokalnej dla urządzenia, stosuje się bufor pośredniczący (ang. *staging buffer*): tymczasowy bufor w pamięci widocznej dla procesora, wypełniany przez CPU, którego zawartość kopiowana jest następnie przez rodzinę kolejek transferowych do docelowej pamięci układu graficznego.

Flagi użycia determinują dozwolone operacje na buforze i wpływają na wybór stosu pamięci przez VMA. Tabela 2.2 zestawia typy buforów istotne dla niniejszej implementacji.

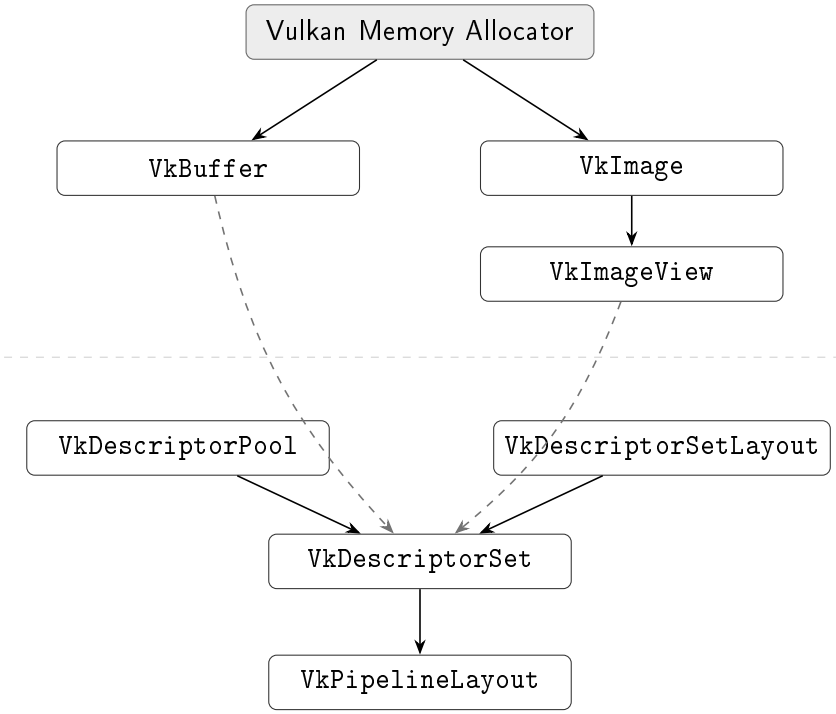
Tabela 2.2: Typy buforów Vulkana i ich zastosowanie

Typ bufora	Zastosowanie
Wierzchołkowy	Dane wierzchołków przekazywane bezpośrednio do potoku graficznego
Jednorodny (UBO)	Małe, stałe dane odczytywane w shaderach: macierze transformacji, parametry symulacji
Składowania (SSBO)	Duże dane dostępne do odczytu i zapisu w shaderach obliczeniowych; główny nośnik stanu symulacji
Pośredniczący (Staging)	Tymczasowy bufor w pamięci widocznej dla procesora, służący do transferu danych inicjalizacyjnych z CPU do pamięci lokalnej dla urządzenia

Shadery nie adresują buforów i obrazów bezpośrednio przez wskaźnik; zasoby te wiązane są z potokiem obliczeniowym za pośrednictwem zestawów deskryptorów. Schemat powiązań deklarowany jest w obiekcie `VkDescriptorSetLayout`, który precyzuje, jakiego rodzaju zasoby oczekiwane są na każdym miejscu wiązania shaderów. Alokacją zestawów zarządza `VkDescriptorPool`: obiekt rezerwujący z góry blok pamięci przeznaczony wyłącznie na zestawy danego układu, analogicznie do roli, jaką `VkCommandPool` pełni wobec buforów poleceń. Z puli alokowany jest `VkDescriptorSet`, wiążący konkretne uchwyty `VkBuffer` i `VkImageView` ze zdefiniowanymi miejscami wiązania. Układ potoków `VkPipelineLayout` agreguje jeden lub kilka takich układów zestawów i jest wbudowywany w każdy potok obliczeniowy przy jego tworzeniu, dzięki czemu sterownik zna pełen schemat zasobów wymaganych przez shadery już na etapie kompilacji. Zależności między wymienionymi obiektami ilustruje rysunek 2.6.

2.3.5 Potok obliczeniowy i graficzny

Potok graficzny lub obliczeniowy jest skompilowanym, niezmiennym obiektem zawierającym kompletną konfigurację przetwarzania GPU. Vulkan nie przyjmuje shaderów w postaci kodu źródłowego, lecz w formacie SPIR-V (ang. *Standard Portable Intermediate Representation*): przenośnego bytekodu zdefiniowanego przez Khronos Group, który zastępuje tekstowe źródła GLSL przyjmowane przez OpenGL. Kompilacja shaderów do SPIR-V odbywa się statycznie przed uruchomieniem aplikacji,

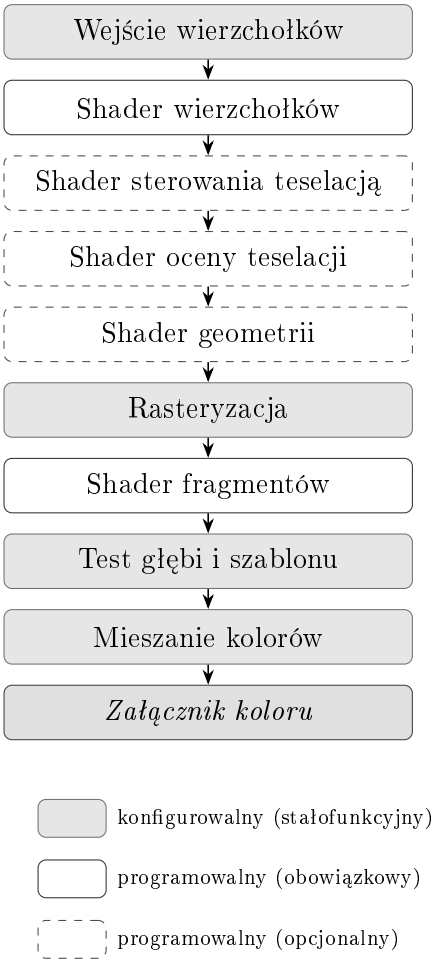


Rysunek 2.6: Schemat zarządzania zasobami i powiązań deskryptorowych w Vulkanie. Źródło: opracowanie własne na podstawie specyfikacji [47]

co umożliwia wczesne wykrycie błędów składniowych i eliminuje niedeterministyczne opóźnienia kompilacji w trakcie działania programu. Sterownik kompiluje ten bytekod do natywnych instrukcji konkretnej karty graficznej przy tworzeniu potoku i scala je z parametrami etapów stałofunkcyjnych. Ponieważ ta operacja jest kosztowna, potoki tworzy się jednorazowo podczas inicjalizacji i wiąże w buforach poleceń bez ponownej kompilacji. Vulkan rozróżnia dwa typy potoków: potok graficzny, służący do rasteryzowanego przetwarzania geometrii, oraz potok obliczeniowy, przeznaczony do równoległych obliczeń ogólnego przeznaczenia.

Potok graficzny (`VkGraphicsPipeline`) realizuje klasyczny potok rasteryzacyjny: dane wierzchołkowe są wprowadzane do potoku, przechodzą przez kolejne etapy przekształceń i dyskretyzacji, a wynikiem końcowym jest renderowany obraz (ang. *color attachment*). Składa się z etapów programowalnych, definiowanych shaderami SPIR-V, oraz etapów stałofunkcyjnych, konfigurowanych strukturami opisowymi przy tworzeniu potoku. Etap wejścia wierzchołków deklaruje format i rozmieszczenie atrybutów wierzchołkowych w buforach. Shader wierzchołków, wywoływany raz dla każdego wierzchołka, przekształca jego pozycję do przestrzeni klipu i może przepuszczać dowolne dane do kolejnych etapów. Po shaderze wierzchołków mogą opcjonalnie wystąpić trzy dalsze etapy programowalne — jeśli żaden z nich nie zostanie dostarczony przy tworzeniu potoku, etap ten jest całkowicie pomijany i geometria trafia bezpośrednio do rasteryzatora. Shader sterowania teselacją określa, na ile mniejszych trójkątów na-

leży podzielić każdy trójkąt wejściowy (tzw. poziom teselacji), natomiast shader oceny teselacji oblicza dokładne pozycje nowo wygenerowanych wierzchołków — razem pozwalają dynamicznie zagęszczać siatkę bliżej kamery bez przesyłania dodatkowych danych z procesora. Shader geometrii z kolei operuje na kompletnych prymitywach i może emitować nowe wierzchołki lub zmieniać typ prymitywu (np. zamieniać punkty w czworokąty), co pozwala generować geometrię pomocniczą — taką jak sylwetki czy cząstki — bezpośrednio na karcie graficznej. Rasteryzacja przekształca prymitywy geometryczne w dyskretne fragmenty pikseli na podstawie ich pozycji ekranowej. Shader fragmentów, wywoływany dla każdego wygenerowanego fragmentu, oblicza jego końcowy kolor. Etapy testu głębi i szablonu oraz mieszania kolorów są stałofunkcyjne, lecz w pełni konfigurowalne. Pełną sekwencję etapów i ich charakterystykę ilustruje rysunek 2.7.



Rysunek 2.7: Etapy potoku graficznego Vulkana. Etapy z przerywaną ramką są opcjonalne i mogą być pominięte przy tworzeniu potoku; etapy szare są stałofunkcyjne i konfigurowane strukturami opisowymi; etapy białe z ciągłą ramką są programowalne przez shadery SPIR-V. Źródło: opracowanie własne na podstawie specyfikacji [47]

Potok obliczeniowy (`VkComputePipeline`) zawiera wyłącznie jeden etap programowalny: shader obliczeniowy. Nie posiada wejścia wierzchołków, rasteryzacji ani wyjścia do załącznika koloru. Użycie potoku obliczeniowego w buforze poleceń sprowadza się do trzech kroków: powiązania potoku z buforem poleceń, powiązania zestawów deskryptorów z zasobami obliczeniowymi i wydania operacji dispatch.

Oba typy potoków różnią się sposobem inicjowania pracy na GPU. Polecenie rysowania, wydawane funkcją `vkCmdDraw` lub `vkCmdDrawIndexed`, przepuszcza dane wierzchołkowe przez kolejne etapy potoku graficznego: każdy wierzchołek przetwarzany jest przez shader wierzchołków, prymitywy podlegają rasteryzacji, a dla każdego fragmentu wywoływany jest shader fragmentów. Polecenie dispatch, wydawane funkcją `vkCmdDispatch(x, y, z)`, uruchamia $x \cdot y \cdot z$ grup roboczych shadera obliczeniowego bez geometrii wejściowej; każda grupa składa się z tylu wątków, ile zdefiniowano w shaderze dyrektywą `local_size`, a wyniki zapisywane są bezpośrednio do buforów składowania lub obrazów obliczeniowych przez deskryptory. Przekazanie wyników dispatch do kolejnych etapów renderowania wymaga jawnego wstawienia bariery potoku poleceniem `vkCmdPipelineBarrier`, która gwarantuje widoczność zapisanych danych przed ich odczytem przez shader graficzny.

Ponieważ procesor i GPU pracują asynchronicznie, a kolejki mogą działać równolegle, Vulkan udostępnia dwa mechanizmy synchronizacji. Semafor (`VkSemaphore`) synchronizuje pracę między kolejkami GPU: jedna kolejka sygnalizuje semafor po zakończeniu przesłanego zestawu pracy, a inna deklaruje oczekiwanie na ten sygnał przed rozpoczęciem własnych poleceń. Fence (`VkFence`) służy do synchronizacji między GPU a procesorem: po przesłaniu pracy do kolejki procesor może zablokować się wywołaniem `vkWaitForFences` do chwili, gdy GPU zasygnalizuje ukończenie tej partii pracy.

Powyższe mechanizmy łączą się w pętlę renderowania każdej klatki. Wywołanie `vkAcquireNextImageKHR` pobiera indeks dostępnego obrazu z łańcucha wymiany i sygnalizuje semafor dostępności obrazu. Wypełniony bufor poleceń przesyłany jest do kolejki graficznej z deklaracją oczekiwania na semafor dostępności i sygnalizowania semafora gotowości do prezentacji po zakończeniu; jednocześnie przekazywane jest fence powiadamiający procesor o ukończeniu klatki. Wywołanie `vkQueuePresentKHR`, oczekujące na semafor gotowości, prezentuje obraz na ekranie. Procesor czeka na fence przed ponownym użyciem zasobów danej klatki.

Jak pokazuje powyższy opis, Vulkan wymaga jawnego zarządzania każdym aspektem inicjalizacji, synchronizacji i przepływu zasobów, co przekłada się na znaczną ilość kodu infrastrukturalnego.

2.4 Biblioteka Vulkano

Vulkan jest interfejsem zdefiniowanym w języku C: nagłówki standardu eksponują funkcje i struktury zgodne z binarnym interfejsem wywołań języka C (ang. *ABI*), a sterowniki dostarczane przez producentów sprzętu implementują ten interfejs jako biblioteki dynamiczne systemu operacyjnego. Korzystanie z Vulkana w języku Rust wymaga zatem mechanizmu pozwalającego na wywoływanie obcych funkcji C bezpośrednio z kodu Rust. Mechanizm ten, zwany interfejsem funkcji zewnętrznych (ang. *Foreign Function Interface*, *FFI*), umożliwia deklarowanie w kodzie Rust sygnatur funkcji zdefiniowanych w innych językach, a następnie wywoływanie ich z przekazaniem kontroli do obcej biblioteki. Każde wywołanie przez FFI jest z definicji niebezpieczne w rozumieniu kompilatora Rust, gdyż nie może on zweryfikować poprawności konwencji wywołania, czasu życia wskaźników ani nienaruszalności reguł własności po drugiej stronie granicy językowej.

Niskopoziomowym przykładem takiego podejścia w ekosystemie Rust jest biblioteka `ash` [2]. Dostarcza ona bezpośrednie odwzorowania wszystkich typów, funkcji i stałych specyfikacji Vulkana przez FFI, lecz bez jakichkolwiek gwarancji bezpieczeństwa: wszystkie wywołania wymagają bloków `unsafe`, zarządzanie czasem życia obiektów spoczywa w całości na programiście, a błędy takie jak użycie zniszczonego uchwytu czy brak wymaganej bariery pozostają niewidoczne dla kompilatora. Użycie `ash` jako bezpośredniej warstwy API w projekcie pisanym w Rust, gdzie kluczową motywacją jest statyczne bezpieczeństwo pamięci, czyniłoby tę motywację w znacznej mierze iluzoryczną.

Odpowiedzią na tę sprzeczność jest biblioteka *Vulkano* [53], niezależna wysokopoziomowa warstwa abstrakcji nad Vulkanem, tworzona od 2016 roku. Vulkano realizuje własne powiązania z API Vulkana, jednocześnie eksponując interfejs idiomatyczny dla języka Rust. Każdy obiekt Vulkana owinięty jest w typ Rust zarządzany przez licznik referencji, co przenosi gwarancje czasu życia do systemu typów: zniszczenie urządzenia logicznego przed zależnymi od niego buforami lub potokami staje się błędem kompilacji, a nie niezdefiniowanym zachowaniem w czasie wykonania. Konfiguracja zasobów odbywa się przez struktury realizujące wzorzec budowniczego (ang. *builder pattern*), które wymuszają podanie wszystkich wymaganych pól i odrzucają niespójne kombinacje parametrów już na etapie kompilacji, eliminując całą klasę błędów inicjalizacyjnych, które bez takiej ochrony ujawniałyby się dopiero przez warstwę walidacyjną Vulkana w czasie wykonania.

Najistotniejszą cechą Vulkano z punktu widzenia niniejszej pracy jest automatyczne zarządzanie synchronizacją po stronie GPU przez typ `AutoCommandBufferBuilder`. Jak opisano w sekcji dotyczącej Vulkana, ręczne programowanie barier potoku polece-

niem `vkCmdPipelineBarrier` jest obowiązkiem programisty: pominięcie bariery między zapisem obliczeniowym a odczytem graficznym prowadzi do wyścigu danych, trudnego do wykrycia i niedeterministycznego w objawach. `AutoCommandBufferBuilder` śledzi stan każdego zasobu w wewnętrznym grafie zależności i wstawia odpowiednie bariery automatycznie między kolejnymi operacjami rejestrowanymi w buforze poleceń, bez konieczności ręcznego określania etapów źródłowych i docelowych. W analogiczny sposób obsługiwane są semafory i obiekty typu fence powiązane z łańcuchem wymiany: zależności między kolejnymi przesłaniami są wykrywane automatycznie, a semafory zarządzane wewnętrznie, co uwalnia programistę od ręcznego pilnowania gotowości zasobów przed kolejną klatką.

Konsekwencją tych abstrakcji jest znaczna redukcja kodu infrastrukturalnego. Inicjalizacja pełnej ścieżki renderowania w czystym Vulkanie, obejmująca jawne tworzenie pul poleceń, pul deskryptorów, obiektów synchronizacji i ręczne zarządzanie przejściami obrazów, zastępowana jest wyrażeniami Rust korzystającymi z ergonomicznych interfejsów biblioteki. Pozwala to skoncentrować implementację na logice symulacyjnej zamiast na ceremoniałowym kodzie obsługi API, przy jednoczesnym zachowaniu pełnej niskopoziomowej kontroli nad wyborem urządzenia, kolejkami i układem pamięci, którą zapewnia Vulkan.

2.5 Narzędzia programistyczne

Niskopoziomowa natura Vulkana, będąca źródłem jego wydajności i elastyczności, pociąga za sobą istotną konsekwencję: API nie oferuje żadnych wbudowanych mechanizmów diagnostycznych. Sterownik nie sygnalizuje błędnego użycia przez wyjątki ani kody błędów o wysokiej granularności, a nieprawidłowa kolejność operacji lub brak wymaganej bariery synchronizacji może prowadzić do cichego wyścigu danych, nieokreślonego zachowania lub artefaktów wizualnych trudnych do powiązania z konkretną przyczyną w kodzie.

Częściowym rozwiązaniem tego problemu są warstwy walidacyjne Khronos Group, aktywowane przez rozszerzenie `VK_EXT_debug_utils`. Po włączeniu przechwytują one każde wywołanie API i weryfikują je względem specyfikacji, zgłaszając naruszenia przez skonfigurowany komunikator diagnostyczny. Nawet na bezpiecznej ścieżce oferowanej przez Vulkano pewne klasy błędów ujawniają się dopiero w czasie wykonania: nieprawidłowy układ deskryptorów, niezgodny format załącznika czy próba użycia zasobu w stanie niezgodnym z aktualną barierą potoku są przykładami sytuacji, w których komunikaty warstwy walidacyjnej stanowią jedyną bezpośrednią informację zwrotną od środowiska uruchomieniowego. Warstwy walidacyjne rozwiązują zatem problem poprawności użycia API, lecz nie wyczerpują całości potrzeb diagnostycznych złożonej

aplikacji graficznej.

Analiza wydajności poszczególnych kroków symulacji oraz wizualna inspekcja stanu bufora ramki w danym momencie są zagadnieniami, których warstwy walidacyjne w ogóle nie adresują. Vulkan nie dostarcza narzędzi do pomiaru czasu wykonania shaderów, identyfikacji wąskich gardeł w kolejkach poleceń ani do zatrzymania i odtworzenia sekwencji renderowania klatki. Lukę tę wypełniają zewnętrzne narzędzia profilowania i debugowania ramki, omówione w kolejnych podsekcjach.

2.5.1 Tracy Profiler

Warstwy walidacyjne informują o błędnym użyciu API, lecz nie odpowiadają na pytanie, dlaczego aplikacja działa wolno. W przypadku symulatora cieczy opartego na wielu kolejnych operacjach dispatch pytanie to ma fundamentalne znaczenie: bez precyzyjnych pomiarów nie można stwierdzić, który krok obliczeniowy dominuje czasowo, gdzie procesor czeka bezczynnie na zakończenie pracy GPU ani czy czas klatki jest zdominowany przez obliczenia na karcie graficznej czy przez narzut po stronie procesora głównego.

Tracy [57] jest profilatorem hybrydowym działającym w czasie rzeczywistym, zdolnym do rejestrowania zdarzeń z rozdzielczością nanosekundową zarówno po stronie procesora, jak i karty graficznej. Działa w modelu klient-serwer: aplikacja profilowana pełni rolę klienta i strumieniuje dane telemetryczne przez sieć do osobnego procesu serwera Tracy, który wizualizuje je na bieżąco w postaci interaktywnej osi czasu. Instrumentacja kodu polega na wstawieniu makr definiujących strefy pomiarowe wokół wybranych bloków kodu procesora. Analogiczna instrumentacja po stronie GPU odbywa się przez wstawienie zapytań czasowych do buforów poleceń Vulkana: Tracy rejestruje znaczniki czasowe GPU dla wybranych etapów i wyrównuje je z osią czasu procesora, umożliwiając precyzyjne porównanie obciążenia obu stron potoku.

Efektem jest oś czasu pokazująca równolegle aktywność wątków procesora i kolejek GPU, dzięki czemu identyfikacja wąskich gardeł sprowadza się do wizualnej inspekcji, gdzie pusta przestrzeń oznacza bezczynność, a nakładanie się stref wskazuje na potencjalne konflikty synchronizacji. Tracy nie rozwiązuje jednak problemów wizualnych: nie pozwala zatrzymać klatki i zbadać zawartości buforów ani prześledzić wykonania shadera dla konkretnego wierzchołka czy cząstki. Do tego celu służy osobne narzędzie opisane w kolejnej podsekcji.

2.5.2 RenderDoc

Artefakty wizualne, takie jak błędne pozycje cząstek, nieprawidłowe cieniowanie lub znikające fragmenty geometrii, są trudne do debugowania wyłącznie przez analizę

kodu, gdyż ich przyczyna może leżeć na dowolnym etapie potoku: w nieprawidłowych danych wejściowych shadera, błędnym stanie deskryptora, złym formacie obrazu lub błędnej logice samego shadera. Vulkan nie oferuje żadnego mechanizmu wstrzymania wykonania i inspekcji stanu GPU w arbitralnym momencie klatki.

RenderDoc [26] jest autonomicznym debuggerem ramki, dystrybuowanym na licencji MIT i wspieranym przez wszystkie główne API graficzne, w tym Vulkan. Po podłączeniu do uruchomionej aplikacji pozwala przechwycić pojedynczą klatkę naciśnięciem skrótu klawiszowego, bez modyfikacji kodu źródłowego. Zarejestrowane przechwycenie zawiera kompletny zapis wszystkich poleceń GPU wykonanych w danej klatce wraz z pełnym stanem potoku i zasobów w każdym momencie. Programista może przejść do dowolnego wywołania rysowania lub dispatch, obejrzeć zawartość buforów wejściowych i wyjściowych, sprawdzić aktywny zestaw deskryptorów, skontrolować układ pamięci obrazów oraz uruchomić debugger shadera śledzący wykonanie wybranego wątku obliczeniowego krok po kroku.

Narzędzie rozwiązuje zatem problem diagnostyki poprawności wizualnej i logicznej, który pozostaje poza zakresem zarówno warstw walidacyjnych, jak i Tracy. Jego ograniczeniem jest natomiast brak ciągłości: przechwycenie obejmuje wyłącznie jedną wybraną klatkę i nie zastępuje profilowania wydajnościowego rozciągniętego w czasie. Oba narzędzia są wobec siebie komplementarne i łącznie pokrywają przestrzeń diagnostyczną, której Vulkan celowo nie adresuje w swoim rdzeniu.

2.6 Wybór funkcji wygładzającej

Pojęcie funkcji wygładzającej i jej rola w metodzie SPH zostały wprowadzone w podrozdziale 1.2.2. W niniejszej sekcji przedstawiono dwa konkretne jądra rozważane w ramach niniejszej implementacji wraz z ich wzorami analitycznymi, właściwościami oraz wykresami wartości i gradientu.

2.6.1 Sześcienna funkcja sklejana

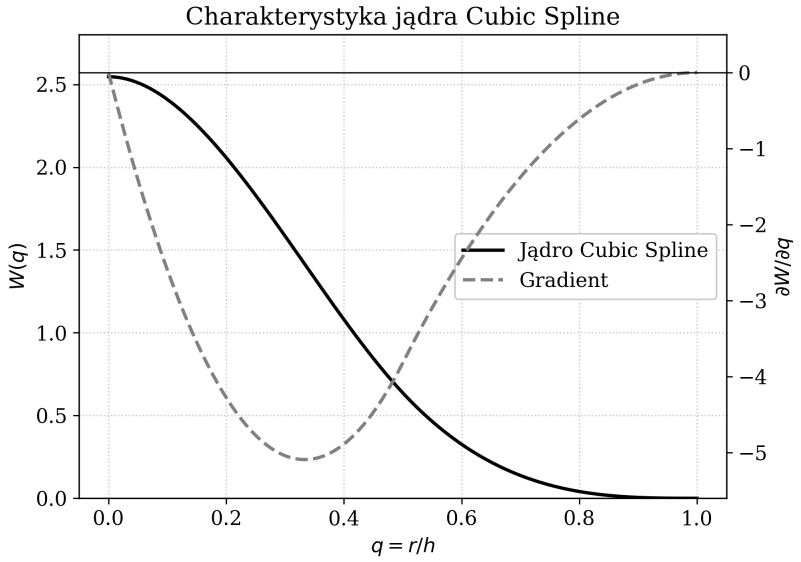
Sześcienna funkcja sklejana (ang. *cubic spline kernel*) pochodzi od Monaghana [38] i jest jednym z najszerzej stosowanych jąder w literaturze SPH. Dla trójwymiarowej przestrzeni jądro przyjmuje postać:

$$W(q,\, h) = \frac{8}{\pi h^3} \begin{cases} 6q^3 - 6q^2 + 1 & \text{dla } 0 \leqslant q \leqslant \frac{1}{2}, \\ 2(1-q)^3 & \text{dla } \frac{1}{2} < q \leqslant 1, , \\ 0 & \text{dla } q > 1, \end{cases} \tag{2.1}$$

gdzie $q = r/h$, $r = \|\mathbf{r}\|$ jest odległością między cząstkami, a h promieniem wygładzania. Gradient przestrzenny jądra wyrażony jest wzorem:

$$\nabla W(\mathbf{r},\, h) = \frac{48}{\pi h^3} \begin{cases} q(3q-2)\,\dfrac{\mathbf{r}}{r\,h} & \text{dla } 0 < q \leqslant \frac{1}{2}, \\ -(1-q)^2\,\dfrac{\mathbf{r}}{r\,h} & \text{dla } \frac{1}{2} < q \leqslant 1, \\ \mathbf{0} & \text{dla } q > 1. \end{cases} \tag{2.2}$$

Funkcja jest złożona z dwóch przedziałów wielomianowych sklejonych w punkcie $q = 0{,}5$ z zachowaniem ciągłości pierwszej i drugiej pochodnej. Dzięki temu gradient jest ciągły w całej dziedzinie nośnika, co jest istotne dla stabilności solvera ciśnienia. Jądro osiąga wartość zerową i ma zerową pochodną na granicy nośnika $q = 1$, co zapewnia gładkie zanikanie wpływu odległych cząstek.



Rysunek 2.8: Wartość $W(q)$ oraz pochodna radialna $\partial W/\partial q$ sześciennej funkcji sklejanej w funkcji znormalizowanej odległości $q = r/h$. Źródło: opracowanie własne na podstawie [30]

2.6.2 Funkcja Wendlanda C2

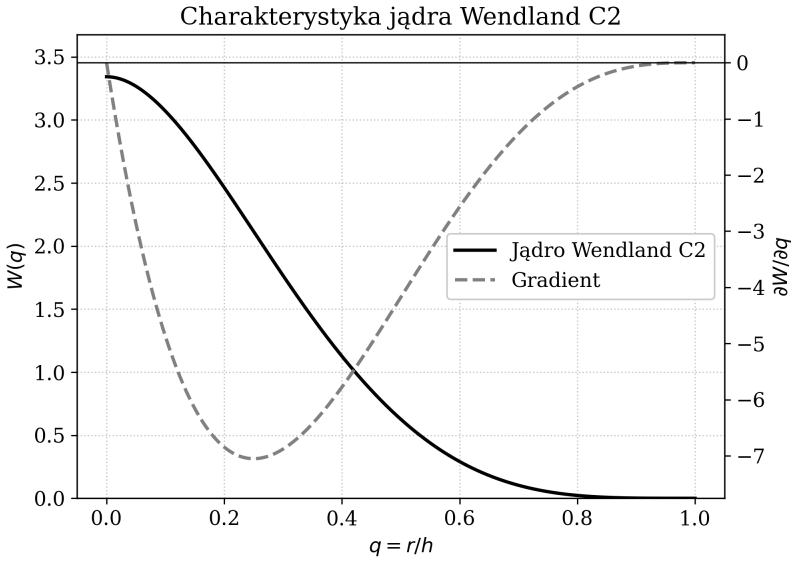
Jądro Wendlanda C2 (ang. *Wendland quintic C2 kernel*) pochodzi z teorii radialnych funkcji bazowych [55]. W trójwymiarowej przestrzeni przyjmuje postać wielomianu piątego stopnia:

$$W(q,\, h) = \frac{21}{2\pi h^3} \begin{cases} (1-q)^4(4q+1) & \text{dla } 0 \leqslant q \leqslant 1, \\ 0 & \text{dla } q > 1, \end{cases} \tag{2.3}$$

a jego gradient przestrzenny:

$$\nabla W(\mathbf{r},\, h) = -\frac{210}{\pi h^3} \begin{cases} q(1-q)^3 \, \dfrac{\mathbf{r}}{r\, h} & \text{dla } 0 < q \leqslant 1, \\ \mathbf{0} & \text{dla } q > 1. \end{cases} \tag{2.4}$$

W odróżnieniu od jądra sześciennego, funkcja Wendlanda C2 jest jednym wyrażeniem wielomianowym na całym nośniku, bez wewnętrznych punktów sklejenia. Posiada gwarantowaną dodatniość i monotonicznie malejący profil, a gradient zaczyna się od zera w $q = 0$, osiąga maksimum w pewnym $q \in (0, 1)$ i powraca do zera na granicy nośnika, co ilustruje rysunek 2.9.



Rysunek 2.9: Wartość $W(q)$ oraz pochodna radialna $\partial W/\partial q$ jądra Wendlanda C2 w funkcji znormalizowanej odległości $q = r/h$. Źródło: opracowanie własne na podstawie [55]

2.6.3 Dobór jądra dla solvera DFSPH

Porównując oba jądra pod kątem ich zachowania w pobliżu zera, widoczna jest zasadnicza różnica wynikająca bezpośrednio z rysunków 2.8 i 2.9. Gradient sześciennej funkcji sklejanej jest zerowy w punkcie $q = 0$, co oznacza, że gdy dwie cząstki znajdą się bardzo blisko siebie, siła odpychająca wynikająca z jądra zanika. Prowadzi to do efektu skupiania się cząstek (ang. *particle clumping*): cząstki, które raz do siebie nadmiernie się zbliżą, nie napotykają wystarczającego oporu obliczeniowego i zaczynają się zagęszczać w sposób niekontrolowany. Jądro Wendlanda C2 pozbawione jest tej wady: jego gradient rośnie niemal liniowo od zera przy $q = 0$ i nie posiada płaskiej strefy przy początku układu współrzędnych, co zapewnia niezerową siłę odpychającą nawet przy bardzo małych odległościach.

Wydawałoby się to wystarczającym argumentem na korzyść jądra Wendlanda C2, jednak badanie Lavrova i współpracowników [33] wskazuje na istotny koszt tej stabilności. Spośród dziesięciu analizowanych jąder SPH w przestrzeni trójwymiarowej, jądra Wendlanda należą do grupy wymagającej największej liczby cząstek sąsiadujących, aby zapewnić tę samą dokładność reprezentacji gęstości. Różnica w wymaganej liczbie sąsiadów między jądrami sześciennymi a jądrami Wendlanda wynosi w 3D nawet trzykrotność, przy czym kontrast ten pogłębia się wraz ze wzrostem liczby wymiarów. W implementacji GPU, gdzie każda operacja sąsiedztwa przekłada się bezpośrednio na liczbę instrukcji wykonywanych przez wątek obliczeniowy, szerszy nośnik efektywny jądra Wendlanda oznacza proporcjonalnie większy koszt obliczeniowy każdego kroku symulacji.

Kluczowym argumentem przesądzającym o wyborze sześciennej funkcji sklejanej jest właściwość algorytmu DFSPH. Solver rozbieżności, stanowiący rdzeń metody, wymusza w każdym kroku czasowym warunek zerowej rozbieżności pola prędkości, co zapobiega akumulacji błędów gęstości już u ich źródła. Cząstki nie mają możliwości nabycia prędkości zbliżającej je do siebie na tyle, by efekt skupiania się stał się problemem, gdyż solver koryguje prędkości zanim dojdzie do znacznego zaburzenia gęstości. Główna słabość sześciennej funkcji sklejanej jest zatem neutralizowana przez sam algorytm. Na zbieżność tego wniosku z praktyką wskazuje fakt, że autorzy algorytmu DFSPH w oryginalnej publikacji [6] sami posługują się sześcienną funkcją sklejaną jako jądrem obliczeniowym. Warto przy tym podkreślić dodatkowe ograniczenie wskazane przez tych samych autorów: oba solvery składające się na metodę DFSPH, solver gęstości oraz solver rozbieżności, muszą korzystać z tej samej funkcji wygładzającej [6]. Oznacza to, że wybór jądra nie jest decyzją lokalną dotyczącą jednego etapu algorytmu, lecz ma podwójny wpływ na koszt obliczeniowy całego kroku czasowego. Wybór jądra wymagającego szerszego sąsiedztwa, jak Wendland C2, obciążałby proporcjonalnie oba solvery jednocześnie. Wobec powyższego, w niniejszej implementacji przyjęto sześcienną funkcję sklejaną jako jądro wygładzające, czerpiąc korzyść z mniejszej liczby sąsiadów i tym samym niższego kosztu obliczeniowego przy zachowaniu pełnej stabilności numerycznej gwarantowanej przez solver DFSPH.

2.7 Wyszukiwanie sąsiedztwa cząstek

Każda cząstka w metodzie SPH oddziałuje wyłącznie z cząstkami znajdującymi się w odległości nie większej niż promień wygładzania h. Poza tym promieniem wartość jądra jest równa zero i dana para cząstek nie wnosi żadnego wkładu do sum interpolacyjnych. Wyznaczenie zbioru sąsiadów jest zatem fundamentalnym krokiem obliczeniowym wykonywanym co najmniej raz na krok czasowy, a przy iteracyjnych so-

lverach nieściśliwości, takich jak DFSPH, listy sąsiedztwa są wielokrotnie odczytywane w każdej klatce.

Naiwne podejście, polegające na sprawdzeniu każdej pary cząstek, ma złożoność $\mathcal{O}(N^2)$, gdzie N jest liczbą cząstek. Dla symulacji zawierającej 10^5 cząstek oznacza to 10^{10} porównań na krok, co jest w praktyce nieakceptowalne. Rozwiązaniem jest zastosowanie przestrzennej struktury danych, które ograniczają proces wyszukiwania do najbliższego otoczenia cząstki, redukując złożoność zapytania do $\mathcal{O}(1)$ lub $\mathcal{O}(\log N)$. Najszerzej stosowaną klasą takich struktur w metodzie SPH są siatki jednolite (ang. *uniform grids*), których rozmiar komórki dobierany jest równy promieniowi wygładzania h. Wówczas sąsiedzi danej cząstki mogą znajdować się wyłącznie w sąsiednich komórkach siatki, co w przestrzeni trójwymiarowej ogranicza przeszukiwany obszar do co najwyżej 27 komórek niezależnie od N.

2.7.1 Siatka haszująca według Simona Greena

Podejście zaproponowane przez Simona Greena [22] jest praktyczną realizacją idei siatki jednolitej przystosowaną do masowego zrównoleglenia na GPU. Przestrzeń symulacji dzielona jest na sześcienne komórki o boku równym h. Każda cząstka o pozycji $\mathbf{r}_i$ przypisywana jest do komórki o współrzędnych całkowitych:

$$(k,\, l,\, m) = \left\lfloor \frac{\mathbf{r}_i}{h} \right\rfloor, \tag{2.5}$$

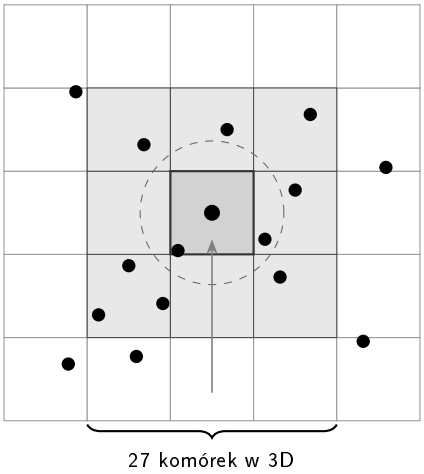
a następnie te trzy współrzędne całkowite odwzorowywane są na jeden indeks tablicy przez funkcję haszującą, co pozwala uniknąć jawnego przechowywania całej siatki w pamięci. Tablica haszująca przechowuje dla każdej komórki zakres indeksów cząstek posortowanej tablicy cząstek. Budowa struktury sprowadza się do posortowania cząstek według ich indeksu komórki, co na GPU realizowane jest przez szybkie sortowanie pozycyjne [22].

Wyszukiwanie sąsiadów dla cząstki i przebiega przez iterację po wszystkich 27 komórkach sąsiednich w trójwymiarowej siatce, pobranie odpowiednich zakresów z tablicy haszującej i sprawdzenie warunku $\|\mathbf{r}_i - \mathbf{r}_j\| < h$ dla każdego kandydata. Schemat tej procedury ilustruje rysunek 2.10.

Metoda zapewnia złożoność budowy $\mathcal{O}(N)$ i złożoność zapytania $\mathcal{O}(1)$ niezależną od liczby cząstek. Zakłada ona jednak z góry określony rozmiar tablicy haszującej i stałą domenę symulacji, co jest akceptowalne dla zamkniętych scen symulacyjnych.

2.7.2 Skompresowane listy sąsiedztwa jako alternatywa

Metodą stanowiącą bardziej nowoczesną alternatywę jest podejście zaproponowane przez Banda, Gisslera i Teschnera [3], będące wariantem struktury połączonych list



Rysunek 2.10: Schemat wyszukiwania sąsiedztwa metodą siatki haszującej w 2D. Źródło: opracowanie własne na podstawie [22]

komórek (ang. *Cell-Linked-List*, CLL). W odróżnieniu od podejścia Greena, siatka nie jest jawnie reprezentowana w pamięci: przechowywana jest jedynie zwarta lista niepustych komórek, co eliminuje koszt pamięciowy rzadko wypełnionych domen i umożliwia obsługę nieograniczonych dziedzin przestrzennych bez z góry zdefiniowanego rozmiaru siatki.

Kluczowym elementem metody jest posortowanie cząstek według indeksu komórki wyznaczonego krzywą Mortona, przestrzenną krzywą wypełniającą odwzorowującą współrzędne komórki na jednowymiarowy indeks tak, by sąsiednie przestrzennie cząstki trafiały do sąsiednich pozycji w tablicy. Poprawia to lokalność danych w pamięci podręcznej podczas iteracji po listach sąsiedztwa, co ilustruje rysunek 2.11.



Rysunek 2.11: Schemat struktury CLL z kompaktową tablicą komórek. Źródło: opracowanie własne na podstawie [3]

Drugą cechą wyróżniającą metodę jest kompresja list sąsiedztwa. Ponieważ cząstki są posortowane, indeksy sąsiadów są niemalejące, a ich różnice są małe. Autorzy wykorzystują tę właściwość, kodując różnice kolejnych indeksów sąsiadów za pomocą schematu inspirowanego algorytmem StreamVByte, osiągając oszczędność pamięci sięgającą 87% w porównaniu z nieskompresowaną reprezentacją. W iteracyjnych solverach nieściśliwości, takich jak DFSPH, gdzie listy sąsiedztwa są wielokrotnie odczytywane w każdym kroku, przechowywanie skompresowanych list i ich odczytywanie na żądanie

jest naturalnym rozwiązaniem. Metoda ta jest szczególnie korzystna dla dużych symulacji w otwartych scenach, gdzie domena nie jest z góry ograniczona, a siatka haszująca wymagałaby rozbudowanej tablicy. Jej implementacja opisana w [3] jest ukierunkowana na procesory wielordzeniowe; autorzy wskazują implementację GPU jako kierunek dalszych badań. Z tego względu, przy implementacji opartej w całości na GPU, zastosowano prostsze podejście siatki haszującej opisane w podrozdziale poprzednim.

2.8 Modelowanie zjawisk fizycznych

Równanie pędu Naviera-Stokesa zawiera trzy człony siłowe: gradient ciśnienia, naprężenia lepkie oraz siły zewnętrzne. Solver DFSPH, opisany w poprzednich rozdziałach, rozwiązuje wyłącznie człon ciśnieniowy, wymuszając nieściśliwość. Pozostałe zjawiska fizyczne, mianowicie opór lepki, kohezja cząstek na powierzchni swobodnej oraz oddziaływanie z granicami domeny, wymagają osobnych modeli dyskretnych. Każde z nich może być pominięte w minimalnej implementacji solvera, lecz każde odpowiada za inny aspekt fizycznej poprawności symulacji. W niniejszej sekcji omówiono teoretyczne podstawy tych modeli.

2.8.1 Lepkość

Lepkość jest miarą wewnętrznego oporu cieczy na odkształcenie ścinające. W równaniu pędu odpowiada za nią tensor naprężeń dewiacyjnych $\boldsymbol{\tau}$. Dla cieczy newtonowskich tensor ten jest proporcjonalny do symetrycznego gradientu prędkości:

$$\boldsymbol{\tau} = \mu \left(\nabla \mathbf{u} + (\nabla \mathbf{u})^{\top} \right), \tag{2.6}$$

gdzie μ jest dynamicznym współczynnikiem lepkości, a $\mathbf{u}$ — polem prędkości cieczy. Dywergencja tego tensora wchodzi do równania pędu jako przyspieszenie lepkie $\mathbf{a}^{\text{visc}}$. W dyskretyzacji SPH najczęściej stosowanym podejściem jest sformułowanie Morrisa i współpracowników [39], które zastępuje człon $\nabla \cdot \boldsymbol{\tau}$ operatorem opartym na laplasjanie prędkości:

$$\mathbf{a}_i^{\text{visc}} = \frac{\mu}{\rho_i} \sum_j \frac{m_j}{\rho_j} (\mathbf{v}_j - \mathbf{v}_i) \frac{1}{r_{ij}} \frac{\partial W_{ij}}{\partial r}, \tag{2.7}$$

gdzie $\mathbf{a}_i^{\text{visc}}$ jest przyspieszeniem lepkim cząstki i, μ — dynamicznym współczynnikiem lepkości, ρ_i oraz ρ_j — gęstościami cząstek, m_j — masą sąsiada, $\mathbf{v}_i$ oraz $\mathbf{v}_j$ — ich prędkościami, $r_{ij} = \|\mathbf{r}_i - \mathbf{r}_j\|$ — odległością między nimi, a $W_{ij} = W(\mathbf{r}_i - \mathbf{r}_j, h)$ — wartością jądra dla danej pary cząstek. Wyraz ten tłumi względne prędkości sąsiadów proporcjonalnie do odległości, modelując dyssypację energii kinetycznej charakterystyczną dla cieczy lepkich. Brak członu lepkiego sprawia, że ciecz zachowuje się jak

idealna, zachowując energię mechaniczną bez strat, co jest fizycznie nierealistyczne dla większości cieczy, choć nie destabilizuje solvera.

Odrębnym mechanizmem pokrewnym lepkości jest korekcja XSPH [37], w której prędkość cząstki jest nieznacznie uśredniana względem prędkości jej sąsiadów. Nie modeluje ona jawnie dyssypacji energii, lecz wygładza pole prędkości, zapobiegając przenikaniu i lokalnym skupieniom cząstek. Mechanizm ten działa niezależnie od fizycznej lepkości i może być stosowany zarówno przy $\mu = 0$, jak i przy niezerowej jej wartości.

2.8.2 Napięcie powierzchniowe

Napięcie powierzchniowe jest efektem makroskopowym niezrównoważonych sił kohezji działających na cząstki cieczy leżące przy granicy fazy swobodnej. Cząstka w głębi cieczy jest otoczona sąsiadami ze wszystkich stron i suma sił kohezji znosi się; cząstka na powierzchni ma sąsiadów tylko po stronie cieczy, co wytwarza wypadkową siłę skierowaną do jej wnętrza. Efektem jest minimalizacja pola powierzchni swobodnej i charakterystyczny kształt kropli.

W sformułowaniu opartym na równaniu Younga-Laplace'a [4] różnica ciśnienia Δp po obu stronach zakrzywionej powierzchni wynosi:

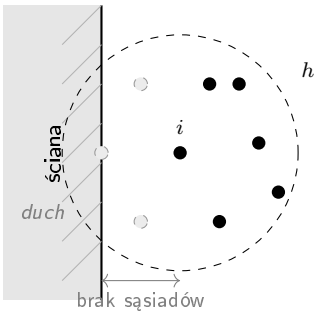
$$\Delta p = \gamma \kappa, \tag{2.8}$$

gdzie γ jest współczynnikiem napięcia powierzchniowego, a κ krzywizną powierzchni. W metodzie SPH krzywizna wyznaczana jest na podstawie pola normalnego szacowanego z gradientu jądra, a wypadkowa siła kohezji przykładana jest jako przyspieszenie cząstek leżących przy powierzchni. Bez napięcia powierzchniowego symulacja pozostaje stabilna, lecz traci zdolność do formowania kropli i realistycznego zachowania cienkiej warstwy cieczy. W niniejszej pracy napięcie powierzchniowe nie zostało zaimplementowane; symulowane sceny obejmują swobodną powierzchnię bez zjawisk kapilarnych, gdzie jego pominięcie nie wpływa istotnie na poprawność wyników [30].

2.8.3 Warunki brzegowe

Granice domeny symulacji wprowadzają fundamentalny problem geometryczny: cząstka znajdująca się w pobliżu ściany ma niepełny obszar wsparcia jądra, ponieważ część sfery sąsiedztwa o promieniu h wychodzi poza domenę i nie zawiera żadnych cząstek. Sytuację tę ilustruje rysunek 2.12.

Konsekwencje niekompletnego sąsiedztwa są wielorakie. Gęstość wyznaczana sumą SPH jest zaniżona względem wartości spoczynkowej ρ_0, co powoduje, że solver ciśnienia generuje w pobliżu ścian fałszywy gradient przyciągający cząstki do granicy. Gradient



Rysunek 2.12: Problem niepełnego sąsiedztwa przy granicy domeny. Źródło: opracowanie własne na podstawie [44]

ciśnienia jest dodatkowo błędnie skierowany, ponieważ suma po sąsiadach nie jest symetrycznie rozkładana wokół cząstki. Przy dużych krokach czasowych możliwe jest też przenikanie cząstek przez ścianę.

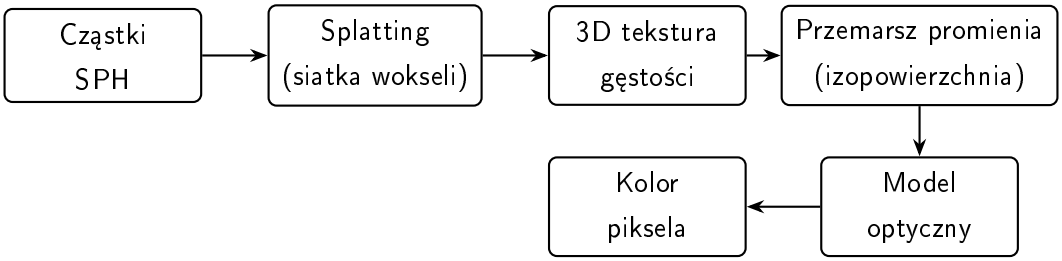
Powszechnym rozwiązaniem w literaturze są cząstki-duchy [44], tworzone symetrycznie po drugiej stronie granicy jako odbicia cząstek fluidu. Uzupełniają one brakujący obszar wsparcia, przywracając symetrię sumy SPH, i mogą być wyposażone w odwróconą prędkość normalną, modelując sprężyste lub lepkie odbicie od ściany. Alternatywą są stałe warstwy cząstek brzegowych reprezentujące geometrię przeszkody, których cząstki uczestniczą w sumach SPH, lecz nie podlegają integracji ruchu. W niniejszej pracy zastosowano uproszczone podejście: po każdym kroku całkowania pozycja cząstek przekraczających granicę domeny jest korygowana do jej wnętrza, a składowa normalna prędkości jest odbijana.

2.9 Potok wizualizacji

Wynikiem każdego kroku symulacji DFSPH jest zbiór N cząstek z zaktualizowanymi pozycjami. Cząstki stanowią dyskretne punkty masy; same w sobie nie tworzą żadnej ciągłej powierzchni, którą mógłby przetworzyć renderer. Aby uzyskać fotorealistyczny obraz cieczy, potok wizualizacji wykonuje trzy kolejne etapy: rekonstrukcję ciągłego pola gęstości ze zbioru punktów, wyznaczenie niejawnej izopowierzchni metodą przemarszu promienia oraz obliczenie koloru piksela z uwzględnieniem fizycznych właściwości optycznych wody. Ogólną strukturę potoku ilustruje rysunek 2.13.

2.9.1 Rekonstrukcja pola gęstości

Pierwszym krokiem potoku jest przekształcenie listy pozycji cząstek w trójwymiarową teksturę wolumetryczną [27]. Domenę symulacji dzieli się na regularną siatkę wokseli o rozdzielczości $G_x \times G_y \times G_z$, gdzie każdy woksel przechowuje skumulowaną



Rysunek 2.13: Potok wizualizacji: od listy cząstek SPH do końcowego koloru piksela. Źródło: opracowanie własne

gęstość wyrażoną liczbą całkowitą. Dla każdej cząstki i wyznaczany jest odpowiadający jej woksel centralny $\mathbf{c}_i$, a następnie do każdego woksela $\mathbf{c}$ w promieniu R wokseli dodawana jest waga:

$$w(\delta) = \left(1 - \left(\frac{\delta}{R}\right)^2\right)^3, \qquad \delta = \|\mathbf{c} - \mathbf{c}_i\|. \tag{2.9}$$

Funkcja ta jest gładkim wielomianem wygasającym do zera na granicy kuli, co gwarantuje ciągłość pola bez nieciągłości widocznych jako artefakty wizualne.

Ponieważ każda cząstka przetwarzana jest przez osobny wątek GPU, wiele wątków może jednocześnie aktualizować ten sam woksel. Konflikty zapisu rozwiązuje instrukcja atomowa `imageAtomicAdd`, która gwarantuje spójność sumy bez konieczności synchronizacji. Wynikiem etapu rzutowania cząstek na siatkę wokseli (ang. *splatting*) jest trójwymiarowa tekstura liczb całkowitych: woksele wewnątrz masy cieczy osiągają wartości rzędu setek, woksele powietrza zbliżone są do zera. Granica między tymi obszarami stanowi niejawną definicję powierzchni cieczy.

2.9.2 Wyznaczanie izopowierzchni metodą Raymarching

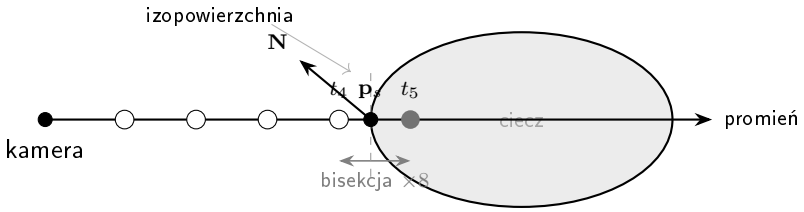
Mając pole gęstości $\rho(\mathbf{p})$ zapisane w teksturze 3D, powierzchnię cieczy definiuje się jako izopowierzchnię:

$$S = \left\{ \mathbf{p} \in \mathbb{R}^3 : \rho(\mathbf{p}) = \rho_{\text{thr}} \right\}. \tag{2.10}$$

Wyznaczenie przecięcia promienia kamery z tą powierzchnią realizuje przemarsz promienia [41, 52]: promień emitowany ze środka projekcji przemieszcza się równomiernymi krokami Δt wzdłuż swojego kierunku, w każdym punkcie próbkując teksturę, aż wartość gęstości przekroczy próg ρ_{thr} (gdzie S to zbiór punktów $\mathbf{p}$ przestrzeni $\mathbb{R}^3$ tworzących powierzchnię). Przed rozpoczęciem przemarszu promienia wykonywany jest test przecięcia promienia z prostopadłościanem wyrównanym do osi (ang. *Axis-Aligned Bounding Box*, AABB), czyli minimalnym prostopadłościanem wyrównanym do osi układu współrzędnych, który szczelnie otacza domenę symulacji. Test ten, oparty na

metodzie płaszczyzn równoległych, wyznacza w sposób analityczny zakres parametru $t \in [t_{\min}, t_{\max}]$ odpowiadający wejściu i wyjściu promienia z domeny; promienie nie trafiające w AABB odrzucane są na poziomie shadera fragmentów, zanim pętla przemarszu w ogóle się rozpocznie.

Próbkowanie tekstury odbywa się z ręczną interpolacją trójliniową ośmiu sąsiednich tekseli, zapewniającą ciągłość sygnału niezbędną do poprawnego wyliczenia normalnych. Aby uniknąć pominięcia cienkiej warstwy cieczy przy dużym kroku, po wykryciu pierwszego przekroczenia progu wykonuje się ośmiokrotną bisekcję przedziału $[t_{\text{prev}}, t_{\text{hit}}]$, lokalizując punkt powierzchni z dokładnością $\Delta t \,/\, 2^8$ bez zmniejszania globalnego kroku. Schemat algorytmu ilustruje rysunek 2.14.



Rysunek 2.14: Schemat przemarszu promienia z binarnym zawężaniem. Białe kółka to próbki poniżej progu gęstości, szare kółko to pierwsza próbka powyżej progu (ciecz). Bisekcja przedziału $[t_4, t_5]$ wyznacza punkt powierzchni $\mathbf{p}_s$ z dokładnością $\Delta t/2^8$. Źródło: opracowanie własne na podstawie [42]

Normalna powierzchni wyznaczana jest metodą różnic skończonych na polu gęstości [52]:

$$\mathbf{N}(\mathbf{p}) = -\nabla\rho(\mathbf{p}) \approx -\frac{1}{2\varepsilon}\begin{pmatrix} \rho(\mathbf{p}+\varepsilon\hat{x}) - \rho(\mathbf{p}-\varepsilon\hat{x}) \\ \rho(\mathbf{p}+\varepsilon\hat{y}) - \rho(\mathbf{p}-\varepsilon\hat{y}) \\ \rho(\mathbf{p}+\varepsilon\hat{z}) - \rho(\mathbf{p}-\varepsilon\hat{z}) \end{pmatrix}. \tag{2.11}$$

gdzie $\mathbf{N}(\mathbf{p})$ jest wektorem normalnym w punkcie $\mathbf{p}$, ε — krokiem różnicy skończonej (przesunięciem próbkowania pola gęstości), a $\hat{x}$, $\hat{y}$, $\hat{z}$ — wersorami osi układu współrzędnych. Podejście to nie wymaga jawnej siatki trójkątów i automatycznie odzwierciedla bieżącą topologię pola gęstości, co jest kluczowe przy dynamicznie zmieniającej się powierzchni cieczy.

2.9.3 Model optyczny cieczy

Woda jest dielektrycznym, częściowo transparentnym ośrodkiem, w którym na powierzchni jednocześnie zachodzi odbicie i załamanie światła, a wewnątrz objętości promienie ulegają absorpcji i rozproszeniu [40]. Poniżej omówiono kolejno efekty zaimplementowane w shaderze fragmentów.

Środowisko HDRI. Otoczenie sceny reprezentuje sferyczna mapa środowiskowa w formacie równokątnym (ang. *equirectangular*), próbkowana przez osobny shader nieba. Każde odbicie i załamanie na powierzchni wody próbkuje tę samą teksturę w przeliczonym kierunku, zapewniając spójność oświetlenia między cieczą a tłem.

Efekt Fresnela. Stosunek energii odbitej do załamanej zależy od kąta padania i właściwości dielektrycznych ośrodka [12]. Dla granicy powietrze–woda, o współczynnikach załamania odpowiednio $n_1 \approx 1{,}0$ oraz $n_2 = 1{,}333$, współczynnik odbicia przy normalnym kącie padania F_0 wynosi:

$$F_0 = \left(\frac{n_1 - n_2}{n_1 + n_2}\right)^2 \approx 0{,}02, \tag{2.12}$$

i wzrasta do jedności przy kącie ślizgowym. W shaderze stosuje się przybliżenie Schlicka:

$$F(\theta) = F_0 + (1 - F_0)(1 - \cos\theta)^5, \tag{2.13}$$

gdzie θ jest kątem między wektorem widoku $\mathbf{V}$ a normalną $\mathbf{N}$. Kierunek odbitego promienia próbkuje mapę środowiskową, kierunek załamanego wyznaczany jest prawem Snella. Końcowy kolor piksela to interpolacja liniowa między kolorem załamania a odbicia z wagą F.

Absorpcja Beer-Lamberta. Światło przenikające przez grubość cieczy d ulega wykładniczej absorpcji:

$$\mathbf{c}_{\text{abs}} = \mathbf{c}_{\text{bg}} \cdot \exp(-\boldsymbol{\mu}_a \cdot d), \tag{2.14}$$

gdzie $\mathbf{c}_{\text{abs}}$ jest barwą światła po przejściu przez ciecz, $\mathbf{c}_{\text{bg}}$ — kolorem tła widzianym wzdłuż promienia, a $\boldsymbol{\mu}_a = (\mu_R, \mu_G, \mu_B)$ — kanałowymi współczynnikami absorpcji. Silna absorpcja czerwonego ($\mu_R = 0{,}45$) przy słabej niebieskiego ($\mu_B = 0{,}025$) nadaje wodzie charakterystyczną niebieskozieloną barwę. Grubość d wyznaczana jest osobnym przebiegiem promienia od punktu powierzchni do tylnej ściany AABB.

Rozproszenie podpowierzchniowe. Część energii słonecznej nie jest pochłaniana, lecz wielokrotnie rozpraszana wewnątrz cieczy i wychodzi jako miękka turkusowa poświata widoczna zwłaszcza w cienkich partiach fali. Wynikowy wkład L_{sss} modeluje się przybliżeniem łączącym zawiniętą dyfuzję (ang. *wrapped diffuse*) z tylnym rozproszeniem:

$$L_{\text{sss}} = c_s \cdot L_{\odot} \cdot \Big(0{,}6\ \sigma_{\text{wrap}} + 0{,}4\ \sigma_{\text{back}}\Big) \cdot \Big(1 - e^{-3d}\Big), \tag{2.15}$$

gdzie c_s jest barwą rozproszenia podpowierzchniowego, $L_{\odot}$ — natężeniem światła słonecznego, $\mathbf{L}$ — znormalizowanym kierunkiem do źródła światła, a d — grubością cieczy na drodze promienia. Człon $\sigma_{\text{wrap}} = \max\Big(0,\ \mathbf{N}{\cdot}\mathbf{L}\cdot 0{,}5 + 0{,}5\Big)$ modeluje oświetlenie przechodzące przez krawędź fali, a $\sigma_{\text{back}} = \max(0,\ -\mathbf{N}{\cdot}\mathbf{L})$ odpowiada za tylne rozproszenie w kierunku obserwatora.

Odbłysk spekularny GGX. Mikrostruktury falującej powierzchni modeluje rozkład normalnych GGX [54] o parametrze szerokości $\alpha = 0{,}04$:

$$D_{\mathrm{GGX}}(\mathbf{H}, \alpha) = \frac{\alpha^2}{\pi \left[(\mathbf{N}\cdot\mathbf{H})^2(\alpha^2 - 1) + 1 \right]^2}, \tag{2.16}$$

gdzie $\mathbf{H} = \mathrm{normalize}(\mathbf{L} + \mathbf{V})$ jest półwektorem między kierunkiem światła a wektorem widoku, a α — parametrem szerokości rozkładu GGX odpowiadającym chropowatości powierzchni.

Piana i mapowanie tonalne. Regiony, w których normalna jest prawie pionowa ($N_y \approx 1$) przy małej grubości cieczy, maskowane są kolorem pianki, naśladując bąble powietrza na grzbietach fali. Na koniec kolor HDR przekształcany jest operatorem Reinharda [43] $\mathbf{c}' = \mathbf{c} \,/\, (1 + \mathbf{c})$ do zakresu $[0, 1]$ przed zapisem do bufora ramki.

Rozdział 3

Implementacja i oryginalne rozwiązania techniczne

Przejście od matematycznego opisu dynamiki cieczy do działającej implementacji obliczeniowej jest samo w sobie wyzwaniem inżynierskim. Równania różniczkowe opisujące zachowanie cieczy są precyzyjne i eleganckie; ich dyskretyzacja oraz wykonanie na rzeczywistym sprzęcie odsłaniają natomiast ograniczenia, które w modelu teoretycznym nie istnieją. Każdy krok symulacji musi zostać zdekomponowany na operacje równoległe wykonywane przez tysiące wątków GPU, a ta równoległość niesie ze sobą nowe klasy problemów nieobecne w algorytmach sekwencyjnych: równoczesny odczyt i zapis buforów, konieczność koherencji pamięci podręcznej, wrażliwość na wzorzec dostępu do pamięci globalnej.

Implementacja w modelu GPGPU (ang. *General-Purpose computing on Graphics Processing Units*) nie jest przepisaniem algorytmu sekwencyjnego na kod shaderów. Wymaga świadomego projektowania struktur danych pod kątem przepustowości pamięci, doboru algorytmów sortowania dostosowanych do architektury SIMT (ang. *Single Instruction, Multiple Threads*), zarządzania synchronizacją między etapami potoku obliczeniowego oraz kontroli arytmetyki zmiennoprzecinkowej, której błędy akumulacji mogą zdestabilizować pozornie poprawną symulację.

Niniejszy rozdział przedstawia implementację silnika symulacji cieczy opartego na metodzie DFSPH, realizowanej na GPU, w formie ciągu wyzwań i kompromisów technicznych. Każda decyzja projektowa — od wyboru struktury danych i algorytmu sortowania, przez strategię doboru kroku czasowego, po architekturę potoku renderowania — jest uzasadniona analizą strukturalną lub bezpośrednim pomiarem.

3.1 Założenia projektowe

Niniejsza praca stanowi implementację algorytmu DFSPH z wykorzystaniem nowoczesnego stosu technologicznego: języka Rust oraz API Vulkan. W odróżnieniu od omawianych systemów, projekt skupia się na pełnym przeniesieniu potoku obliczeniowego na procesor graficzny przy zachowaniu niskopoziomowej kontroli nad zasobami sprzętowymi. Bezpośrednie porównanie cech technicznych opisanych rozwiązań z systemem opracowanym w ramach pracy przedstawiono w tabeli 3.1. Wybór tego podejścia wynika z chęci zbadania granic wydajności i jakości, jakie można osiągnąć dzięki nowoczesnym technologiom programistycznym, oraz z potrzeby stworzenia elastycznej platformy do dalszych eksperymentów z algorytmami SPH.

Tabela 3.1: Porównanie istniejących rozwiązań symulacji cieczy z niniejszą pracą

Cecha	Blender	Unreal Engine 5	SPlisHSPlasH	Niniejsza praca
Metoda symulacji	FLIP	FLIP	SPH	SPH (DFSPH)
Czas rzeczywisty	Nie	Tak	Nie	Tak
Obliczenia GPU	Częściowe	Tak	Częściowe	Tak (w pełni)
Otwarte źródło	Tak	Tak	Tak	Tak
Język implementacji	C++	C++	C++	Rust
API graficzne	OpenGL	DX12 / Vulkan	OpenGL	Vulkan

Uruchomienie silnika wymaga spełnienia wymagań sprzętowych i programowych opisanych poniżej. Projekt jest przeznaczony wyłącznie dla systemu Windows — kod źródłowy korzysta z rozszerzenia `winit::platform::windows`, które udostępnia funkcjonalności specyficzne dla tej platformy (m.in. ikona na pasku zadań), co wyklucza kompilację na systemach Linux i macOS bez uprzedniej modyfikacji.

Wymagania sprzętowe i programowe zestawiono odpowiednio w tabelach 3.2 i 3.3.

Tabela 3.2: Wymagania sprzętowe

Komponent	**Wymaganie**
GPU	Obsługa Vulkan 1.3 (karty graficzne klasy GTX 1000 / RX 400 lub nowsze)
System operacyjny	Windows 10 w wersji 2004 lub nowszy
Sterownik GPU	Aktualny sterownik producenta z obsługą Vulkan 1.3

Tabela 3.3: Wymagania do kompilacji

Narzędzie	Wersja minimalna
Rust (edycja 2024)	1.85
Kompilator C++	MSVC (Visual Studio Build Tools)
Vulkan SDK	1.3.x (zawiera `shaderc`)

Kompilator C++ jest wymagany przez bibliotekę `tracy-client` z włączoną funkcją `enable`, która kompiluje bibliotekę profilowania Tracy ze źródeł C++. Shadery GLSL są kompilowane do formatu SPIR-V w czasie budowania projektu przez bibliotekę `vulkano-shaders`, korzystającą z narzędzia `shaderc` dołączonego do Vulkan SDK.

Silnik oczekuje obecności następujących plików w katalogu projektu:

- `assets/logo.png` — ikona okna i paska zadań,
- `assets/hdri/citrus_orchard_road_puresky_4k.exr` — panorama HDRI (ang. *High Dynamic Range Image*) — obraz o rozszerzonym zakresie dynamiki, przechowujący wartości jasności wykraczające poza standardowy zakres wyświetlania, dzięki czemu wiernie odwzorowuje zarówno jasne, jak i ciemne obszary — w formacie OpenEXR, używana jako środowisko oświetleniowe i tło sceny.

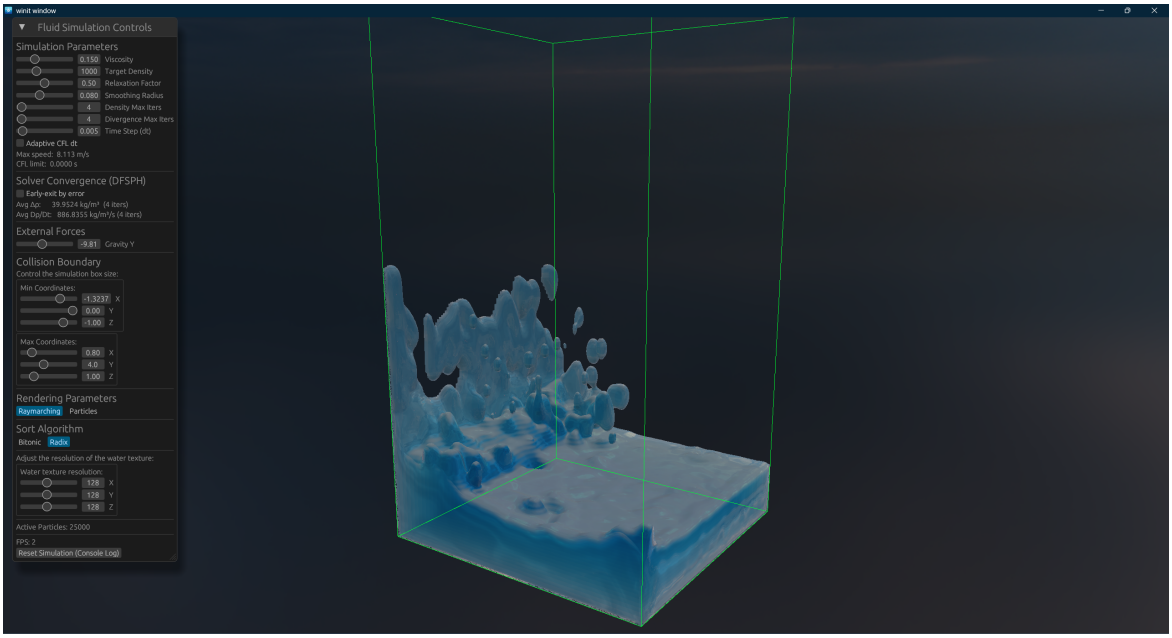
3.2 Prezentacja aplikacji

Aplikacja stanowi interaktywny silnik symulacji cieczy działający w czasie rzeczywistym, łączący metodę DFSPH z wolumetrycznym renderowaniem na GPU. Po uruchomieniu użytkownik widzi scenę 3D z prostopadłościennym obszarem symulacji, wewnątrz którego odbywa się symulacja cieczy, oraz panel konfiguracyjny umożliwiający modyfikację parametrów w czasie rzeczywistym (rysunek 3.3).

Motywacją do stworzenia projektu była chęć głębszego zrozumienia sposobu, w jaki współczesne silniki gier i aplikacji realizują symulacje cząsteczkowe — symulacja cieczy metodą SPH stanowi pod tym względem szczególnie reprezentatywny przypadek, ponieważ łączy w sobie wszystkie kluczowe wyzwania obliczeń równoległych: nieregularny dostęp do danych sąsiedztwa, iteracyjne solvery zbieżnościowe i konieczność ścisłej synchronizacji między setkami tysięcy wątków GPU. Realizacja projektu pozwoliła na praktyczne opanowanie programowania równoległego w modelu GPGPU oraz pracy z nowoczesnym interfejsem graficznym Vulkan, który wymaga jawnego za-

rządzania synchronizacją, pamięcią i potokami poleceń — w odróżnieniu od bardziej abstrakcyjnych API jak OpenGL czy DirectX 11.

Silnik może służyć jako platforma badawcza do testowania algorytmów symulacji cieczy, demonstracyjna aplikacja edukacyjna lub punkt wyjścia do integracji fizyki cieczy z systemem renderowania w czasie rzeczywistym.



Rysunek 3.1: Okno aplikacji podczas symulacji. Źródło: opracowanie własne

3.3 Architektura systemu

Silnik jest podzielony na dwie warstwy: warstwę CPU, która orkiestruje wykonanie i zarządza stanem aplikacji, oraz warstwę GPU, która realizuje wszystkie obliczenia fizyczne i renderowanie. Procesor nie wykonuje żadnych obliczeń symulacji — jego jedyną rolą jest zbudowanie buforów komend Vulkan i przesłanie ich do kolejki GPU. Podział modułów między obie warstwy przedstawia tabela 3.4.

Pętla ramki jest sterowana zdarzeniem `RedrawRequested` biblioteki `winit` i wykonuje się w trybie `ControlFlow::Poll`, co oznacza brak ograniczenia liczby klatek przez system operacyjny. Algorytm 3.1 przedstawia kolejność operacji w obrębie jednej klatki.

Architektura silnika opiera się na kilku abstrakcjach rozdzielających odpowiedzialności między komponentami. *ApplicationHandler* — cecha biblioteki `winit` — realizuje odwrócenie kontroli nad pętlą zdarzeń, omówione szczegółowo w podrozdziale 3.4.1.

[1] CFL — warunek Couranta–Friedrichsa–Lewy'ego (ang. *Courant–Friedrichs–Lewy condition*) [34]; szczegóły w podrozdz. 3.9.5

Algorithm 3.1: Pętla ramki

$\Delta t \leftarrow$ `fps_counter.tick()`;
$\Delta t_{\text{bezp}} \leftarrow \min(\Delta t,\ 0{,}1\,\text{s})$;
if okno sfokusowane **then**
    wykonanie komend z `Controller` na obiektach `Scene`;
`// Faza fizyki -- GPU`
odczytanie statystyk z poprzedniej klatki ($v_{\max}$, błąd gęstości, błąd dywergencji);
if tryb CFL[1] włączony i $v_{\max} > 0{,}01$ **then**
    $dt \leftarrow \text{clamp}(0{,}4 \cdot h/v_{\max},\ 0{,}001,\ 0{,}05)$;
    $dt \leftarrow \min(dt,\ dt_{\text{poprzedni}} \cdot 1{,}1)$;
uruchomienie `neighbor_search`, `density_alpha`;
$t \leftarrow 0$;
while $t < \Delta t_{bezp}$ **do**
    uruchomienie `viscosity`, `density_source_term`;
    for $i \leftarrow 1$ **to** N_ρ **do**
        uruchomienie `pressure_force`, `pressure_update`;
    uruchomienie `pressure_integration`;
    $t \leftarrow t + dt$;
    uruchomienie `neighbor_search`, `density_alpha`;
    uruchomienie `divergence_source_term`;
    for $i \leftarrow 1$ **to** $N_{\nabla v}$ **do**
        uruchomienie `pressure_force`, `pressure_update`;
    uruchomienie `divergence_integration`;
uruchomienie `stats`;
skopiowanie pozycji i kolorów cząstek do bufora renderowania;
`// Faza renderowania -- GPU`
if tryb przemarszu promienia **then**
    uruchomienie `density_texture`;
narysowanie nieba, wody (lub cząstek), granic kolizji;
nałożenie interfejsu użytkownika (egui);
zaprezentowanie klatki;

Tabela 3.4: Podział modułów systemu

Moduł	Platforma	Rola
`Engine`	CPU	Pętla zdarzeń `winit`, timing, zarządzanie sceną i wejściem
`Scene`	CPU	Parametry symulacji, pozycje startowe cząstek, kamera, granice kolizji
`Controller`	CPU	Odwzorowanie klawiatury i myszy na obiekty komend
`Renderer`	CPU	Budowanie buforów komend Vulkan, synchronizacja faz fizyki i renderowania
`GpuSceneResources`	GPU	Bufory cząstek, parametry symulacji, tekstury, dane kamery
`ComputePipelines`	GPU	11 potoków obliczeniowych realizujących solver DFSPH
`Pipelines`	GPU	Potoki graficzne: niebo, cząstki, woda (przemarsz promienia), granice

ComputeStep jest autorską abstrakcją nadającą wszystkim potokom obliczeniowym GPU jednolity interfejs, rozdzielającą jednorazową inicjalizację zasobów od wielokrotnie powtarzanego zapisu poleceń do bufora komend. Do obsługi wejścia zastosowano wzorzec projektowy *Command* [18]: każde naciśnięcie klawisza lub ruch myszy przekładane są na obiekt komendy, który wie, jak zmienić scenę, ale nie wie, skąd pochodzi sygnał. *Actor* to uproszczona koncepcja popularna w społeczności twórców gier — stosowana m.in. w Unreal Engine 5 [14] — abstrahująca przemieszczanie i obrót obiektu w przestrzeni 3D niezależnie od jego konkretnego typu.

3.4 Warstwa aplikacji

Zanim GPU wykona choćby jeden krok symulacji, warstwa CPU musi otworzyć okno, przetworzyć wejście użytkownika i zsynchronizować parametry sceny z pamięcią karty graficznej. Trzy moduły odpowiadają za te zadania: biblioteka `winit` zarządzająca oknem i pętlą zdarzeń, model kamery przekształcający ruch myszy w macierze widoku, oraz struktura sceny, która stanowi jedyne źródło parametrów przekazywanych do każdego shadera.

3.4.1 Zarządzanie oknem

Okno i pętla zdarzeń są obsługiwane przez bibliotekę `winit` za pośrednictwem cechy *ApplicationHandler*. Zamiast samodzielnej pętli `while`, silnik implementuje tę cechę i przekazuje sterowanie systemowi zdarzeń — jest to wzorzec odwrócenia kontroli, w którym to system operacyjny wywołuje metody aplikacji w odpowiedzi na zdarzenia.

Metoda `resumed()` należy do modelu cyklu życia *ApplicationHandler* i jest wywoływana za każdym razem, gdy aplikacja przechodzi ze stanu zawieszonego do aktywnego. Na platformach bez jawnego cyklu zawieszania, takich jak Windows, zdarzenie to jest emitowane jednorazowo po inicjalizacji i pełni rolę sygnału startu. Na platformach mobilnych może być wywoływana wielokrotnie — na Androidzie tworzenie powierzchni renderowania przed tym zdarzeniem jest niemożliwe. `Winit` zaleca [56] więc inicjalizację okna i kontekstu graficznego dopiero w odpowiedzi na pierwsze `Resumed`, co w kodzie przekłada się na tworzenie obiektów `Window` i `Renderer` wewnątrz tej metody. Metoda `window_event()` obsługuje wszystkie zdarzenia okna, a metoda `device_event()` przechwytuje ruch myszy jako surowe przesunięcia, niezależnie od pozycji kursora.

Silnik rozróżnia dwa stany okna: *aktywny* — kursor widoczny, brak obsługi wejścia z kamery — oraz *sfokusowany* — kursor ukryty i zablokowany, aktywna kamera. Przejście do trybu sfokusowanego następuje przez kliknięcie lewym przyciskiem myszy lub zdarzenie `Focused(true)`; powrót do trybu aktywnego — przez naciśnięcie klawisza Escape lub zdarzenie `Focused(false)`.

Pętla działa w trybie `ControlFlow::Poll`, co oznacza, że `RedrawRequested` jest generowane natychmiast po każdej iteracji bez oczekiwania na zdarzenia systemowe. Łańcuch wymiany używa trybu `PresentMode::Immediate`, który wyświetla ukończoną klatkę natychmiast, bez oczekiwania na sygnał synchronizacji pionowej. Liczba obrazów łańcucha wymiany wynosi 3; stała `MAX_FRAMES_IN_FLIGHT = 3` ogranicza liczbę ramek budowanych przez CPU równolegle z pracą GPU — razem tworzą pełne potrójne buforowanie.

3.4.2 Kamera

Kamera jest podzielona na dwie struktury. `Camera` przechowuje stan logiczny po stronie CPU; `CameraGPU` zawiera dane gotowe do przesłania na GPU. Pola obu struktur opisują tabele 3.5 i 3.6.

Obrót kamery stosuje podejście charakterystyczne dla kamer pierwszoosobowych. Obrót wokół osi Y odchylenie (ang. *yaw*) jest aplikowany w przestrzeni globalnej, a wokół osi X pochylenie (ang. *pitch*) — w przestrzeni lokalnej kamery:

$$q_{\text{new}} = q_{\text{yaw}} \cdot q_{\text{current}} \cdot q_{\text{pitch}}. \tag{3.1}$$

Tabela 3.5: Pola struktury `Camera`

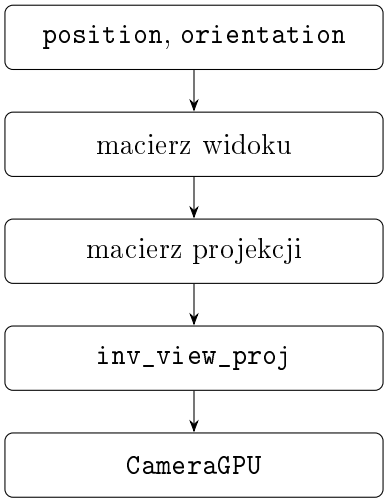
Pole	Typ	Opis
`position`	`Vec3`	Położenie obserwatora w przestrzeni świata
`orientation`	`Quat`	Orientacja przestrzenna; kwaternion jest standardową reprezentacją obrotu w grafice 3D
`fov`	`f32`	Kąt widzenia (ang. *field of view*, 45°) — kąt między lewą a prawą krawędzią pola widzenia; większy kąt daje szerszy widok, ale silniejsze zniekształcenie perspektywiczne na brzegach
`aspect_ratio`	`f32`	Proporcje ekranu, domyślnie 16/9
`near` / `far`	`f32`	Bliższa i dalsza płaszczyzna obcinania (ang. *clipping planes*, 0.1 m i 1000.0 m) — przedział odległości, w którym GPU renderuje geometrię; obiekty spoza tego przedziału są pomijane

Tabela 3.6: Pola struktury `CameraGPU` przesyłanej na GPU

Pole	Typ	Opis
`view`	`Mat4`	Macierz widoku — przekształca punkty z przestrzeni świata do przestrzeni kamery
`proj`	`Mat4`	Macierz projekcji — odwzorowuje przestrzeń kamery na NDC
`inv_view_proj`	`Mat4`	Odwrotność $(\mathbf{PV})^{-1}$ — używana przez przemarsz promienia do odtworzenia kierunku promienia
`camera_pos`	`Vec3`	Pozycja kamery w przestrzeni świata (+ 4 bajty wyrównania)

Dzięki temu linia horyzontu pozostaje zawsze pozioma, a obrót nie kumuluje skręcenia wzdłuż osi Z.

Bufor kamery jest tworzony z flagą `SHADER_DEVICE_ADDRESS`, co pozwala przekazywać go do shaderów jako surowy adres GPU zamiast wiązać przez deskryptor. Macierz widoku wyznaczana jest funkcją `look_at_lh` na podstawie pozycji i orientacji kamery, macierz projekcji — funkcją `perspective_lh`, a `inv_view_proj` [20] stanowi odwrotność ich iloczynu $(\mathbf{PV})^{-1}$. Rysunek 3.2 ilustruje kolejność tych obliczeń.



Rysunek 3.2: Łańcuch obliczeń kamery. Źródło: opracowanie własne

Macierz `inv_view_proj` jest potrzebna dla przemarszu promienia. Shader operuje na współrzędnych NDC (ang. *Normalized Device Coordinates*) — układzie, w którym środek ekranu to (0, 0), a narożniki (± 1, ± 1). Aby wyznaczyć kierunek promienia dla danego piksela, shader mnoży jego współrzędne NDC przez `inv_view_proj` i odejmuje pozycję kamery — bez żadnych dodatkowych przesyłań danych między CPU a GPU.

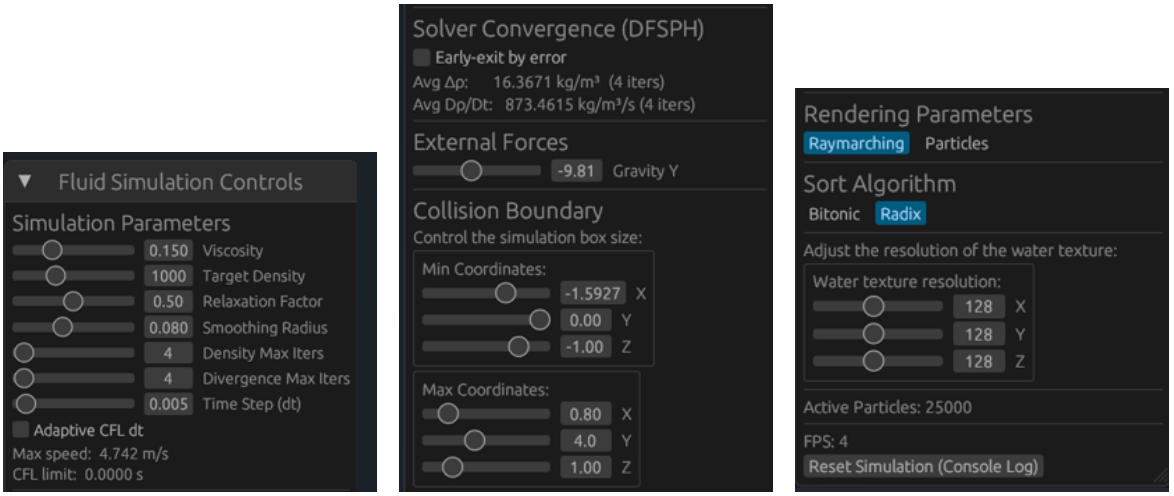
Vulkan definiuje układ NDC z osią Y skierowaną w dół [47], odwrotnie niż OpenGL, co powoduje, że obraz renderowany standardową macierzą projekcji jest odwrócony pionowo. Zamiast modyfikować macierze, problem rozwiązuje odwrócenie okienka renderowania (ang. *viewport*) — prostokątnego obszaru okna, na którym GPU wyświetla wynik. W implementacji ustawiono ujemną wysokość okienka przy przesuniętym punkcie startowym, co przesuwa układ do konwencji Y-w-górę bez żadnych zmian w shaderach ani macierzach.

3.4.3 Scena i parametry symulacji

Struktura `Scene` grupuje cztery elementy: wektor startowych pozycji cząstek, strukturę `SimulationParams`, obiekt kamery oraz granice kolizji (`CollisionBox`). Jest jedynym źródłem danych konfiguracyjnych — wszystkie shadery odczytują parametry

wyłącznie z bufora `sim_params_buffer`, który jest odświeżany na początku każdej klatki wywołaniem `sync_with_scene()`.

Tabela 3.7 zestawia parametry struktury `SimulationParams` wraz z wartościami domyślnymi ustawionymi w konstruktorze `Scene::new()`; rysunek 3.3 przedstawia wygląd panelu konfiguracyjnego w interfejsie `egui` podczas działania aplikacji.



(a) (b) (c)

Rysunek 3.3: Panel parametrów symulacji w interfejsie `egui`: (a) parametry symulacji, (b) zbieżność solvera DFSPH, siły zewnętrzne i granice kolizji, (c) parametry renderowania, algorytm sortowania i rozdzielczość tekstury. Źródło: opracowanie własne

Tabela 3.7: Parametry symulacji i ich wartości domyślne

Parametr	Wartość domyślna	Opis
`particle_radius`	0.020 m	Promień cząstki
`smoothing_radius`	0.080 m	Zasięg jądra SPH ($= 4r$)
`target_density`	1000 kg/m^3	Gęstość referencyjna
`dt`	0.005 s	Stały krok czasowy
`viscosity`	0.15	Współczynnik XSPH
`relax_factor`	0.5	Współczynnik relaksacji solvera
`density_solver_iterations`	4	Iteracje korekcji gęstości
`divergence_solver_iterations`	4	Iteracje korekcji dywergencji
`gravity`	$(0, -9{,}81, 0)$ m/s^2	Przyspieszenie grawitacyjne
`grid_res`	128^3	Rozdzielczość siatki przestrzennej

3.5 Inicjalizacja GPU z Vulkano

Konfiguracja silnika Vulkan skupia się wokół dwóch elementów: doboru minimalnego zestawu funkcji urządzenia wymaganych przez solver DFSPH oraz autorskiej cechy `ComputeStep` — oryginalnej warstwy abstrakcji, która nadaje wszystkim potokom obliczeniowym jednolity cykl życia: jednorazową kompilację shadera przy starcie i ustandaryzowany zapis poleceń do bufora komend co klatkę.

3.5.1 Kontekst Vulkan

W podrozdziale 2.3.1 opisano hierarchię inicjalizacyjną Vulkan — instancja, urządzenie fizyczne, urządzenie logiczne, kolejki — jako sekwencję jawnych operacji. W kodzie ta hierarchia jest ukryta za strukturą `VulkanoConfig`: silnik wypełnia ją listą potrzebnych funkcji i rozszerzeń, a `VulkanoContext::new()` przeprowadza całą inicjalizację, wybierając urządzenie i budując obiekty niskopoziomowe.

Wymagania solvera narzuciły trzy funkcje obliczeniowe. `buffer_device_address` pozwala shaderom pobierać dane bezpośrednio ze wskazanego adresu w pamięci GPU zamiast przez deskryptor — bufor kamery jest przekazywany jako wskaźnik bezpośredni, co eliminuje konieczność deklarowania zestawu deskryptorów dla jednego bufora. Rozszerzenie `scalar_block_layout` wymusza wyrównanie pól struktur GLSL zgodne z układem C — bez tej funkcji kompilator GLSL może wstawiać wypełnienie (ang. *padding*) między polami inaczej niż Rust, co powoduje odczyt błędnych wartości z buforów fizyki. `shader_int64` jest potrzebny do operacji indeksowania w shaderach obliczeniowych.

Warstwa renderowania wprowadziła dodatkowe wymagania techniczne. Funkcja `dynamic_rendering` — opisana w podrozdziale 2.3.3 jako alternatywa dla jawnych obiektów RenderPass — eliminuje konieczność deklarowania kompatybilności potoków z konkretnym formatem bufora ramki. `large_points` i `fill_mode_non_solid` pozwalają przełączać tryb wizualizacji cząstek — renderowanie jako punkt, bryłę lub siatkę — bez przebudowy potoków. Filtrowanie anizotropowe (ang. *anisotropic filtering*), aktywowane przez `sampler_anisotropy`, kompensuje rozmycie tekstur oglądanych pod ukośnym kątem — bez niego panorama HDRI traci szczegółowość w obszarach, gdzie promień kamery pada pod małym kątem do powierzchni. Rozszerzenie instancji `ext_debug_utils` aktywuje warstwy walidacyjne opisane w sekcji 2.5 — w trakcie tworzenia silnika wielokrotnie ujawniały one błędy synchronizacji niewidoczne w standardowym wyjściu.

Obsługę łańcucha wymiany i powierzchni renderowania powierzono komponentowi `VulkanoWindowRenderer` [53]. Definiuje on ścisły cykl życia klatki oparty na parze

operacji `acquire` i `present`. Metoda `acquire` pobiera indeks dostępnego obrazu z łańcucha wymiany i zwraca obiekt synchronizacji; po zarejestrowaniu i przesłaniu wszystkich buforów poleceń wywołanie `present` kolejkuje obraz do wyświetlenia, przekazując gotowy wynik do menedżera okien systemu operacyjnego.

3.5.2 Wzorzec ComputeStep

Gdy kontekst Vulkan jest skonfigurowany, silnik dysponuje urządzeniem i kolejkami potrzebnymi do tworzenia potoków obliczeniowych. Każdy potok wymaga tej samej sekwencji inicjalizacyjnej: załadowania shadera SPIR-V, wyprowadzenia układu deskryptorów z jego refleksji i kompilacji obiektu `ComputePipeline`. Cecha `ComputeStep` hermetyzuje tę sekwencję w domyślnej implementacji `new()`; implementator musi dostarczyć jedynie dwie metody — ładującą skompilowany shader oraz opakowującą gotowy obiekt potoku. Listing 3.1 przedstawia pełną definicję cechy.

```
pub trait ComputeStep: Sized {
    fn load_shader_module(device: Arc<Device>) -> EntryPoint;
    fn from_pipeline(pipeline: Arc<ComputePipeline>) -> Self;
    fn new(device: Arc<Device>) -> Self {
        let entry_point = Self::load_shader_module(device.clone());
        let stage = PipelineShaderStageCreateInfo::new(entry_point);
        let layout = PipelineLayout::new(
            device.clone(),
            PipelineDescriptorSetLayoutCreateInfo::from_stages(
                [&stage])
                .into_pipeline_layout_create_info(device.clone())
                .unwrap(),
        ).unwrap();
        let pipeline = ComputePipeline::new(
            device.clone(), None,
            ComputePipelineCreateInfo::stage_layout(stage, layout),
        ).unwrap();
        Self::from_pipeline(pipeline)
    }
    fn prepare(&mut self,
               allocator: Arc<StandardDescriptorSetAllocator>,
               physics_data: &GpuPhysicsData,
               sim_params: &Subbuffer<SimulationParams>);
    fn execute<Cb>(&self,
                   builder: &mut AutoCommandBufferBuilder<Cb>);
}
```

Listing 3.1: Cecha `ComputeStep`

Inicjalizacja potoku przebiega w trzech krokach. Najpierw bajtkod SPIR-V shadera — skompilowany już w czasie budowania projektu przez `vulkano-shaders` — jest ładowany do pamięci i przekazywany jako punkt wejścia identyfikujący funkcję główną shadera. Następnie `PipelineDescriptorSetLayoutCreateInfo::from_stages` analizuje ten bajtkod i automatycznie wyprowadza układ deskryptorów — tabelę opisującą GPU, ile buforów i tekstur potok odczytuje oraz pod jakimi numerami wiązań je znajdzie; bez tej refleksji programista musiałby deklarować tę tabelę ręcznie, duplikując informacje już zawarte w shaderze. Na końcu Vulkan kompiluje shader do natywnych instrukcji karty graficznej i tworzy obiekt `ComputePipeline`, który pozostaje gotowy przez cały czas życia aplikacji — ta kompilacja zdarza się tylko raz, przy starcie.

Metoda `prepare()` jest wywoływana jednorazowo przy starcie — przydziela zestaw deskryptorów i trwale wiąże z nim bufory fizyki; deskryptory pozostają ważne przez cały czas życia aplikacji, bo bufory nie są realokowane. Co klatkę wywoływane jest wyłącznie `execute()`, które zapisuje do buildera komend pojedyncze polecenie dispatchu — żądanie uruchomienia na GPU określonej liczby grup wątków obliczeniowych — bez żadnych alokacji ani zmiany stanu deskryptorów.

Razem implementatory cechy `ComputeStep` tworzą kompletny potok obliczeniowy odpowiadający kolejnym etapom pętli ramki z algorytmu 3.1. Tabela 3.8 zestawia wszystkich implementatorów wraz z ich rolami.

3.5.3 Deskryptory i synchronizacja

Deskryptor (ang. *descriptor*) jest uchwytem wiążącym bufor lub teksturę z punktem wiązania (ang. *binding*) zadeklarowanym w shaderze. Aby GPU wiedziało, z jakiej pamięci pobrać dane podczas wykonania shadera, każde polecenie dispatchu musi być poprzedzone dowiązaniem zestawu deskryptorów.

Układ zestawu deskryptorów każdego potoku wyprowadzany jest automatycznie przez vulkano z refleksji skompilowanego shadera SPIR-V — bez ręcznej deklaracji typów ani rozmiarów wiązań w kodzie Rust. Metoda `prepare()` alokuje zestaw deskryptorów ze wspólnego `StandardDescriptorSetAllocator` i wiąże z nim bufory fizyki pobrane z `GpuPhysicsData`; ponieważ alokacja następuje wyłącznie w `prepare()`, metoda `execute()` nie wykonuje żadnych operacji na stercie.

Między kolejnymi dispatchami niezbędne są bariery pamięci (ang. *pipeline barriers*), wymuszające widoczność zapisów wykonanych przez poprzedni potok przed odczytem przez następny. `AutoCommandBufferBuilder` wpisuje te bariery bezpośrednio do bufora komend po każdym poleceniu dispatchu, gwarantując spójność danych w całym potoku obliczeniowym.

Tabela 3.8: Implementatory cechy `ComputeStep`

Implementator	**Rola**
`GpuSorter`	Sortowanie cząstek po kluczu siatki przestrzennej
`NeighborSearch`	Budowanie siatki przestrzennej; deleguje sortowanie do `GpuSorter`
`DensityAlphaPipeline`	Gęstość SPH i współczynnik α
`ViscosityPipeline`	Lepkość XSPH i grawitacja
`DensitySourceTermPipeline`	Wyraz źródłowy korekcji gęstości
`PressureForcePipeline`	Siły ciśnienia w iteracji solvera
`PressureUpdatePipeline`	Aktualizacja pola ciśnienia
`PressureIntegrationPipeline`	Całkowanie prędkości po korekcji gęstości
`DivergenceSourceTermPipeline`	Wyraz źródłowy korekcji dywergencji
`DivergenceIntegrationPipeline`	Całkowanie prędkości po korekcji dywergencji
`StatsPipeline`	Statystyki klatki: $v_{\max}$, błędy gęstości i dywergencji
`DensityTexturePipeline`	Zapis gęstości do tekstury 3D

3.6 Struktury danych cząstek

Zanim zbudowany zostanie potok obliczeniowy, konieczne jest podjęcie decyzji dotyczącej organizacji danych cząstek w pamięci GPU. Wybór ten determinuje zarówno poprawność symulacji — ze względu na model pamięci Vulkan — jak i wydajność obliczeniową, poprzez wpływ na wzorzec dostępu do pamięci podręcznej procesorów strumieniowych.

3.6.1 Organizacja danych: AOS i SOA

Organizacja danych cząstek w pamięci GPU decyduje o wydajności każdego kroku obliczeniowego — shaderowy solver SPH odczytuje atrybuty kilkudziesięciu sąsiadów dla każdej z tysięcy cząstek w każdej klatce. Istnieją dwa fundamentalne układy: AOS (ang. *Array of Structures*), w którym wszystkie atrybuty jednej cząstki leżą obok siebie, oraz SOA (ang. *Structure of Arrays*), gdzie każdy atrybut tworzy osobny bufor.

Listing 3.2 przedstawia hipotetyczny układ AOS zastosowany w pierwszej wersji silnika, a listing 3.3 pokazuje faktyczny układ SOA, do którego przejście okazało się konieczne.

```
struct Particle {
    position: [f32; 4],  // 16 B
    velocity: [f32; 4],  // 16 B
    density:  f32,       //  4 B
    factor:   f32,       //  4 B
    pressure: f32,       //  4 B
}
// bufor: [Particle; N] -- atrybuty przeplatane co 44 B
```

Listing 3.2: Hipotetyczny układ AOS (porzucony)

```
pub struct GpuPhysicsData {
    pub count: u32,

    // podwojne buforowanie pozycji i predkosci
    pub position_a: Subbuffer<[[f32; 4]]>,
    pub position_b: Subbuffer<[[f32; 4]]>,
    pub velocity_a: Subbuffer<[[f32; 4]]>,
    pub velocity_b: Subbuffer<[[f32; 4]]>,

    pub colors:     Subbuffer<[[f32; 4]]>,

    // pola fizyczne solvera DFSPH
    pub densities:              Subbuffer<[f32]>,
    pub factors:                Subbuffer<[f32]>,
```

```
15      pub source_terms:             Subbuffer<[f32]>,
16      pub pressures:                Subbuffer<[f32]>,
17      pub pressure_accelerations:   Subbuffer<[[f32; 4]]>,
18
19      // siatka przestrzenna (wyszukiwanie sasiadow)
20      pub grid_entries: Subbuffer<[Entry]>,
21      pub grid_start:   Subbuffer<[u32]>,
22
23      // v_max, blad gestosci, blad dywergencji
24      pub stats_buffer: Subbuffer<[u32]>,
25  }
```

Listing 3.3: Kompletna struktura `GpuPhysicsData` – układ SOA

Bezpośrednią przyczyną porzucenia układu AOS był błąd ujawniający się natychmiast po uruchomieniu pierwszej wersji solvera. Shader obliczający nowe prędkości cząstek odczytywał prędkości sąsiednich cząstek i jednocześnie zapisywał zaktualizowane wartości do tego samego bufora. Vulkan traktuje taki wzorzec odczytu-zapisu bez odpowiedniej bariery jako nieokreślone zachowanie — wynik zależy od wewnętrznej kolejności wykonania wątków GPU, która nie jest gwarantowana. W praktyce część wątków odczytywała już zaktualizowane wartości sąsiadów zamiast wartości sprzed kroku, co prowadziło do niestabilnych wyników lub zgłoszenia błędu walidacji przez bibliotekę Vulkano.

Układ SOA rozwiązuje ten problem strukturalnie: każdy atrybut fizyczny zajmuje osobny bufor, a pozycje i prędkości wyposażono w dodatkowe bufory umożliwiające technikę podwójnego buforowania opisaną w następnej podsekcji. Dodatkowym argumentem za SOA jest wydajność pamięci podręcznej GPU. Shader obliczający gęstość potrzebuje wyłącznie pozycji sąsiednich cząstek — w układzie AOS załadowanie ich dla ośmiu sąsiadów wymagałoby pobrania ośmiu bloków pamięci oddalonych od siebie o rozmiar całego rekordu `Particle`, wypełniając linie pamięci podręcznej prędkościami i ciśnieniami, które w tym shaderze są bezużyteczne. W układzie SOA wszystkie pozycje leżą gęsto w jednym ciągłym buforze; procesory strumieniowe GPU ładują je sekwencyjnie, maksymalizując wykorzystanie przepustowości. Separacja atrybutów jest ponadto zgodna z modelem wiązań Vulkan — każde wiązanie (ang. *binding*) odpowiada jednemu buforowi, a sterownik może planować ładowanie danych z różnych buforów niezależnie.

3.6.2 Bufory cząstek na GPU

Spośród wszystkich buforów struktury `GpuPhysicsData` to pozycje i prędkości wymagają szczególnego traktowania. Każdy shader SPH odczytuje dane sąsiednich czą-

stek i jednocześnie zapisuje zaktualizowane wartości dla cząstki bieżącej; gdyby obie operacje trafiały do tego samego bufora, wynik zależałby od wewnętrznej kolejności wykonania wątków GPU, która nie jest przez Vulkan gwarantowana. Rozwiązaniem jest podwójne buforowanie, zwane w kontekście potoków obliczeniowych mechanizmem *ping-pong*.

W każdym kroku symulacji bufor `_a` stanowi stabilne źródło danych: potok wyszukiwania sąsiadów oblicza na jego podstawie klucze przestrzenne cząstek i po posortowaniu przepisuje dane do `_b` w kolejności sprzyjającej lokalności odczytu podczas iteracji solvera. Cały algorytm DFSPH — od wyznaczania gęstości ρ_i i współczynnika α_i, przez kolejne iteracje korekcji ciśnienia, po całkowanie prędkości — operuje wyłącznie na buforze `_b`, dzięki czemu żaden wątek nie czyta danych już nadpisanych przez inny. Na zakończenie kroku zaktualizowane wartości trafiają z `_b` z powrotem do `_a`, zamykając cykl.

Bufor `stats_buffer` pełni odmienną rolę: po każdym kroku zbiera na GPU maksymalną prędkość cząstek $v_{\max}$ oraz skumulowane błędy solvera gęstości i dywergencji, które CPU odczytuje przed kolejną klatką i wykorzystuje do regulacji kroku czasowego CFL.

3.7 Wyszukiwanie sąsiadów

Złożoność naiwnego wyszukiwania par w metodzie SPH oraz wynikające z niej ograniczenia omówiono w podrozdziale 2.7; niniejsza sekcja przedstawia implementację rozwiązania, które umożliwia efektywne wyszukiwanie sąsiadów w czasie stałym. Zastosowane rozwiązanie opiera się na technice haszowania przestrzennego opisanej przez Greena [22] i zrealizowanej jako czterostopniowy potok shaderów obliczeniowych, który przekształca nieuporządkowany zbiór cząstek w strukturę umożliwiającą przeszukiwanie otoczenia każdej cząstki w czasie stałym.

3.7.1 Haszowanie przestrzenne

Przestrzeń symulacji podzielona jest na jednolitą siatkę sześciennych komórek o boku równym promieniowi wygładzania h. Komórkę, do której należy cząstka o pozycji $\mathbf{p}_i$, wyznacza operacja $\mathbf{c}_i = \lfloor \mathbf{p}_i / h \rfloor$ wykonana składowo, a wynikowa trójka liczb całkowitych (c_x, c_y, c_z) jest odwzorowana na pojedynczy indeks tablicy funkcją haszującą:

$$\mathrm{hash}(\mathbf{c}) = \Big(c_x \cdot p_1 \;\oplus\; c_y \cdot p_2 \;\oplus\; c_z \cdot p_3 \Big) \bmod M, \tag{3.2}$$

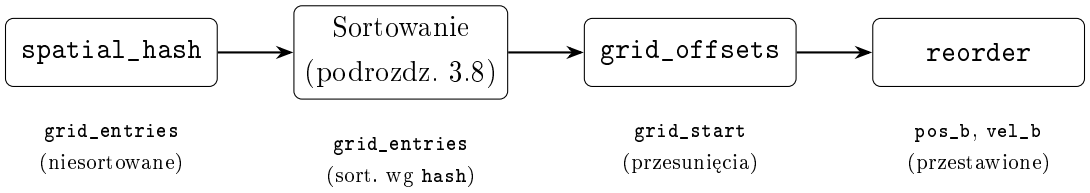
gdzie $p_1 = 73856093$, $p_2 = 19349663$, $p_3 = 83492791$ to trzy wzajemnie pierwsze stałe redukujące kolizje przy typowych rozkładach cząstek [46], a M jest rozmiarem tablicy

równym potędze dwójki.

Wynikiem tej operacji dla każdej cząstki jest rekord `Entry`, złożony z pola `hash` — obliczonego skrótu komórki — oraz pola `index` zawierającego oryginalny indeks cząstki w buforze pozycji. Przechowanie indeksu obok skrótu jest niezbędne, ponieważ tablica `grid_entries` zostanie posortowana według `hash`, lecz nie naruszy to kolejności w buforach fizycznych — `index` pozwala po sortowaniu jednoznacznie powiązać każdy wpis z odpowiadającymi mu danymi. Sortowanie rekordu `Entry` (8 bajtów) zamiast bezpośredniego przenoszenia wszystkich buforów fizycznych redukuje koszty sortowania o rząd wielkości.

3.7.2 Potok wyszukiwania

Wyszukiwanie sąsiadów realizowane jest przez cztery kolejne etapy (rysunek 3.4).



Rysunek 3.4: Czterostopniowy potok wyszukiwania sąsiadów. Źródło: opracowanie własne

Shader `spatial_hash` uruchamia jeden wątek na cząstkę: wątek oblicza komórkę przestrzenną swojej pozycji i skrót komórki, a wynikową parę `hash`–`index` zapisuje do bufora `grid_entries`. Sam skrót bez posortowania nie daje jeszcze żadnych korzyści, ponieważ znalezienie wszystkich wpisów danej komórki wymagałoby przejrzenia całego bufora — dlatego etap drugi sortuje `grid_entries` według pola `hash`, skupiając wpisy tej samej komórki w ciągłym fragmencie bufora; zastosowane algorytmy sortowania na GPU omówiono w rozdziale 3.8. Etap trzeci `grid_offsets` skanuje posortowaną tablicę i dla każdego miejsca, gdzie wartość `hash` zmienia się, zapisuje ten indeks do `grid_start`: wyszukiwanie sąsiadów sprowadza się wówczas do jednego odczytu z `grid_start`, po czym iteracja po wpisach tej komórki przebiega sekwencyjnie bez dalszych porównań.

Po sortowaniu tablica `grid_entries` opisuje właściwy porządek przestrzenny, lecz fizyczne bufory pozycji i prędkości wciąż przechowują dane w oryginalnym, przypadkowym układzie. Shader obliczający gęstość iteruje po kilkudziesięciu sąsiadach w pętli wewnętrznej; gdyby każde pobranie danych sąsiada wymagało odczytu spod adresu wskazywanego przez `grid_entries[j].index`, każda iteracja trafiałaby do innego, nieprzewidywalnego miejsca w pamięci GPU. Procesor strumieniowy nie jest wtedy w stanie prefetchować kolejnych wartości — każde trafienie w linię pamięci podręcznej

zastępuje poprzednią, zanim zdąży zostać ponownie użyta, co niweluje korzyść z samego sortowania. Etap `reorder` ponosi ten koszt raz i dla wszystkich: wątek o numerze i kopiuje pozycję i prędkość cząstki `grid_entries[i].index` na pozycję i w buforze `_b`, po czym kolejni sąsiedzi w przestrzeni leżą pod kolejnymi adresami w pamięci i GPU może ładować je w jednej transakcji z całą linią pamięci podręcznej.

3.8 Sortowanie na GPU

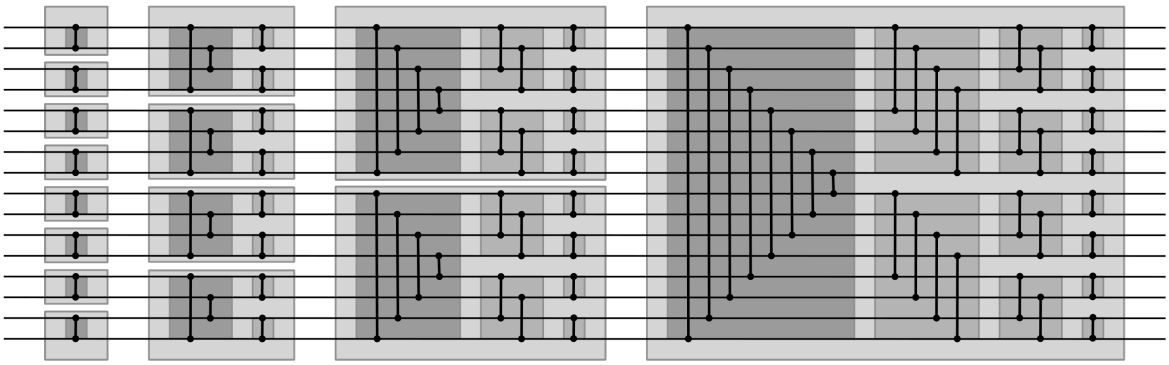
Sortowanie bufora `grid_entries` jest najkosztowniejszą operacją potoku wyszukiwania sąsiadów i wywiera bezpośredni wpływ na czas wykonania klatki. Pierwszą zaimplementowaną metodą było sortowanie bitoniczne — algorytm oparty na sieci porównań, który dobrze nadaje się na GPU ze względu na zerową rozbieżność wątków (ang. *thread divergence*) i prostą strukturę sterowania. W praktyce okazało się jednak, że przy dużej liczbie cząstek czas sortowania dominuje całkowity koszt kroku fizyki: złożoność $O(N \log^2 N)$ sprawia, że każde podwojenie liczby cząstek niemal czterokrotnie wydłuża sortowanie. Skłoniło to do implementacji sortowania pozycyjnego — algorytmu liniowego $O(kN)$ o stałej liczbie przebiegów niezależnej od N — jako alternatywy dostępnej do przełączenia w czasie działania programu. Oba algorytmy omówiono poniżej, a ich porównanie ilościowe przedstawiono w podrozdziale 3.8.3.

3.8.1 Sortowanie bitoniczne

Sieć sortowania bitonicznego [25] jest deterministycznym grafem komparatorów, w którym kolejność porównań ustalona jest przed wykonaniem i nie zależy od sortowanych danych. Każdy komparator porównuje dwa elementy i zamienia je miejscami, gdy są w złej kolejności; ponieważ wzorzec porównań jest stały dla każdej liczności N, wszystkie wątki GPU wykonują dokładnie tę samą sekwencję instrukcji niezależnie od zawartości buforów. Własność ta eliminuje rozbieżność wątków — jedno z podstawowych źródeł degradacji wydajności w architekturze SIMT, gdzie odejście od wspólnej ścieżki wykonania zmusza sprzęt do serializacji rozbieżnych gałęzi.

Dla N elementów sieć składa się z $\frac{\log_2 N \cdot (\log_2 N + 1)}{2}$ przebiegów podzielonych na $\log_2 N$ grup zewnętrznych. Rysunek 3.5 przedstawia kompletną sieć dla $N = 16$: dziesięć przebiegów pogrupowanych w cztery etapy ($k = 1$, $k = 2$, $k = 3$, $k = 4$), każdy z rosnącą liczbą przebiegów wewnętrznych, widocznych jako zagnieżdżone prostokąty. Każdy wiersz odpowiada jednemu elementowi utożsamianemu z wątkiem GPU, każda pionowa kreska z wypełnionymi kółkami — jednemu komparatorowi. W każdym przebiegu aktywnych jest $N/2 = 8$ komparatorów niezależnych od siebie, co oznacza, że wszystkie osiem operacji porównaj-i-zamień wykonywanych jest równolegle bez jakichkolwiek

zależności między wątkami i bez konieczności synchronizacji wewnątrz przebiegu.



Rysunek 3.5: Sieć sortowania bitonicznego dla $N = 16$: dziesięć przebiegów pogrupowanych w cztery zewnętrzne etapy ($k = 1, 2, 3, 4$). Każdy wiersz odpowiada jednemu elementowi, każda pionowa kreska — komparatorowi obsługiwanemu przez osobny wątek GPU; zagnieżdżone prostokąty zaznaczają granice kolejnych etapów zewnętrznych [11]

Implementacja na GPU mapuje każdy przebieg na osobne wywołanie potoku obliczeniowego. Dwa parametry charakteryzujące dany przebieg — wysokość aktualnego bloku bitonicznego oraz bieżący krok wewnętrzny — są przekazywane jako stałe wbudowane (ang. *push constants*) bezpośrednio przed wywołaniem shadera; dzięki temu ten sam skompilowany moduł obsługuje wszystkie sześć przebiegów bez konieczności tworzenia oddzielnych potoków. Rozmiar grupy roboczej wynosi 256 wątków, a jedno wywołanie shadera pokrywa $256 \cdot 2 = 512$ sortowanych elementów, co dla N cząstek przekłada się na $\lceil N/512 \rceil$ grup roboczych na przebieg.

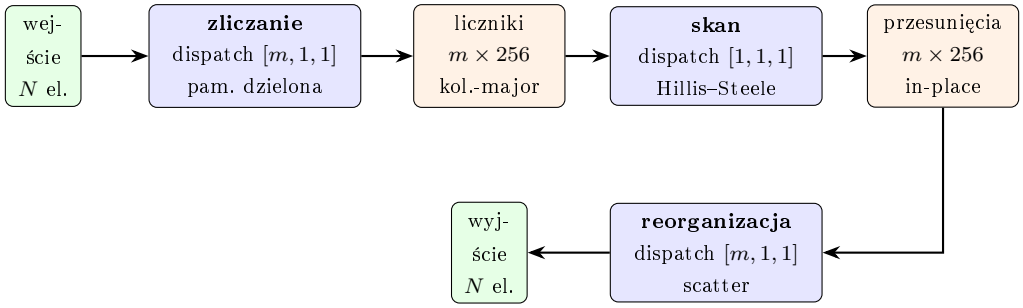
Zerowa rozbieżność wątków w połączeniu z prostotą operacji porównaj-i-zamień sprawia, że sortowanie bitoniczne osiąga wysoką przepustowość dla zbiorów o rozmiarze rzędu kilkudziesięciu tysięcy elementów — skali typowej dla symulacji małej i średniej liczby cząstek. Złożoność $O(N \log^2 N)$ staje się jednak coraz mniej korzystna w miarę wzrostu N, co skłoniło do zbadania alternatywy opartej na zliczaniu, omówionej w kolejnym podrozdziale.

3.8.2 Sortowanie pozycyjne

Sortowanie pozycyjne jest algorytmem sortowania przez zliczanie, a nie przez porównywanie, co stanowi fundamentalną różnicę w stosunku do sortowania bitonicznego. Zamiast parować elementy, każdy przebieg algorytmu zlicza, ile razy w danych pojawia się każda z 2^b możliwych wartości wybranego b-bitowego fragmentu klucza, i na tej podstawie oblicza adresy docelowe każdego elementu. Przy sortowaniu 32-bitowych kluczy po $b = 8$ bitów na raz ($r = 2^8 = 256$ kubełków) potrzebne są dokładnie cztery przebiegi, co daje złożoność $O(kN)$ przy stałym $k = 4$ niezależnym od N — w praktyce

liniowy wzrost kosztu z liczbą sortowanych elementów [23].

Każdy z czterech przebiegów rozkłada się na trzy kolejne shadery obliczeniowe uruchamiane sekwencyjnie z barierami pamięci GPU między nimi: *zliczanie*, *skan prefiksowy* i *reorganizacja*. Przepływ danych w jednym przebiegu ilustruje rysunek 3.6.



Rysunek 3.6: Przepływ danych w jednym przebiegu sortowania pozycyjnego. Źródło: opracowanie własne na podstawie [23]

Shader *zliczanie* jest wywoływany z $m = \lceil N/2048 \rceil$ grupami roboczymi, gdzie każda przetwarza dokładnie $256 \times 8 = 2048$ elementów — wynik przyjętego rozmiaru grupy roboczej wynoszącego 256 wątków przy ośmiu elementach na wątek (stała `ELEMENTS_PER_INVOCATION`). Każdy wątek wyodrębnia b-bitową cyfrę z każdego ze swoich ośmiu elementów przez przesunięcie bitowe i maskowanie, a następnie zlicza jej wystąpienia operacją atomową `atomicAdd` na tablicy `local_counts[256]` przechowywanej w pamięci dzielonej grupy roboczej. Koszt operacji atomowej na pamięci dzielonej jest wielokrotnie niższy niż na pamięci globalnej GPU — bez tego kroku wszystkie wątki wszystkich grup roboczych konkurowałyby o te same 256 globalnych liczników, co skutkowałoby serializacją i niwelowałoby zyski z równoległości. Po przetworzeniu całego segmentu bariera synchronizuje lokalne liczniki w obrębie grupy, a jeden wątek zapisuje 256 wartości do globalnego bufora liczników pod indeksem $d \cdot m + \mathrm{wg}$, gdzie d to wartość cyfry, a wg — numer grupy roboczej. Ten kolumnowy układ bufora jest nieprzypadkowy: podczas skanowania kolejne wątki odczytują wartości tego samego kubełka d z kolejnych segmentów pod kolejnymi adresami pamięci, co zapewnia koalescencję dostępów do pamięci globalnej.

Shader *skan prefiksowy* wywoływany jest zawsze jako pojedyncza grupa robocza o rozmiarze 256 wątków (dispatch $[1, 1, 1]$), niezależnie od liczności N. Jedna grupa przekształca tablicę $m \times 256$ liczników w tablicę globalnych przesunięć metodą Hillisa–Steele'a [23] w trzech fazach: najpierw obliczana jest wyłączna suma prefiksowa w obrębie każdego wiersza tablicy, dając lokalne przesunięcia wewnątrz grupy roboczej; następnie przeprowadzana jest włączna suma prefiksowa po sumarycznych zliczeniach każdego kubełka; na końcu globalne przesunięcia bloków są dodawane z powrotem, kompletując tablicę adresów docelowych. Wybór pojedynczej grupy roboczej eliminuje

konieczność wielopoziomowej synchronizacji globalnej, która byłaby niezbędna przy wielogrupowym skanowaniu — jest to możliwe właśnie dlatego, że liczba kubełków wynosi 256, co odpowiada typowej szerokości SIMD architektury GPU i mieści się w jednej grupie roboczej.

Shader *reorganizacja* przywraca pełny paralelizm: wywoływany z m grupami roboczymi przetwarzającymi po 2048 elementów każda. Dla każdego elementu wątek oblicza adres docelowy jako sumę globalnego przesunięcia odczytanego z tablicy przesunięć oraz lokalnego indeksu tego elementu w obrębie grupy roboczej i kubełka, po czym zapisuje go pod ten adres w buforze wyjściowym. Operacja ta jest stabilnym rozproszonym zapisem (ang. *scatter*) — zachowuje wzajemną kolejność elementów o tej samej wartości cyfry, co jest warunkiem koniecznym poprawności całego algorytmu: przebiegi od najmniej znaczących bitów (ang. *LSB first*) ku najbardziej znaczącym zachowują poprawną kolejność z poprzednich przebiegów wyłącznie wtedy, gdy każdy bieżący przebieg jest stabilny.

Implementacja korzysta z dwóch buforów sortowania: głównego `grid_entries` oraz pomocniczego `entries_tmp`, wewnętrznego do struktury `RadixSorter`. Parzyste przebiegi kierują dane z `grid_entries` do `entries_tmp`, nieparzyste — w odwrotnym kierunku; po czterech przebiegach wynik jest zawsze z powrotem w `grid_entries`. Złożoność $O(kN)$ ze stałym $k = 4$ sprawia, że koszt sortowania rośnie liniowo z liczbą cząstek, co czyni sortowanie pozycyjne algorytmem z wyboru dla dużych symulacji, gdzie logarytmiczne czynniki sortowania bitonicznego stają się odczuwalne.

3.8.3 Porównanie algorytmów

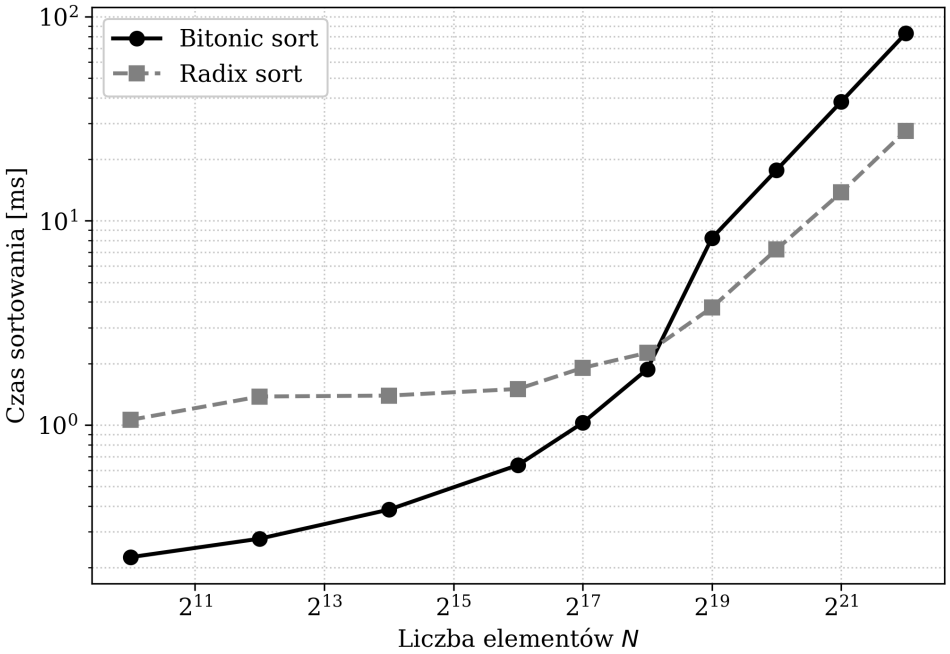
Mając zaimplementowane oba algorytmy sortowania, można je porównać zarówno pod kątem właściwości strukturalnych, jak i rzeczywistej wydajności na GPU. Tabela 3.9 zestawia najważniejsze cechy obu metod, podkreślając te różnice, które bezpośrednio wpływają na koszt wykonania w potoku obliczeniowym.

Bezpośredni pomiar czasu sortowania w funkcji liczby elementów N przeprowadzono w środowisku testowym opartym na bezokienkowym kontekście Vulkan, z buforem wypełnianym pseudolosowymi kluczami przed każdym uruchomieniem. Czas dla każdego punktu pomiaru wyznaczono jako medianę z piętnastu niezależnych przebiegów, poprzedzonych trzema przebiegami rozgrzewkowymi pokrywającymi koszt kompilacji shaderów przez sterownik. Wyniki, dla N rosnącego od 2^{10} do 2^{22}, przedstawia rysunek 3.7.

Dla małej liczby elementów ($N \lesssim 2^{18}$) sortowanie bitoniczne jest szybsze mimo asymptotycznie gorszej złożoności. Wynika to z bardzo niskiego stałego narzutu pojedynczego komparatora: każdy przebieg bitoniczny składa się wyłącznie z odczytu pary

Tabela 3.9: Porównanie własności sortowania bitonicznego i pozycyjnego

Cecha	Sortowanie bitoniczne	Sortowanie pozycyjne
Złożoność czasowa	$O(N \log^2 N)$	$O(kN)$, $k = 4$
Liczba dispatchów dla $N = 2^{15}$	120	12
Rozbieżność wątków	Zerowa	Zerowa
Pamięć pomocnicza	Brak (sort. w miejscu)	Bufor `entries_tmp` i bufor liczników
Stabilność sortowania	Niestabilna	Stabilna
Wymóg na N	Potęga dwójki	Dowolne



Rysunek 3.7: Czas sortowania bufora `grid_entries` jako funkcja liczby elementów N, w skali logarytmicznej na obu osiach. Źródło: opracowanie własne

elementów, porównania i warunkowej zamiany, podczas gdy każdy przebieg sortowania pozycyjnego wymaga trzech kolejnych dispatchów ze wzajemną synchronizacją barierami pamięci. Przy $N \leqslant 2^{14}$ cała praca obliczeniowa sortowania mieści się w czasie porównywalnym z kosztem wprowadzenia poleceń do kolejki GPU — pomiar jest zdominowany przez ten stały narzut, a różnica między algorytmami pochodzi głównie z liczby dispatchów składowych.

Punkt równowagi obserwuje się między $N = 2^{18}$ a $N = 2^{19}$. Powyżej tego progu logarytmiczny czynnik bitonicznego $O(N \log^2 N)$ zaczyna dominować nad stałą liczbą dwunastu wywołań sortowania pozycyjnego. Dla $N = 2^{22}$ (ponad cztery miliony elementów) sortowanie pozycyjne jest niemal trzykrotnie szybsze: 30,5 ms względem 82,6 ms dla bitonicznego.

Pomiar wyizolowanego sortowania nie odzwierciedla jednak w pełni zachowania algorytmu w pełnym potoku obliczeniowym, w którym dispatche sortowania są przeplatane innymi etapami solvera DFSPH wraz z wymaganymi między nimi barierami pamięci. Profil Tracy zarejestrowany dla symulacji z 25 000 cząstek (tabele 3.10 i 3.11) pokazuje, że średni czas wykonania etapu `neighbor_search_post_integrate` — obejmującego haszowanie, sortowanie, wyznaczenie przesunięć i reorganizację buforów — wynosi 3,64 ms dla wariantu bitonicznego i 616,3 µs dla wariantu sortowania pozycyjnego. Oba pomiary dotyczą tej samej konfiguracji symulacji, więc niemal sześciokrotna różnica wynika wyłącznie z wyboru algorytmu sortowania.

Tabela 3.10: Profil wydajności dla wariantu z sortowaniem bitonicznym ($N = 25\,000$ cząstek)

Nazwa etapu	Czas całkowity	Wywołania	Średni czas
`neighbor_search_post_integrate`	37,40 s (14,24%)	10 276	3,64 ms
`neighbor_search_init`	5,69 s (2,17%)	1 468	3,88 ms
`divergence_solver`	3,48 s (1,32%)	10 276	338,46 µs
`density_solver`	3,43 s (1,31%)	10 276	333,98 µs
`density_alpha_post_integrate`	477,31 ms (0,18%)	10 276	46,45 µs
`divergence_source_term`	425,08 ms (0,16%)	10 276	41,37 µs
`density_source_term`	421,67 ms (0,16%)	10 276	41,03 µs
`viscosity`	412,09 ms (0,16%)	10 276	40,10 µs
`pressure_integration`	391,08 ms (0,15%)	10 276	38,06 µs
`divergence_integration`	294,14 ms (0,11%)	10 276	28,62 µs
`density_alpha_init`	63,68 ms (0,02%)	1 468	43,38 µs
`stats`	58,12 ms (0,02%)	1 468	39,59 µs
`substep`	12,80 ms (0,00%)	10 276	1,25 µs

Tabela 3.11: Profil wydajności dla wariantu z sortowaniem pozycyjnym ($N = 25\,000$ cząstek)

Nazwa etapu	Czas całkowity	Wywołania	Średni czas
`neighbor_search_post_integrate`	1,21 s (4,95%)	1 960	616,30 μs
`divergence_solver`	634,15 ms (2,60%)	1 960	323,54 μs
`density_solver`	624,33 ms (2,56%)	1 960	318,54 μs
`neighbor_search_init`	78,88 ms (0,32%)	98	804,93 μs
`density_source_term`	70,83 ms (0,29%)	1 960	36,14 μs
`density_alpha_post_integrate`	70,20 ms (0,29%)	1 960	35,82 μs
`divergence_source_term`	70,08 ms (0,29%)	1 960	35,75 μs
`pressure_integration`	68,99 ms (0,28%)	1 960	35,20 μs
`viscosity`	67,89 ms (0,28%)	1 960	34,64 μs
`divergence_integration`	52,44 ms (0,21%)	1 960	26,76 μs
`stats`	5,16 ms (0,02%)	98	52,65 μs
`density_alpha_init`	3,78 ms (0,02%)	98	38,54 μs
`substep`	1,01 ms (0,00%)	1 960	514 ns

Rozbieżność między izolowanym testem porównawczym (gdzie sortowanie pozycyjne dla $N = 2^{15}$ jest nieznacznie wolniejsze) a pomiarem produkcyjnym (gdzie ten sam N daje sześciokrotne przyspieszenie) ma uzasadnienie strukturalne. Sortowanie bitoniczne 32 768 elementów ($\lceil 25\,000 \rceil_2$) wymaga $\frac{\log_2^2 32768 + \log_2 32768}{2} = 120$ dispatchów, z których każdy w pełnym potoku jest oddzielony od kolejnego barierą pamięci wstawioną przez `AutoCommandBufferBuilder`; sortowanie pozycyjne wymaga ich jedynie dwunastu. W teście izolowanym, gdzie GPU wykonuje wyłącznie sortowanie, koszt pojedynczej bariery jest amortyzowany przez wysoką zajętość jednostek obliczeniowych. W pełnym potoku DFSPH bariera staje się jawnym punktem synchronizacji, ponieważ poprzedza ją zapis innego shadera obliczeniowego — a sumaryczny koszt skaluje się z liczbą dispatchów algorytmu sortowania.

Łączny obraz prowadzi do jednoznacznego wyboru: dla każdego rozmiaru symulacji od kilku tysięcy cząstek wzwyż sortowanie pozycyjne jest szybszym wariantem, a jego przewaga rośnie zarówno z N, jak i z liczbą innych etapów obliczeniowych dzielących GPU. Sortowanie bitoniczne pozostaje w kodzie jako alternatywa dostępna do przełączenia w czasie wykonania przez wariant `SortAlgorithm::Bitonic`, użyteczna w celach porównawczych oraz w scenariuszach, w których prostota sieci komparatorów ma znaczenie większe niż mierzona wydajność.

3.9 Solver DFSPH

Wszystkie wcześniejsze sekcje tej części pracy — organizacja pamięci w układzie SOA z parami buforów dla pozycji i prędkości, haszowanie przestrzenne wraz z sortowaniem i reorganizacją buforów oraz autorska cecha `ComputeStep` unifikująca cykl życia potoków obliczeniowych — istnieją wyłącznie po to, by w tym miejscu mogła zostać zrealizowana sama symulacja dynamiki cieczy. Dyskretyzacja jądrowa metody SPH wprowadzona w podrozdziale 1.2.2, dwuetapowa korekcja DFSPH zdefiniowana algorytmami 1.1 i 1.2 oraz konkretne struktury danych z sekcji 3.6 zbiegają się tutaj w jeden spójny potok obliczeniowy. Każdy z jego etapów jest osobnym implementatorem cechy `ComputeStep` i odpowiada konkretnemu fragmentowi matematycznego sformułowania metody; ich kolejność oraz wzajemne zależności określają, w jaki sposób abstrakcyjne równania teoretyczne odwzorowują się na rzeczywiste shadery obliczeniowe wykonywane na GPU.

Z implementacyjnego punktu widzenia kluczowa jest asymetria między obydwoma poziomami korekcji: *korekcja gęstości* kończy się adwekcją pozycji $\mathbf{p}_i \leftarrow \mathbf{p}_i + \Delta t\, \mathbf{v}_i$, natomiast *korekcja dywergencji* działa wyłącznie na buforze prędkości i nie modyfikuje pozycji cząstek. Ta różnica determinuje kolejność shaderów w pętli podkroków (ang. *substep*). Poza nią oba poziomy współdzielą identyczną strukturę: iteracyjna pętla ciśnienia z tym samym wzorcem dostępu do sąsiedztwa, różniąca się wyłącznie wyrazem źródłowym i finalnym krokiem integracji.

Wszystkie shadery realizujące fizykę SPH dzielą jeden fundamentalny wzorzec dostępu do danych. Niezależnie od tego, czy obliczają gęstość, wyraz źródłowy, siłę ciśnienia czy korekcję lepkościową, każdy z nich akumuluje sumę po geometrycznym sąsiedztwie cząstki i, ważoną jądrem W lub jego gradientem ∇W. Konkretna postać akumulowanej wielkości różni się między etapami, lecz struktura pętli pozostaje niezmiennikiem całego potoku. Wykorzystując bufor `grid_start` przygotowany przez potok wyszukiwania sąsiadów (sekcja 3.7), każdy shader wyznacza komórkę przestrzenną swojej cząstki i iteruje po $3 \times 3 \times 3 = 27$ komórkach sąsiednich, używając ich skrótów jako indeksów wejścia do listy posortowanych wpisów. Jądro $W(\|\mathbf{r}\|, h)$ zwraca skalarną wagę, malejącą z odległością; jego gradient $\nabla W(\mathbf{r}, h)$ jest wektorem wskazującym kierunek i tempo zmian tej wagi. W obliczeniach SPH W pojawia się wszędzie tam, gdzie sumujemy wielkości skalarne — jak gęstość — natomiast ∇W wszędzie tam, gdzie potrzebna jest siła lub przyspieszenie. Algorytm 3.2 przedstawia ten uniwersalny wzorzec.

Identyczna pętla iteracji po sąsiadach występuje we wszystkich shaderach solvera DFSPH — różni je wyłącznie wyrażenie pod sumą oraz atrybut, do którego trafia wynik akumulacji. Dzięki przepisaniu pozycji i prędkości na bufory `_b` przez etap

Algorithm 3.2: Wspólny wzorzec iteracji po sąsiadach w shaderach SPH

```
1  c_i ← ⌊p_i/h⌋;
2  for dz ∈ {−1, 0, +1} do
3      for dy ∈ {−1, 0, +1} do
4          for dx ∈ {−1, 0, +1} do
5              k ← hash(c_i + (dx, dy, dz));
6              s ← grid_start[k];
7              while s < M i grid_entries[s].hash = k do
8                  j ← grid_entries[s].index;
9                  r_ij ← p_i − p_j;
10                 if ‖r_ij‖ < h then
11                     akumulacja wkładu pary (i, j) z wagą W(‖r_ij‖, h) lub
                       ∇W(r_ij, h);
12                 s ← s + 1;
```

`reorder` (3.7.2), kolejne wpisy danej komórki leżą gęsto w pamięci, a odczyt całego 27-elementowego sąsiedztwa wykonuje się w niewielkiej liczbie skoalescowanych transakcji pamięci globalnej GPU.

Tabela 3.12 odwzorowuje wielkości figurujące w sformułowaniu matematycznym DFSPH na konkretne bufory struktury `GpuPhysicsData` oraz na shadery odpowiedzialne za ich zapis. Ze względu na podwójne buforowanie cząstek (3.6.2), pozycje i prędkości występują w dwóch wariantach: bufor `_a` przechowuje stabilny stan wejściowy, na podstawie którego haszowanie przestrzenne wyznacza komórki cząstek; bufor `_b`, do którego `reorder` przepisuje dane w kolejności sprzyjającej lokalności odczytu, stanowi wejście dla wszystkich kolejnych etapów solvera. Pozostałe wielkości to jednowymiarowe tablice indeksowane numerem cząstki.

Tabela 3.12: Odwzorowanie wielkości teoretycznych DFSPH na bufory GPU

Wielkość	Bufor	Etap zapisujący
$\mathbf{p}_i$ — pozycja	`position_a`, `position_b`	`reorder`, `pressure_integration`
$\mathbf{v}_i$ — prędkość	`velocity_a`, `velocity_b`	`reorder`, `viscosity`, korekcje ciśnienia
ρ_i — gęstość	`densities`	`density_alpha`
α_i — współczynnik DFSPH	`factors`	`density_alpha`
s_i — wyraz źródłowy	`source_terms`	`density_source_term`, `divergence_source_term`
p_i — ciśnienie	`pressures`	`pressure_update`
$\mathbf{a}_i^p$ — przysp. ciśnienia	`pressure_accelerations`	`pressure_force`

Pozostałe podrozdziały tej sekcji szczegółowo omawiają poszczególne komponenty solvera. Podrozdział 3.9.1 przedstawia pełną pętlę podkroków ustalającą kolejność wszystkich etapów wewnątrz pojedynczej klatki. Podrozdział 3.9.2 opisuje obliczanie gęstości ρ_i oraz współczynnika α_i, pełniącego rolę prekondycjonera w solverze ciśnienia. Podrozdziały 3.9.3 i 3.9.4 analizują kolejno korekcję gęstości i korekcję dywergencji — dwie iteracyjne pętle ciśnienia, które łącznie wymuszają nieściśliwość pola prędkości. Podrozdział 3.9.5 omawia warunek CFL i strategię doboru kroku czasowego Δt.

3.9.1 Pętla podkroków

Pełna sekwencja wywołań shaderów w obrębie podkroku ukazana jest w Algorytmie 3.1. Bezpośrednio po adwekcji następuje przebudowa struktury wyszukiwania sąsiadów. To drugie wywołanie `neighbor_search` w obrębie jednego podkroku nie jest redundancją — jest koniecznością wynikającą z fizycznego przemieszczenia cząstek. Tablica `grid_start`, zbudowana przed podkrokiem, indeksuje pozycje sprzed adwekcji; korekcja dywergencji oparta na nieaktualnej strukturze obliczałaby gradient prędkości $\nabla\cdot\mathbf{v}$ względem błędnych sąsiadów, co prowadziłoby do narastających błędów kompresji szczególnie widocznych przy dużych prędkościach cząstek. Po przebudowie sąsiedztwa i ponownym wyznaczeniu ρ_i, α_i faza dywergencji przebiega analogicznie: iteracyjna pętla N_∇ razy uruchamia ten sam `pressure_force` i `pressure_update` z wyrazem źródłowym opartym na dywergencji pola prędkości, po czym `divergence_integration` aktualizuje prędkości bez modyfikowania pozycji.

Iteracyjny charakter obu solverów wynika z natury problemu. Zmiana ciśnienia cząstki i generuje przyspieszenie oddziałujące na jej sąsiadów, które z kolei wymagają korekty swoich ciśnień — i tak dalej. Jedno przejście pętli realizuje krok analogiczny do iteracji Jacobiego: wyznacza lokalną poprawkę przy założeniu, że ciśnienia sąsiadów pozostają chwilowo zamrożone. Kilka takich kroków ($N_\rho = 2$–5, $N_\nabla = 1$–3) zazwyczaj wystarcza do sprowadzenia średniego błędu poniżej zadanego progu; strategię doboru tych liczb omówiono w podrozdziale 3.9.5.

3.9.2 Obliczanie gęstości

Shader `density_and_alpha` oblicza w jednym przebiegu dwie wielkości: gęstość ρ_i oraz współczynnik α_i [6]. Listing 3.4 przedstawia funkcję `main()` shadera.

```
void main() {
    uint i = gl_GlobalInvocationID.x;
    if (i >= uint(positions.length())) return;

    float density     = 0.0;
    float sum_grad_sq = 0.0;
    vec3  grad_sum    = vec3(0.0);

    // iteracja po 27 sąsiednich komórkach
    float r = sqrt(dot(r_vec, r_vec));
    density += mass * kernel_w(r, h);
    if (r > 1e-6) {
        vec3 mass_grad  = mass * kernel_grad(r_vec, r, h);
        grad_sum       += mass_grad;
        sum_grad_sq    += dot(mass_grad, mass_grad);
    }
    // koniec petli

    densities[i] = density;
    float alpha = sum_grad_sq + dot(grad_sum, grad_sum);
    if (alpha > 1e-6) { factors[i] = alpha; }
    else              { factors[i] = 0.0;   }
}
```

Listing 3.4: Shader `density_and_alpha` — logika akumulacji

Każda wywołanie odpowiada jednej cząstce i. Zmienna `density` akumuluje skalarne wkłady jądra `kernel_w` po 27 sąsiednich komórkach — jest to bezpośrednia realizacja wzorca z Algorytmu 3.2. Zmienne `grad_sum` i `sum_grad_sq` akumulują jednocześnie składniki współczynnika α_i: odpowiednio sumę gradientów $\sum_j m\nabla W_{ij}$ oraz sumę ich kwadratów $\sum_j |m\nabla W_{ij}|^2$; wynikowe $\alpha_i = |\texttt{grad_sum}|^2 + \texttt{sum_grad_sq}$ sta-

nowi mianownik korekcji ciśnienia w każdym kroku iteracyjnym solvera. Obliczenie obu wielkości w jednej pętli zamiast dwóch oddzielnych przebiegów redukuje o połowę liczbę odczytów z pamięci globalnej GPU.

3.9.3 Korekcja gęstości

Faza korekcji gęstości wymusza nieściśliwość, wyznaczając ciśnienia redukujące odchylenia ρ_i od wartości docelowej ρ_0. Shader `density_source_term` oblicza dla każdej cząstki i wyraz źródłowy łączący chwilowy deficyt gęstości z prognozą jego narastania na podstawie prędkości adwekcyjnych wyznaczonych po fazie lepkości:

```
float divergence_sum = 0.0;
// dla każdego sasiada j w promieniu h:
vec3 vel_diff = vel_i - vel_j;
divergence_sum += mass * dot(vel_diff, grad);
// koniec petli

float source = ((rho_0 - rho_i) / dt) - divergence_sum;
source_terms[i] = source;
pressures[i] = 0.0;
```

Listing 3.5: Shader `density_source_term` — wyraz źródłowy solvera gęstości

Składnik `(rho_0 - rho_i) / dt` mierzy bieżący błąd gęstości skalowany krokiem czasowym; `divergence_sum` aproksymuje metodą SPH dywergencję pola prędkości adwekcyjnych, przewidując jak ten błąd będzie dalej narastał. Tablica `pressures` jest zerowana na wejściu każdego wywołania, gwarantując start iteracji ze świeżym wektorem ciśnień.

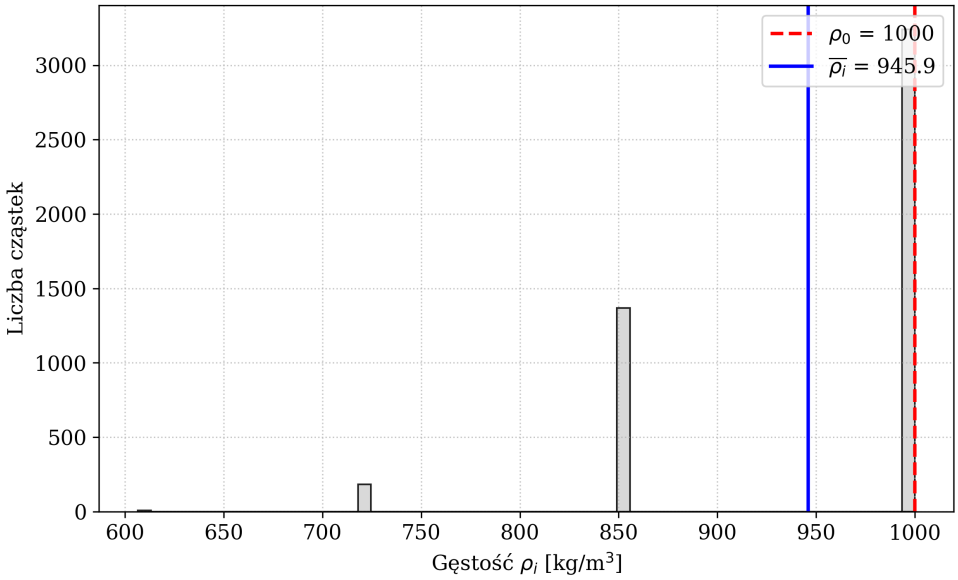
Iteracja Jacobiego przebiega przez naprzemienne wywołania `pressure_force` i `pressure_update`. Pierwszy shader wyznacza dla każdej cząstki przyspieszenie ciśnieniowe będące iloczynem macierzy układu i wektora ciśnień; drugi aktualizuje ciśnienie:

```
float a_ii       = -(dt / (rho_i * rho_i)) * alpha_i;
float error      = source_i - Ap_i;
float correction = (relax_factor * error) / a_ii;
pressures[i]     = max(p_i + correction, 0.0);
```

Listing 3.6: Shader `pressure_update` — krok Jacobiego

Zmienna `a_ii` to przekątny element operatora ciśnieniowego obliczony bezpośrednio z α_i wyznaczonego przez `density_and_alpha`; `relax_factor` ($\omega = 0{,}5$) ogranicza oscylacje typowe dla niezrelaksowanej iteracji Jacobiego. Wywołanie `max(..., 0.0)` eliminuje ujemne ciśnienia: bez niego cząstki przy swobodnej powierzchni, których gęstość spełnia $\rho_i < \rho_0$, generowałyby siły przyciągające, prowadząc do niestabilności tensylnej.

Test `rest_density_diagnostic` ujawnił wyraźną dwumodalność rozkładu gęstości (rysunek 3.8): cząstki wewnętrzne osiągają $\rho_i \approx \rho_0$, natomiast cząstki przy swobodnej powierzchni wykazują systematyczny deficyt wynikający z niepełnej sumy jądra SPH. Właśnie ten deficyt powoduje plateau widoczne w teście zbieżności (rysunek 3.9a) — błąd $\overline{|\rho_i - \rho_0|}$ utrzymuje się na poziomie ~ 200 kg/m^3 niezależnie od liczby iteracji, ponieważ solver poprawnie zeruje ciśnienie dla pod-gęstych cząstek brzegowych, nie próbując korygować efektu wynikającego z geometrii. Właściwa obsługa brzegu za pomocą cząstek-duchów [44] wykracza poza zakres niniejszej pracy.



Rysunek 3.8: Histogram gęstości w konfiguracji spoczynkowej ($n = 4800$ cząstek, bez kroku symulacji). Źródło: opracowanie własne

3.9.4 Korekcja dywergencji

Faza korekcji dywergencji działa na prędkościach cząstek po przesunięciu ich przez solver gęstości. Jej celem jest wyzerowanie dywergencji pola prędkości $\nabla \cdot \mathbf{v} = 0$, eliminując pozostałą składową kompresji. Shader `divergence_source_term` oblicza wyraz źródłowy z prędkości bezpośrednio po całkowaniu ciśnień:

```
float divergence_sum = 0.0;
// dla każdego sasiada j w promieniu h:
vec3 vel_diff = vel_i - vel_j;
divergence_sum += mass * dot(vel_diff, grad);
// koniec petli
source_terms[i] = -divergence_sum;
```

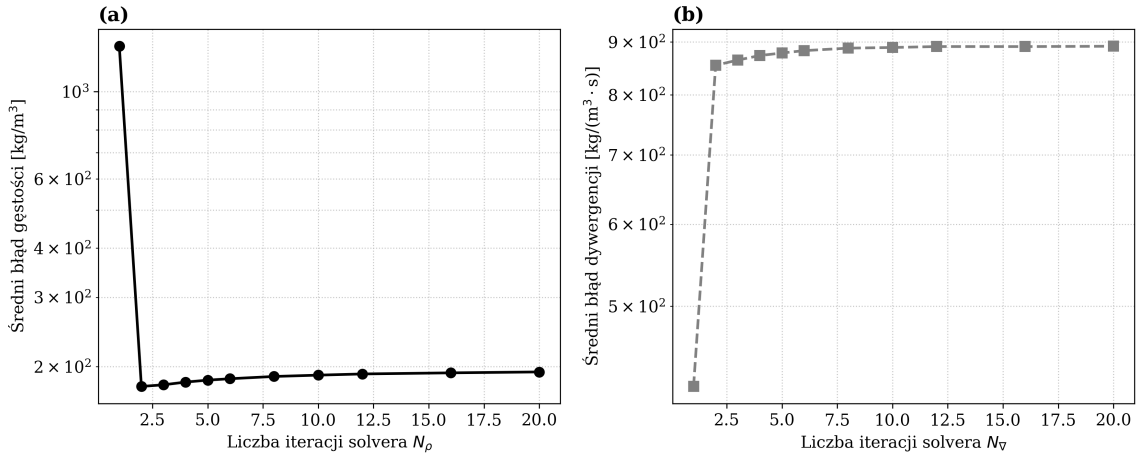
Listing 3.7: Shader `divergence_source_term` — wyraz źródłowy solvera dywergencji

W odróżnieniu od solvera gęstości, wyraz źródłowy nie zawiera składnika $(\rho_0 - \rho_i)/\Delta t$ — celem jest wyłącznie zerowanie dywergencji. Pętla Jacobiego jest identyczna (`pressure_force` i `pressure_update`), natomiast shader `divergence_integration` stosuje wynikowe ciśnienia jako korekty prędkości, nie pozycji:

```
vec3 new_vel = vel + ap * dt;
velocities[i] = vec4(new_vel, 0.0);
```

Listing 3.8: Shader `divergence_integration` — całkowanie prędkości

Rysunek 3.9b ujawnia odwrotną zależność: błąd dywergencji wzrasta wraz z liczbą iteracji i formuje symetryczne plateau od góry. Jest to bezpośrednie sprzężenie z solverem gęstości: jedna iteracja ledwo koryguje gęstość i nieznacznie przesuwa cząstki, pozostawiając pole prędkości prawie bezrozbieżne; dwie i więcej iteracji skutecznie redukuje $\rho_i - \rho_0$ drogą dużych przesunięć, które indukują proporcjonalnie silniejszą dywergencję prędkości. Oba wykresy tworzą zatem lustrzany obraz: plateau błędu gęstości od dołu odpowiada plateau błędu dywergencji od góry, wyznaczonemu przez skalę przesunięć generowanych przez solver gęstości w stanie nasycenia. Ponieważ błąd gęstości nigdy nie spada do zera z powodu deficytu jądra na brzegu, solver gęstości stale generuje impulsy ciśnieniowe, a solver dywergencji nie ma szans ustabilizować pola prędkości — stąd trwałe plateau po obu stronach wykresu.



Rysunek 3.9: Zbieżność solvera DFSPH: uśredniony błąd gęstości (a) i błąd dywergencji (b) w funkcji liczby iteracji Jacobiego. Źródło: opracowanie własne

3.9.5 Warunek CFL i dobór Δt

Dobór kroku czasowego Δt kształtował się przez kilka etapów prób i korekt, z których każdy ujawniał kolejną warstwę ograniczeń implementacji.

Etap 1 — stały Δt i stała liczba iteracji. Pierwsza wersja symulatora operowała stałym krokiem $\Delta t = 5$ ms i czterema iteracjami Jacobiego per solver per krok symulacji. Wynik był akceptowalny: symulacja przebiegała stabilnie przez większość czasu, lecz sporadycznie załamywała się wskutek kumulacji błędu numerycznego. Niska liczba klatek na sekundę skłoniła do poszukiwania optymalizacji.

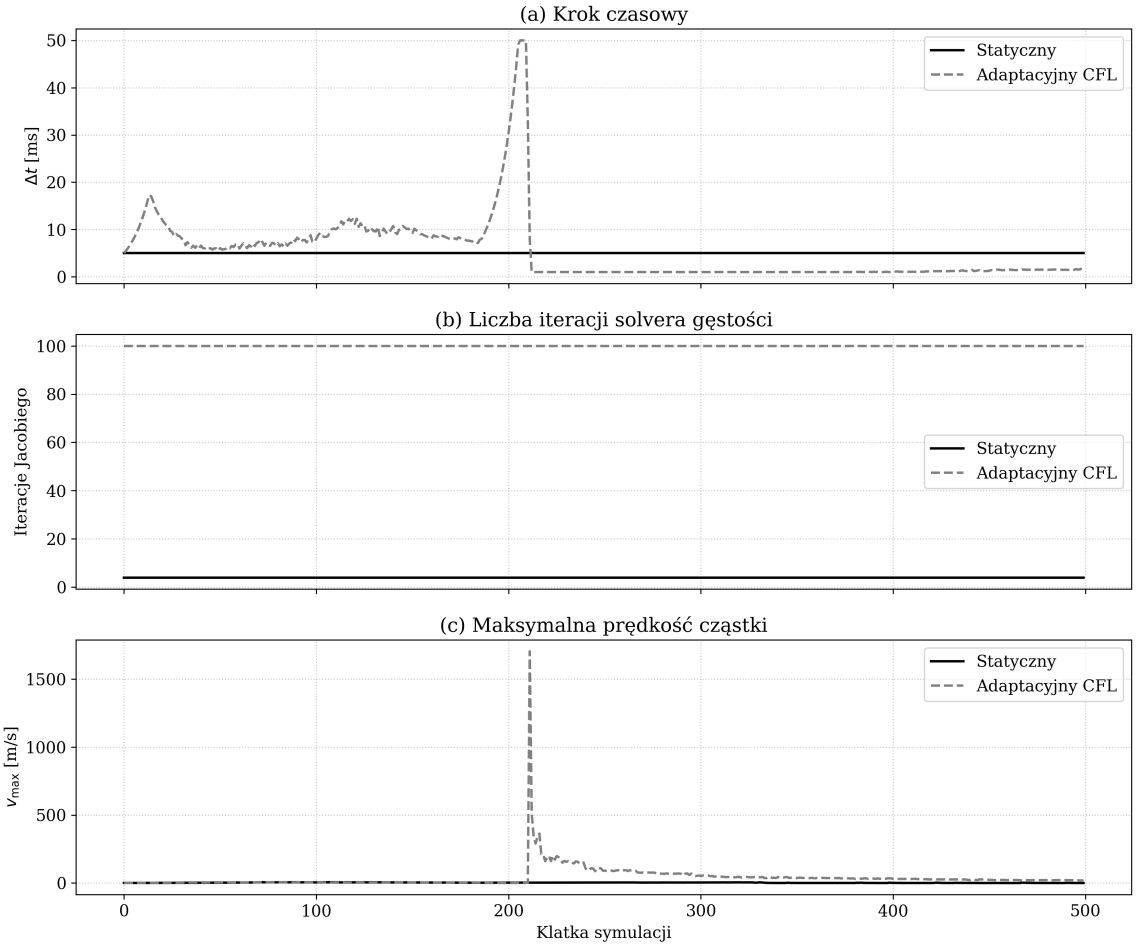
Etap 2 — adaptacyjny Δt według warunku CFL. Warunek CFL ogranicza krok czasowy tak, aby cząstka nie przemieściła się o więcej niż jeden promień wygładzania h:

$$\Delta t \leqslant \lambda \frac{h}{v_{\max}}, \tag{3.3}$$

gdzie $\lambda < 1$ to współczynnik bezpieczeństwa, a $v_{\max}$ to chwilowa prędkość maksymalna odczytywana z bufora statystyk GPU (`stats_buffer[0]`). Gdy symulacja uspokaja się i $v_{\max}$ maleje, Δt może wzrosnąć, co bezpośrednio przekłada się na wyższą liczbę klatek na sekundę. Pojawił się jednak nowy problem: większy Δt powoduje szybsze narastanie błędu gęstości, przez co cztery iteracje Jacobiego okazały się niewystarczające.

Etap 3 — kryterium zatrzymania oparte na progu błędu. Naturalnym rozwinięciem było dodanie drugiego warunku zakończenia iteracji: kontynuuj, dopóki $\overline{|\rho_i - \rho_0|} > \varepsilon$. W założeniu solver miał wykonywać 4–5 iteracji przy spokojnej symulacji i więcej przy gwałtownych ruchach, a rosnące Δt kompensowało koszt dodatkowych iteracji. W praktyce każdy solver wykonywał sto iteracji (skonfigurowane maksimum) i nigdy nie osiągał progu zbieżności. Przyczyna jest dokładnie tą, którą ujawnił test zbieżności (podrozdz. 3.9.3): deficyt jądra SPH na swobodnej powierzchni trwale blokuje błąd gęstości na poziomie $\sim$200 kg/m^3, niezależnie od liczby iteracji.

Decyzja końcowa. Jako kompromis powrócono do stałego $\Delta t = 5$ ms i stałej liczby czterech iteracji na solver. Rozwiązanie zapewnia przewidywalny koszt obliczeniowy klatki i eliminuje ryzyko zawieszenia symulatora w nieskończonej pętli iteracyjnej. Utrzymują się dwa znane ograniczenia: niska liczba klatek na sekundę wynikająca z braku adaptacji kroku oraz ryzyko niestabilności numerycznej przy długich sesjach. Oba są bezpośrednią konsekwencją braku właściwej obsługi brzegu — bez niej kryterium ε-zbieżności nie może działać poprawnie. Rysunek 3.10 zestawia obie strategie: panel (a) pokazuje krok czasowy — wariant adaptacyjny rośnie do 50 ms, po czym w okolicach klatki 205 wywołuje katastrofalną niestabilność ($v_{\max} \approx 1100$ m/s); panel (b) pokazuje liczbę iteracji solvera — wariant adaptacyjny osiąga limit 100 iteracji w każdej klatce bez wyjątku; panel (c) pokazuje $v_{\max}$ — po eksplozji prędkość stabilizuje się na poziomie 20–40 m/s, co wskazuje na stan uszkodzonej symulacji.



Rysunek 3.10: Porównanie strategii statycznego i adaptacyjnego Δt w trakcie 500 klatek symulacji: krok czasowy (a), liczba iteracji solvera Jacobiego (b), maksymalna prędkość cząstki (c). Źródło: opracowanie własne

3.10 Siły fizyczne

Podrozdział ten opisuje implementację dwóch mechanizmów siłowych działających na cząstki poza solverem DFSPH: wygładzenia prędkości metodą XSPH oraz odpowiedzi na kolizje ze ścianami sześcianu symulacji. Podstawy teoretyczne obu metod omówiono w podrozdziale 2.8; poniżej skupiono się na konkretnych wyborach implementacyjnych i stałych liczbowych przyjętych w kodzie.

3.10.1 Lepkość XSPH

Wygładzenie prędkości XSPH realizowane jest w shaderze `viscosity`. Poniższy listing przedstawia rdzeń obliczeń — pętlę po sąsiadach i końcową aktualizację prędkości:

```
vec3 sum_viscosity = vec3(0.0);
// dla każdego sasiada j w promieniu h:
vec3 vel_diff = velocities[j].xyz - velocities[i].xyz;
sum_viscosity += mass / densities[j] * vel_diff * kernel_w(r, h);
// koniec petli

vec3 vel_visco = velocities[i].xyz
               + sim_params.viscosity * sum_viscosity;
new_velocities[i] = vec4(vel_visco + sim_params.gravity.xyz
                                   * sim_params.dt, 0.0);
```

Listing 3.9: Rdzeń shadera `viscosity` — korekcja XSPH i grawitacja

Bezwymiarowy współczynnik wygładzenia ν przechowywany jest w polu `viscosity` struktury `SimulationParams`. Wartość $\nu = 0{,}15$ dobrano empirycznie: wartości poniżej 0,05 dopuszczają widoczne przenikanie cząstek przy zderzeniach, powyżej 0,4 nadmiernie tłumią ruch i spowalniają opadanie. Parametr jest dostępny w czasie wykonania przez suwak interfejsu egui bez potrzeby ponownej kompilacji shaderów.

Grawitacja dodawana jest w tym samym przebiegu tuż po korekcji XSPH, co eliminuje oddzielny dispatch i jedno przejście przez bufor prędkości. Wynik trafia do bufora `new_velocities`; zapis i odczyt dotyczą różnych buforów, więc kolejność wątków nie wpływa na wynik.

3.10.2 Obsługa kolizji

Reakcja na kolizję z ścianą odbywa się na końcu shadera `pressure_integration`, bezpośrednio po całkowaniu Eulera. Listing 3.10 pokazuje obsługę jednej ściany (dolna granica osi y); pozostałe pięć ścian realizowane są analogicznie:

```
float restitution = 0.5;
float friction    = 0.95;
float eps         = 0.001;

if (new_pos.y < min_b.y + r) {
    new_pos.y  = min_b.y + r + eps;
    new_vel.y *= -restitution;
    new_vel.xz *= friction;
}
```

Listing 3.10: Fragment `pressure_integration` — odbicie od dolnej sciany

Stała `eps` przesuwa cząstkę o 1 mm ponad granicę, zapobiegając wielokrotnemu wyzwalaniu warunku w kolejnych krokach. Współczynnik sprężystości $e = 0{,}5$ pochłania połowę energii normalnej przy każdym odbiciu; tarcie styczne $f = 0{,}95$ nieznacznie tłumi ruch równoległy do ściany, nie zatrzymując cząstki całkowicie.

Podejście to nie koryguje deficytu jądra przy brzegu omówionego w podrozdz. 3.9.4, lecz skutecznie zapobiega ucieczce cząstek poza granice sześcianu symulacji i jest wystarczające dla wizualnie wiarygodnych scen zamkniętych.

3.11 Wizualizacja

Potok wizualizacji przekształca listę pozycji cząstek w gotowy kolor piksela w trzech etapach: splatting gęstości do trójwymiarowej tekstury, przemarsz promienia przez tę teksturę do znalezienia powierzchni, oraz model optyczny łączący odbicie, załamanie, absorpcję i rozproszenie podpowierzchniowe. Podstawy teoretyczne wszystkich etapów omówiono w podrozdziale 2.9; poniżej przedstawiono konkretne stałe i fragmenty kodu z implementacji.

3.11.1 Splatting gęstości do tekstury 3D

Shader `splat_density` dla każdej cząstki wyznacza woksel centralny, a następnie iteruje po sześcianie ± 8 wokseli i atomowo dodaje wkład do tekstury:

```
int radius = 8;
// dla każdego woksela (x,y,z) w szescianie [-radius, radius]^3:
float dist   = length(vec3(x, y, z));
if (dist < float(radius)) {
    float x_norm = dist / float(radius);
    float weight = (1.0 - x_norm * x_norm);
    weight = weight * weight * weight;

```

```
9       uint contrib = uint(weight * 200.0);
10      imageAtomicAdd(density_grid, currentVoxel, contrib);
11  }
```

Listing 3.11: Shader `splat_density` — waga woksela i zapis atomowy

Obliczana waga $(1 - x_{\text{norm}}^2)^3$, w której zmienna `x_norm` = `dist`/`radius` odpowiada znormalizowanej odległości δ/R, jest bezpośrednią realizacją gładkiego wielomianu wygaszającego $w(\delta)$ zdefiniowanego równaniem 2.9 — zmienna `dist` pełni tu rolę odległości δ, a `radius` — promienia R. Mnożnik 200 skaluje wartość do zakresu liczby całkowitej bez utraty precyzji; woksele powietrza osiągają wartości bliskie zeru, woksele wewnątrz cieczy — rzędu setek. Tekstura ma format `R32Uint`; stała `DENSITY_OFFSET = 300` odejmowana jest przy odczycie w shaderze fragmentów, eliminując szum tła bez progowania binarnego.

3.11.2 Raymarching

Pętla przemarszu w `raymarch` składa się z trzech faz: testu AABB, grubego kroku oraz bisekcji refinementu.

Test AABB i krok gruby. Przed wejściem w pętlę funkcja `intersectAABB` wyznacza analitycznie przedział $[t_{\min}, t_{\max}]$ wejścia i wyjścia promienia z obszaru symulacji; jeśli $t_{\min} > t_{\max}$, promień w ogóle nie przecina sześcianu otaczającego i jest odrzucany instrukcją `discard`.

```
1   const int   MAX_STEPS = 96;
2   const float STEP_SIZE = 0.007;
3
4   vec2 tHit = intersectAABB(rayOrigin, rayDir, boxMin, boxMax);
5   if (tHit.x > tHit.y || tHit.y < 0.0) discard;
6
7   float stepWorld = STEP_SIZE
8                   * max(max(boxScale.x, boxScale.y), boxScale.z);
9   float tCurrent  = max(tHit.x, 0.0);
10
11  for (int i = 0; i < MAX_STEPS; i++) {
12      if (tCurrent > tHit.y) break;
13      if (getDensity(rayOrigin + rayDir * tCurrent, ...) > 0.01) {
14          hit = true; break;
15      }
16      tCurrent += stepWorld;
17  }
18  if (!hit) discard;
```

Listing 3.12: Fragment `raymarch` — test AABB i petla przemarszu

Krok `stepWorld` skalowany jest rozmiarem sceny, dzięki czemu ta sama stała `STEP_SIZE` działa poprawnie niezależnie od wymiarów sześcianu otaczającego. Promienie, które nie trafią w żaden woksel powyżej progu w 96 krokach, są odrzucane instrukcją `discard` jeszcze na etapie shadera fragmentów.

Bisekcja i normalna. Po wykryciu pierwszego przekroczenia progu przedział $[t_0, t_1]$ zawężany jest ośmiokrotną bisekcją, a normalna wyznaczana jest różnicami skończonymi na polu gęstości.

```
float t0 = tCurrent - stepWorld, t1 = tCurrent;
for (int i = 0; i < 8; i++) {
    float m = (t0 + t1) * 0.5;
    if (getDensity(rayOrigin + rayDir * m, ...) > 0.01)
        t1 = m;
    else
        t0 = m;
}
vec3 surfacePos = rayOrigin + rayDir * t1;
vec3 N = -calcNormal(surfacePos, boxMin, boxMax);
if (dot(N, N) < 0.5) N = vec3(0.0, 1.0, 0.0);
```

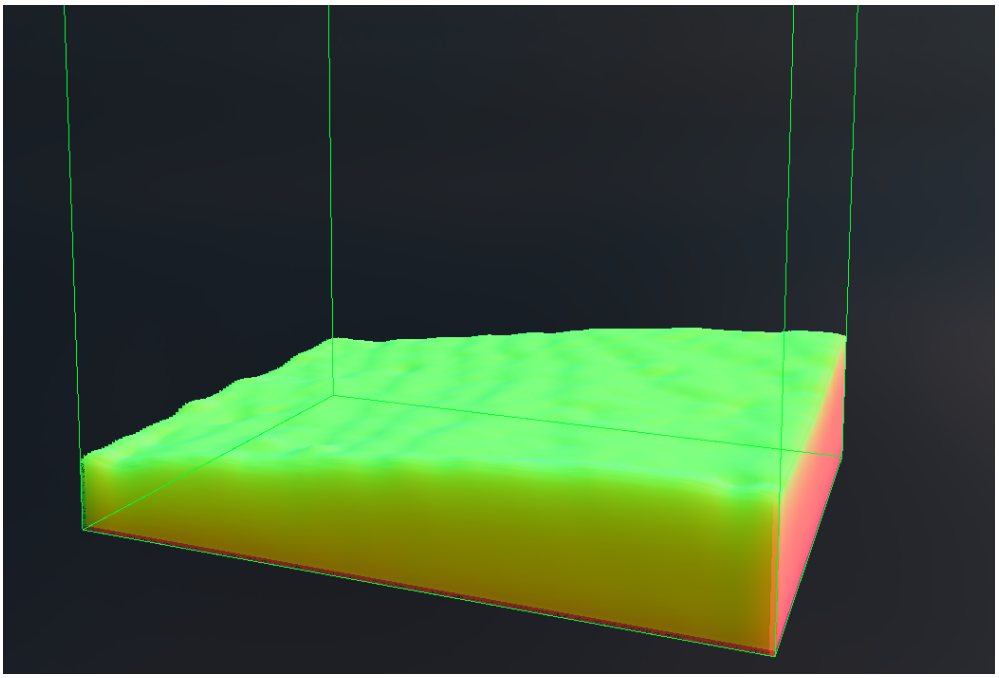
Listing 3.13: Fragment `raymarch.frag` — bisekcja i normalna

Rysunek 3.11 ilustruje wyznaczone normalne zakodowane jako kolor RGB. Osiem iteracji bisekcji zawęża punkt trafiania do dokładności $\Delta t/2^8 \approx 0{,}4\%$ kroku bez zmniejszania globalnego `STEP_SIZE`. Normalna wyznaczana jest sześcioma próbkami pola gęstości metodą różnic skończonych (`eps = 0.018`); warunek `dot(N,N) < 0.5` zastępuje normalną wektorem pionowym w miejscach, gdzie gradient zanika.

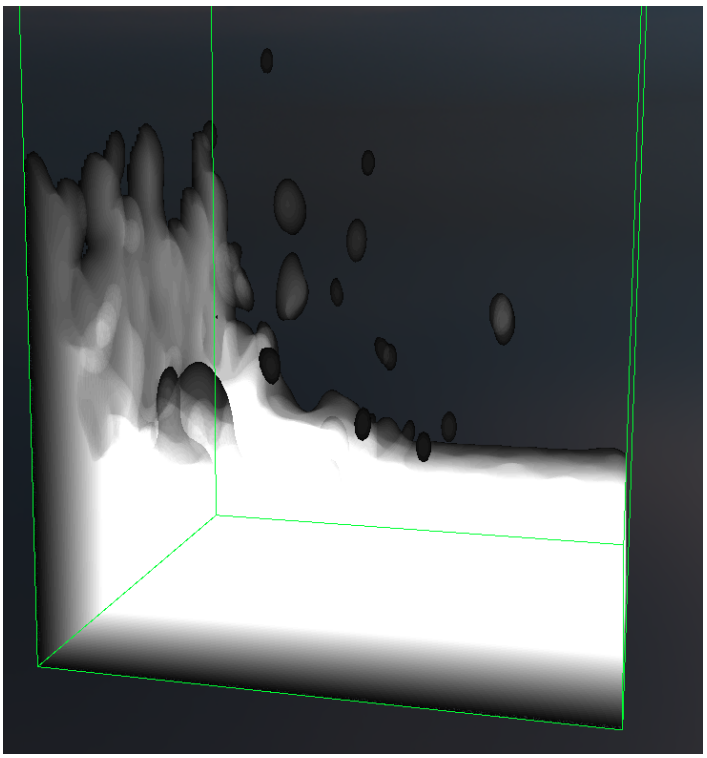
Interpolacja trójliniowa. Próbkowanie tekstury `R32Uint` realizowane jest ręcznie, ponieważ format całkowitoliczbowy nie obsługuje sprzętowej interpolacji:

```
vec3 gridPos = uvw * vec3(sim_params.grid_res.xyz) - 0.5;
ivec3 iobase = ivec3(floor(gridPos));
vec3  f      = fract(gridPos);
// 8 wywolan texelFetch dla wierzcholkow kostki (v000..v111)
float v0 = mix(mix(v000, v100, f.x), mix(v010, v110, f.x), f.y);
float v1 = mix(mix(v001, v101, f.x), mix(v011, v111, f.x), f.y);
return max(0.0, (mix(v0, v1, f.z) - DENSITY_OFFSET)
                    * DENSITY_MULTIPLIER);
```

Listing 3.14: Funkcja `getDensity` — interpolacja trójliniowa



Rysunek 3.11: Wizualizacja normalnych powierzchni cieczy zakodowanych jako kolor RGB. Źródło: opracowanie własne



Rysunek 3.12: Wizualizacja grubości cieczy wzdłuż promienia jako skala szarości. Źródło: opracowanie własne

Na rysunku 3.12 przedstawiono grubość cieczy jako skalę szarości — ciemniejsze obszary odpowiadają cienkim warstwom przy brzegach, jaśniejsze głębokim kolumnom wewnątrz bryły, co bezpośrednio steruje absorpcją Beer-Lamberta. Odjęcie `DENSITY_OFFSET = 300` zeruje szum tła; `max(0.0, ...)` zapobiega ujemnym wartościom gęstości w wokselach powietrza.

3.11.3 Model oświetlenia

Cały model optyczny zamknięty jest w funkcji `main()` shadera `raymarch.frag` i wykonywany po znalezieniu punktu powierzchni.

Fresnel, odbicie i załamanie. Współczynnik odbicia F_0 wyznaczany jest ze wzoru Fresnela (równanie 2.12), a promienie odbitego i załamanego kierunku próbkują mapę środowiskową:

```
const float IOR_WATER = 1.333;
const float IOR_AIR   = 1.0;

float F0 = pow((IOR_AIR - IOR_WATER) / (IOR_AIR + IOR_WATER), 2.0);
float F  = fresnel(NdotV, F0);  // przyblizenie Schlicka

vec3 reflDir    = reflect(rayDir, N);
vec3 reflection = sampleSkybox(reflDir);
reflection += SUN_COLOR * clamp(ggxD(NdotH, 0.04)
                * NdotL * 0.15, 0.0, 3.0);

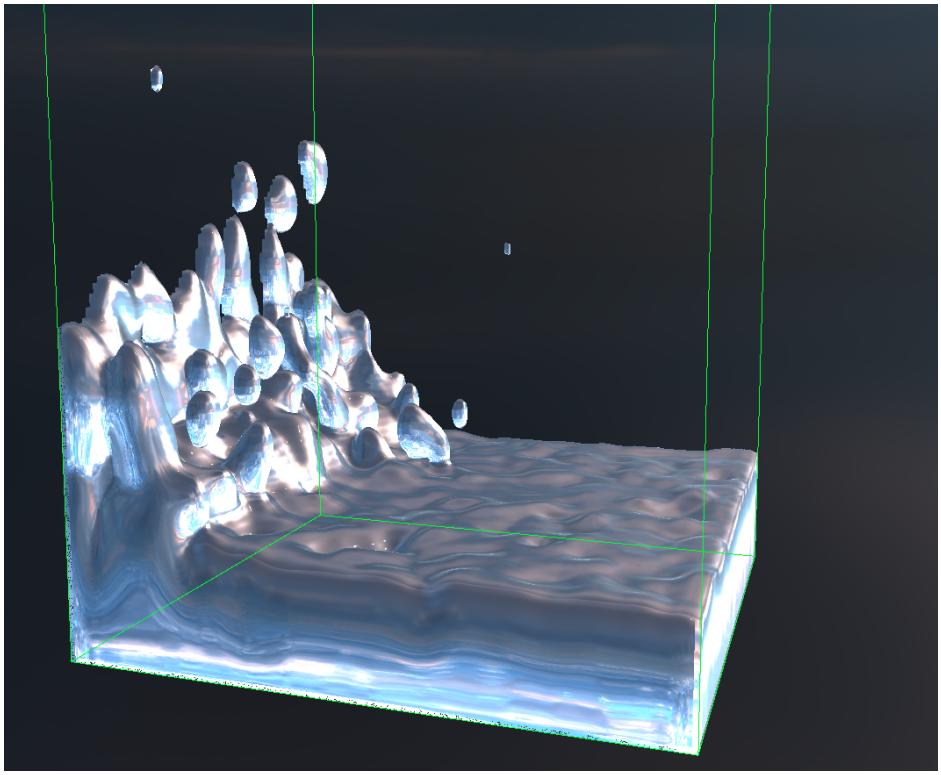
vec3 refrDir    = refract(rayDir, N, IOR_AIR / IOR_WATER);
if (dot(refrDir, refrDir) < 0.01) refrDir = rayDir;
vec3 background = sampleSkybox(normalize(refrDir));
```

Listing 3.15: Fragment `raymarch.frag` — Fresnel, odbicie, zalamania

Rysunek 3.13 pokazuje składnik odbicia bez refrakcji. Wartość $F_0 \approx 0{,}02$ obliczana jest ze wzoru Fresnela dla granicy woda–powietrze; przy kącie ślizgowym wzrasta do jedności zgodnie z przybliżeniem Schlicka. W przypadku całkowitego wewnętrznego odbicia `refract` zwraca wektor zerowy; kierunek zastępowany jest wówczas oryginalnym promieniem.

Absorpcja, SSS i piana. Grubość cieczy wzdłuż promienia steruje absorpcją Beer-Lamberta, rozproszeniem podpowierzchniowym i maską piany; wynik składany jest operatorem Reinharda do zakresu [0, 1]:

```
const vec3 ABSORPTION_COEFF = vec3(0.45, 0.085, 0.025);
const vec3 SCATTER_COLOR    = vec3(0.04, 0.18,  0.28);
```

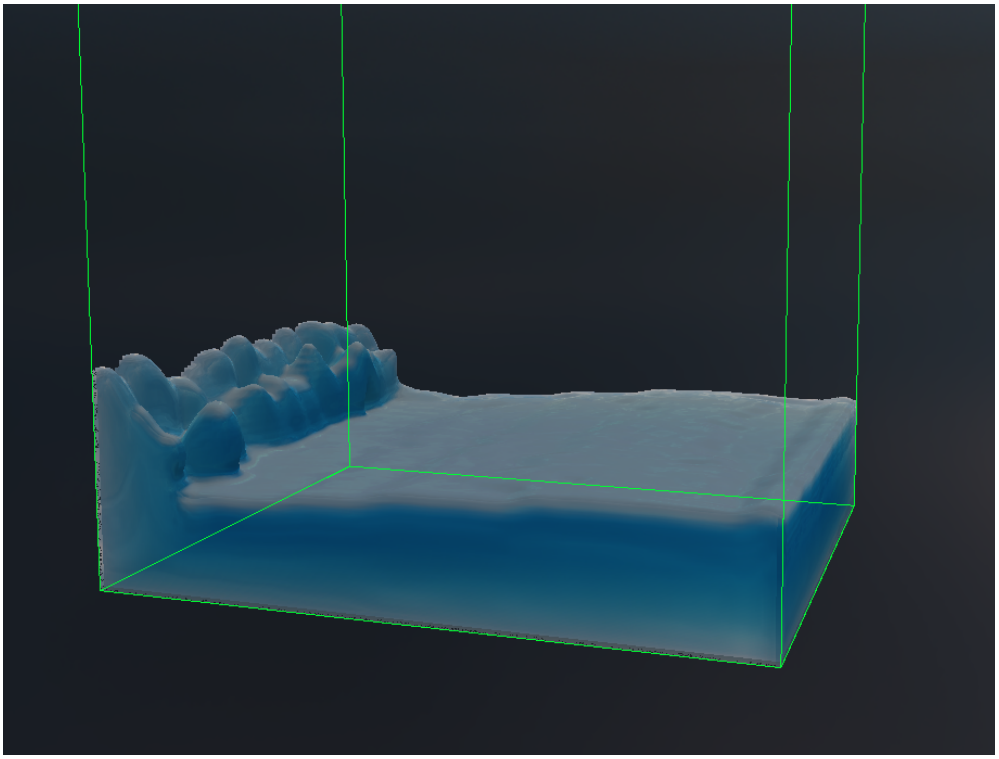


Rysunek 3.13: Wizualizacja składnika odbicia zwierciadlanego powierzchni cieczy. Źródło: opracowanie własne

```
float thickness  = calcThickness(
    surfacePos,
    rayDir,
    tHit.y - t1, ...);
vec3  absorption = exp(-ABSORPTION_COEFF * thickness * 8.0);
float sssWrapped = max(0.0, dot(N,  L) * 0.5 + 0.5);
float sssBack    = max(0.0, dot(-N, L));
float sssFactor  = (sssWrapped * 0.6 + sssBack * 0.4)
                 * (1.0 - exp(-thickness * 3.0));
vec3 sss = SCATTER_COLOR * SUN_COLOR * sssFactor * SCATTER_STRENGTH;
float foamMask = smoothstep(0.0, 0.3, thickness)
               * smoothstep(0.5, 0.95, N.y);
vec3 refracted  = background * absorption + sss;
vec3 waterColor = mix(refracted, reflection, F);
waterColor = mix(
    waterColor,
    FOAM_COLOR * (NdotL * 0.7 + 0.3),
    foamMask * 0.7);
waterColor = waterColor / (waterColor + vec3(1.0));
```

Listing 3.16: Fragment `raymarch.frag` — absorpcja, SSS, piana, tonowanie

Rysunek 3.14 prezentuje składnik refrakcji z absorpcją i SSS. Grubość `thickness`



Rysunek 3.14: Wizualizacja składnika refrakcji z absorpcją i rozproszeniem podpowierzchniowym (SSS). Źródło: opracowanie własne

mierzona jest osobnym przelotem 64 próbek od punktu powierzchni do tylnej ściany AABB (`calcThickness`). Silna absorpcja kanału czerwonego ($\mu_R = 0{,}45$) przy słabej niebieskiego ($\mu_B = 0{,}025$) daje wodzie charakterystyczną niebieskozieloną barwę. Maska piany aktywuje się tam, gdzie normalna jest prawie pionowa (`N.y` bliskie 1) i ciecz jest cienka — naśladując bąble na grzbietach fali. Ostatnia linia to operator Reinharda sprowadzający zakres HDR do [0, 1].

3.11.4 Środowisko HDRI

Otoczenie sceny reprezentuje sferyczna mapa w formacie równokątnym ładowana z pliku `.exr` przy starcie aplikacji. Funkcja `sampleSkybox` przelicza dowolny kierunek przestrzenny na współrzędne UV tej tekstury:

```
vec3 sampleSkybox(vec3 dir) {
    dir = normalize(dir);
    vec2 uv = vec2(atan(dir.z, dir.x) * 0.15915 + 0.5,
                   asin(clamp(dir.y, -1.0, 1.0)) * 0.31831 + 0.5);
    return texture(skyboxTex, uv).rgb;
}
```

Listing 3.17: Funkcja `sampleSkybox` — odwzorowanie równokątne



Rysunek 3.15: Panorama środowiskowa HDRI użyta jako tło i źródło oświetlenia sceny. Źródło: opracowanie własne

Użyta panorama HDRI przedstawiona jest na rysunku 3.15. Stałe $\frac{1}{2\pi} \approx 0{,}15915$ i $\frac{1}{\pi} \approx 0{,}31831$ normalizują kąty azymutowy i elewacji do przedziału [0, 1]. Ta sama funkcja wywoływana jest zarówno dla promienia odbitego, jak i załamanego, zapewniając spójność oświetlenia między powierzchnią cieczy a tłem. Mapa środowiskowa renderowana jest osobnym shaderem `sky.frag` na sferze o promieniu 500 ze odwróconymi normalami, tak by kamera zawsze znajdowała się w jej środku.

Podsumowanie

Celem niniejszej pracy było zaprojektowanie i zaimplementowanie interaktywnego silnika symulacji cieczy, wykonującego pełny potok obliczeniowy — od dynamiki cząstek po gotowy obraz — w czasie rzeczywistym na procesorze graficznym. Punkt wyjścia stanowiła analiza eulerowskiego i lagrange'owskiego opisu ruchu płynów oraz matematyczne sformułowanie metody SPH wraz z wariantem DFSPH, wymuszającym nieściśliwość cieczy przez dwuetapową korekcję gęstości i dywergencji. Tak zdefiniowane podstawy teoretyczne przełożono na implementację w języku Rust z wykorzystaniem API Vulkan, a każdą istotną decyzję projektową uzasadniono analizą strukturalną lub bezpośrednim pomiarem.

W ramach pracy zrealizowano następujące zadania:

- dokonano przeglądu istniejących rozwiązań symulacji cieczy (Blender, Unreal Engine 5, SPlisHSPlasH) i wyznaczono na ich tle miejsce własnego systemu: czysto cząsteczkowej implementacji DFSPH wykonywanej w całości na GPU;
- zaimplementowano kompletny solver DFSPH jako zestaw jedenastu potoków obliczeniowych, obejmujących wyznaczanie gęstości i współczynnika α, iteracyjne pętle korekcji ciśnienia oraz całkowanie ruchu cząstek;
- opracowano wyszukiwanie sąsiedztwa oparte na haszowaniu przestrzennym z reorganizacją buforów, zapewniające stały czas zapytania i skoalescowane odczyty pamięci GPU;
- zaimplementowano i porównano ilościowo dwa algorytmy sortowania na GPU — sortowanie bitoniczne i pozycyjne — wykazując niemal sześciokrotną przewagę tego drugiego w pełnym potoku symulacji;
- zaprojektowano organizację danych SOA z podwójnym buforowaniem pozycji i prędkości, strukturalnie eliminującą wyścigi danych między wątkami GPU;
- zbudowano wolumetryczny potok wizualizacji: splatting gęstości do tekstury trójwymiarowej, przemarsz promienia z bisekcją oraz model optyczny wody łączący

efekt Fresnela, absorpcję Beer-Lamberta, rozproszenie podpowierzchniowe, odbłysk GGX i oświetlenie panoramą HDRI;

- przeprowadzono analizę zbieżności obu solverów oraz eksperymenty z adaptacyjnym krokiem czasowym według warunku CFL, dokumentując źródła ich ograniczeń;

- udostępniono interaktywny panel konfiguracji umożliwiający modyfikację parametrów fizycznych i wizualnych w trakcie działania symulacji.

Wynikiem pracy jest działający symulator zdolny do stabilnej animacji dziesiątek tysięcy cząstek w czasie rzeczywistym, wraz z zestawem narzędzi diagnostycznych i pomiarowych. Przeprowadzone testy ujawniły zarazem najważniejsze ograniczenie przyjętego modelu: brak właściwej obsługi brzegu powoduje trwały deficyt gęstości przy swobodnej powierzchni, który blokuje kryterium zbieżności solvera i uniemożliwia bezpieczne stosowanie adaptacyjnego kroku czasowego (podrozdz. 3.9.5). Świadomy powrót do stałego Δt i stałej liczby iteracji stanowi udokumentowany kompromis między stabilnością a wydajnością.

Niniejsza praca koncentrowała się przede wszystkim na warstwie fizycznej symulacji; ta orientacja wyznacza naturalne kierunki dalszego rozwoju systemu.

- **Uzupełnienie modelu fizycznego.** Pierwszym krokiem powinna być obsługa brzegu za pomocą cząstek-duchów lub stałych warstw cząstek brzegowych [44], która usunęłaby deficyt jądra przy powierzchni i odblokowała zarówno kryterium ε-zbieżności, jak i adaptacyjny warunek CFL. Kolejnym naturalnym rozszerzeniem jest implementacja napięcia powierzchniowego, otwierająca drogę do zjawisk kapilarnych, formowania kropel i realistycznego zachowania cienkich warstw cieczy.

- **Rozbudowa warstwy wizualizacji.** Ponieważ wizualizacja pełniła w pracy rolę służebną wobec fizyki, pozostaje tu szerokie pole eksperymentów. Obejmują one jawną rekonstrukcję powierzchni metodą maszerujących sześcianów (ang. *marching cubes*), techniki renderowania w przestrzeni ekranu oraz cząstki wtórne piany i rozbryzgów. Dodatkowo, wdrożenie sprzętowego śledzenia promieni poprzez rozszerzenie `VK_KHR_ray_tracing_pipeline` pozwoliłoby na uzyskanie fizycznie poprawnych kaustyk i wielokrotnych odbić światła w objętości cieczy.

- **Techniki optymalizacji.** Obiecującym kierunkiem są skompresowane listy sąsiedztwa [3], których pełna realizacja na GPU pozostaje otwartym problemem

badawczym, a ponadto adaptacyjna rozdzielczość cząstek zagęszczająca symulację wyłącznie w obszarach istotnych wizualnie oraz głębsze wykorzystanie pamięci dzielonej i operacji podgrupowych (ang. *subgroup operations*) w shaderach solvera.

- **Integracja z ekosystemem.** Silnik może zostać przekształcony w bibliotekę wielokrotnego użytku lub rozwinięty w stronę pełnoprawnego silnika aplikacji interaktywnych; alternatywą jest opakowanie go we wtyczkę do istniejącego środowiska, analogicznie do roli, jaką NiagaraFluids pełni w Unreal Engine 5. Krokiem poprzedzającym byłoby usunięcie zależności od platformy Windows i udostępnienie wersji dla systemów Linux i macOS.
- **Metody oparte na danych.** Odrębną perspektywę otwiera częściowe lub całkowite zastąpienie obliczeń fizycznych modelami statystycznymi i sieciami neuronowymi: aproksymacja kroku solvera modelem nauczonym na danych z symulacji referencyjnych, generowanie detali wysokiej częstotliwości na podstawie zgrubnej symulacji o małej liczbie cząstek czy statystyczna analiza pól prędkości i gęstości pozwalająca przewidywać zachowanie cieczy bez jawnego rozwiązywania równań ruchu.

Przeprowadzone prace potwierdzają, że połączenie języka Rust i API Vulkan stanowi dojrzały fundament dla wysokowydajnych symulacji fizycznych: statyczne gwarancje kompilatora wyeliminowały całą klasę błędów zarządzania pamięcią, a jawna kontrola nad procesorem graficznym pozwoliła przenieść na niego kompletny potok obliczeniowy. Powstały system — wraz z opisanymi w pracy pomiarami, diagnozami i zidentyfikowanymi ograniczeniami — może służyć jako platforma odniesienia i punkt wyjścia dla wymienionych wyżej kierunków badań.

Spis rysunków

2.1 Interfejs symulacji FLIP w programie Blender z widocznym panelem konfiguracji domeny i podglądem cieczy. Źródło: opracowanie własne na podstawie dokumentacji [8] 23
2.2 Interfejs edytora Niagara w silniku Unreal Engine 5 z aktywną symulacją 3D FLIP zrealizowaną przy użyciu wtyczki NiagaraFluids. Źródło: opracowanie własne na podstawie dokumentacji [14] 24
2.3 Symulacja cieczy w bibliotece SPlisHSPlasH z aktywnym solverem DFSPH i panelem konfiguracji parametrów fizycznych. Źródło: opracowanie własne na podstawie dokumentacji biblioteki [5] 25
2.4 Hierarchia inicjalizacyjna Vulkana. Źródło: opracowanie własne na podstawie specyfikacji [47] 29
2.5 Porównanie ścieżek renderowania w Vulkanie. Źródło: opracowanie własne na podstawie specyfikacji [47] 32
2.6 Schemat zarządzania zasobami i powiązań deskryptorowych w Vulkanie. Źródło: opracowanie własne na podstawie specyfikacji [47] 34
2.7 Etapy potoku graficznego Vulkana. Etapy z przerywaną ramką są opcjonalne i mogą być pominięte przy tworzeniu potoku; etapy szare są stałofunkcyjne i konfigurowane strukturami opisowymi; etapy białe z ciągłą ramką są programowalne przez shadery SPIR-V. Źródło: opracowanie własne na podstawie specyfikacji [47] 35
2.8 Wartość $W(q)$ oraz pochodna radialna $\partial W/\partial q$ sześciennej funkcji sklejanej w funkcji znormalizowanej odległości $q = r/h$. Źródło: opracowanie własne na podstawie [30] 41
2.9 Wartość $W(q)$ oraz pochodna radialna $\partial W/\partial q$ jądra Wendlanda C2 w funkcji znormalizowanej odległości $q = r/h$. Źródło: opracowanie własne na podstawie [55] 42
2.10 Schemat wyszukiwania sąsiedztwa metodą siatki haszującej w 2D. Źródło: opracowanie własne na podstawie [22] 45

2.11 Schemat struktury CLL z kompaktową tablicą komórek. Źródło: opracowanie własne na podstawie [3] 45
2.12 Problem niepełnego sąsiedztwa przy granicy domeny. Źródło: opracowanie własne na podstawie [44] 48
2.13 Potok wizualizacji: od listy cząstek SPH do końcowego koloru piksela. Źródło: opracowanie własne 49
2.14 Schemat przemarszu promienia z binarnym zawężaniem. Białe kółka to próbki poniżej progu gęstości, szare kółko to pierwsza próbka powyżej progu (ciecz). Bisekcja przedziału $[t_4, t_5]$ wyznacza punkt powierzchni $\mathbf{p}_s$ z dokładnością $\Delta t/2^8$. Źródło: opracowanie własne na podstawie [42] 50

3.1 Okno aplikacji podczas symulacji. Źródło: opracowanie własne 56
3.2 Łańcuch obliczeń kamery. Źródło: opracowanie własne 61
3.3 Panel parametrów symulacji w interfejsie `egui`: (a) parametry symulacji, (b) zbieżność solvera DFSPH, siły zewnętrzne i granice kolizji, (c) parametry renderowania, algorytm sortowania i rozdzielczość tekstury. Źródło: opracowanie własne 62
3.4 Czterostopniowy potok wyszukiwania sąsiadów. Źródło: opracowanie własne 70
3.5 Sieć sortowania bitonicznego dla $N = 16$: dziesięć przebiegów pogrupowanych w cztery zewnętrzne etapy ($k = 1, 2, 3, 4$). Każdy wiersz odpowiada jednemu elementowi, każda pionowa kreska — komparatorowi obsługiwanemu przez osobny wątek GPU; zagnieżdżone prostokąty zaznaczają granice kolejnych etapów zewnętrznych [11] 72
3.6 Przepływ danych w jednym przebiegu sortowania pozycyjnego. Źródło: opracowanie własne na podstawie [23] 73
3.7 Czas sortowania bufora `grid_entries` jako funkcja liczby elementów N, w skali logarytmicznej na obu osiach. Źródło: opracowanie własne . 75
3.8 Histogram gęstości w konfiguracji spoczynkowej ($n = 4800$ cząstek, bez kroku symulacji). Źródło: opracowanie własne 83
3.9 Zbieżność solvera DFSPH: uśredniony błąd gęstości (a) i błąd dywergencji (b) w funkcji liczby iteracji Jacobiego. Źródło: opracowanie własne 84
3.10 Porównanie strategii statycznego i adaptacyjnego Δt w trakcie 500 klatek symulacji: krok czasowy (a), liczba iteracji solvera Jacobiego (b), maksymalna prędkość cząstki (c). Źródło: opracowanie własne 86
3.11 Wizualizacja normalnych powierzchni cieczy zakodowanych jako kolor RGB. Źródło: opracowanie własne 91

3.12 Wizualizacja grubości cieczy wzdłuż promienia jako skala szarości. Źródło: opracowanie własne 91
3.13 Wizualizacja składnika odbicia zwierciadlanego powierzchni cieczy. Źródło: opracowanie własne 93
3.14 Wizualizacja składnika refrakcji z absorpcją i rozproszeniem podpowierzchniowym (SSS). Źródło: opracowanie własne 94
3.15 Panorama środowiskowa HDRI użyta jako tło i źródło oświetlenia sceny. Źródło: opracowanie własne 95

Bibliografia

[1] Advanced Micro Devices, Inc. *Mantle Programming Guide and API Reference*. AMD, revision 1.0 edition, march 2015. Official API Specification.

[2] Ash contributors. Ash — a low-level Vulkan bindings generator for Rust. `https://docs.rs/ash/latest/ash/`. Dostęp: 2026.

[3] Stefan Band, Christoph Gissler, and Matthias Teschner. Compressed neighbour lists for SPH. *Computer Graphics Forum*, 39(1):206–220, 2019.

[4] George K. Batchelor. *An Introduction to Fluid Dynamics*. Cambridge University Press, 1967.

[5] Jan Bender et al. SPlisHSPlasH Library.

[6] Jan Bender and Dan Koschier. Divergence-free smoothed particle hydrodynamics. In *Proceedings of the 14th ACM SIGGRAPH/Eurographics Symposium on Computer Animation*, 08 2015.

[7] Blender Foundation. Blender. `https://www.blender.org`. Wersja 5.1, dostęp: 2026.

[8] Blender Foundation. Blender documentation. `https://docs.blender.org/manual/en/latest/physics/fluid/introduction.html#liquid-simulations`. Dostęp: 2026.

[9] J.U. Brackbill and H.M. Ruppel. Flip: A method for adaptively zoned, particle-in-cell calculations of fluid flows in two dimensions. *Journal of Computational Physics*, 65(2):314–343, 1986.

[10] Colin Braley and Adrian Sandu. Fluid simulation for computer graphics: A tutorial in grid based and particle based methods. *Virginia Tech, Blacksburg*, 3, 2010.

[11] Michael Christensen. Bitonic sort in PyRTL. `https://mdko.github.io/2021/06/17/pyrtl-bitonic-sort.html`, 2021. Dostęp: 2026.

[12] Bram De Greve. Reflections and refractions in ray tracing. *Reflections*, 1:1–23, 2006.

[13] egui_winit_vulkano contributors. egui_winit_vulkano — integration layer for egui, winit, and vulkano. `https://docs.rs/egui_winit_vulkano/latest/egui_winit_vulkano/`. Dostęp: 2026.

[14] Epic Games. Fluid simulation in Unreal Engine 5. `https://dev.epicgames.com/documentation/en-us/unreal-engine/fluid-simulation-in-unreal-engine`. Dostęp: 2026.

[15] Emil Ernerfeldt et al. egui — an easy-to-use immediate mode GUI in Rust. `https://docs.rs/egui/latest/egui/`. Dostęp: 2026.

[16] Leonhard Euler. Principes généraux du mouvement des fluides. *Mémoires de l'académie des sciences de Berlin*, pages 274–315, 1757.

[17] Leonhard Euler. Principia motus fluidorum. *Novi Commentarii Academiae Scientiarum Petropolitanae*, pages 271–311, 1761.

[18] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional, 1994.

[19] Robert A Gingold and Joseph J Monaghan. Smoothed particle hydrodynamics: theory and application to non-spherical stars. *Monthly notices of the Royal Astronomical Society*, 181(3):375–389, 1977.

[20] glam contributors. glam — a simple and fast linear algebra library for games and graphics. `https://docs.rs/glam/latest/glam/`. Dostęp: 2026.

[21] GPUOpen, AMD. Vulkan Memory Allocator. `https://gpuopen.com/vulkan-memory-allocator/`. Dostęp: 2026.

[22] Simon Green. Particle simulation using cuda. *NVIDIA whitepaper*, 6:121–128, 2010.

[23] Takahiro Harada and Lee Howes. Introduction to gpu radix sort. *Heterogeneous Computing with OpenCL. Morgan Kaufman*, 2011.

[24] Francis H Harlow and J Eddie Welch. Numerical calculation of time-dependent viscous incompressible flow of fluid with a free surface. *The physics of fluids*, 8(12):2182–2189, 1965.

[25] Megha Jain, Sanjay Kumar, and VK Patle. Bitonic sorting algorithm: A review. *International Journal of Computer Applications*, 113(13):40–43, 2015.

[26] Baldur Karlsson. RenderDoc — a stand-alone graphics debugger. `https://renderdoc.org/`. Dostęp: 2026.

[27] Arie E. Kaufman and Klaus Mueller. Overview of volume rendering. *The Visualization Handbook*, 7:127–174, 2005.

[28] Steve Klabnik, Carol Nichols, and the Rust Community. *The Rust Programming Language*. No Starch Press, 2nd edition, 2025. Accessed: 2026-05-09.

[29] Dan Koschier. CompactNSearch Library.

[30] Dan Koschier, Jan Bender, Barbara Solenthaler, and Matthias Teschner. Smoothed particle hydrodynamics techniques for the physics based simulation of fluids and solids. *arXiv preprint arXiv:2009.06944*, 2020.

[31] Joseph Louis Lagrange. *Mécanique analytique*. Desaint, 1788.

[32] Lev Davidovich Landau and Evgenii Mikhailovich Lifshitz. *Fluid Mechanics*, volume 6. Pergamon Press, 1987.

[33] Alexandre Lavrov, Paal Skjetne, Bjørnar Lund, Erik W. Bjønnes, Finn Olav Bjørnson, Jan Ove Busklein, Térence Coudert, Pascal Klebert, and Kjetil Olsen Lye. Density-consistent initialization of SPH on a regular Cartesian grid: Comparative numerical study of 10 smoothing kernels in 1, 2 and 3 dimensions. *Procedia IUTAM*, 18:85–95, 2015.

[34] Peter D. Lax. Stability of difference schemes. In Carlos A. de Moura and Carlos S. Kubrusly, editors, *The Courant–Friedrichs–Lewy (CFL) Condition: 80 Years After Its Discovery*, pages 1–7. Birkhäuser, Boston, MA, 2013.

[35] Marc Levoy. Volume rendering. *IEEE Computer graphics and Applications*, 8(3):29–37, 1988.

[36] Doug McLean. *Understanding Aerodynamics: Arguing from the Real Physics*. John Wiley & Sons, 2012.

[37] Joseph J. Monaghan. On the problem of penetration in particle methods. *Journal of Computational Physics*, 82(1):1–15, 1989.

[38] Joseph J. Monaghan. Smoothed particle hydrodynamics. *Annual Review of Astronomy and Astrophysics*, 30(1):543–574, 1992.

[39] Joseph P. Morris, Patrick J. Fox, and Yi Zhu. Modeling low Reynolds number incompressible flows using SPH. *Journal of Computational Physics*, 136(1):214–226, 1997.

[40] Matt Pharr, Wenzel Jakob, and Greg Humphreys. *Physically Based Rendering: From Theory to Implementation*. Morgan Kaufmann, 3rd edition, 2016.

[41] Inigo Quilez. Raymarching distance fields. *inigoquilez.org*, 2008.

[42] Inigo Quilez. Advancing the visualization of distance fields. *inigoquilez.org*, 2013.

[43] Erik Reinhard, Michael Stark, Peter Shirley, and James Ferwerda. Photographic tone reproduction for digital images. In *Proceedings of the 29th Annual Conference on Computer Graphics and Interactive Techniques*, pages 267–276, 2002.

[44] Hagit Schechter and Robert Bridson. Ghost sph for animating water. *ACM Transactions on Graphics - TOG*, 31, 07 2012.

[45] simple_logger contributors. simple_logger — a logger that prints all messages with a readable output format. `https://docs.rs/simple_logger/latest/simple_logger/`. Dostęp: 2026.

[46] Matthias Teschner, Bruno Heidelberger, Matthias Müller, Danat Pomerantes, and Markus H Gross. Optimized spatial hashing for collision detection of deformable objects. In *Vmv*, volume 3, pages 47–54, 2003.

[47] The Khronos Group. Vulkan specification. `https://registry.khronos.org/vulkan/specs/latest/html/vkspec.html`. Dostęp: 2026.

[48] The Rust Project Developers. log — a lightweight logging facade for Rust. `https://docs.rs/log/latest/log/`. Dostęp: 2026.

[49] Nils Thuerey and Tobias Pfaff. MantaFlow, 2018. *http://mantaflow.com*.

[50] David Tolnay. anyhow — flexible concrete error type built on `std::error::Error`. `https://docs.rs/anyhow/latest/anyhow/`. Dostęp: 2026.

[51] David Tolnay. thiserror — derive macro for `std::error::Error`. `https://docs.rs/thiserror/latest/thiserror/`. Dostęp: 2026.

[52] Lukasz Jaroslaw Tomczak. Gpu ray marching of distance fields. *Technical University of Denmark*, 8:3, 2012.

[53] Vulkano contributors. Vulkano — safe and rich rust wrapper around the Vulkan api. `https://docs.rs/vulkano/latest/vulkano/`. Dostęp: 2026.

[54] Bruce Walter, Stephen R. Marschner, Hongsong Li, and Kenneth E. Torrance. Microfacet models for refraction through rough surfaces. In *Proceedings of the 18th Eurographics Conference on Rendering Techniques*, pages 195–206, 2007.

[55] Holger Wendland. Piecewise polynomial, positive definite and compactly supported radial functions of minimal degree. *Advances in computational Mathematics*, 4(1):389–396, 1995.

[56] winit contributors. winit — window handling library in pure Rust. `https://docs.rs/winit/latest/winit/`. Dostęp: 2026.

[57] Bartosz Wysocki. Tracy profiler — a real time, nanosecond resolution frame profiler. `https://github.com/wolfpld/tracy`. Dostęp: 2026.